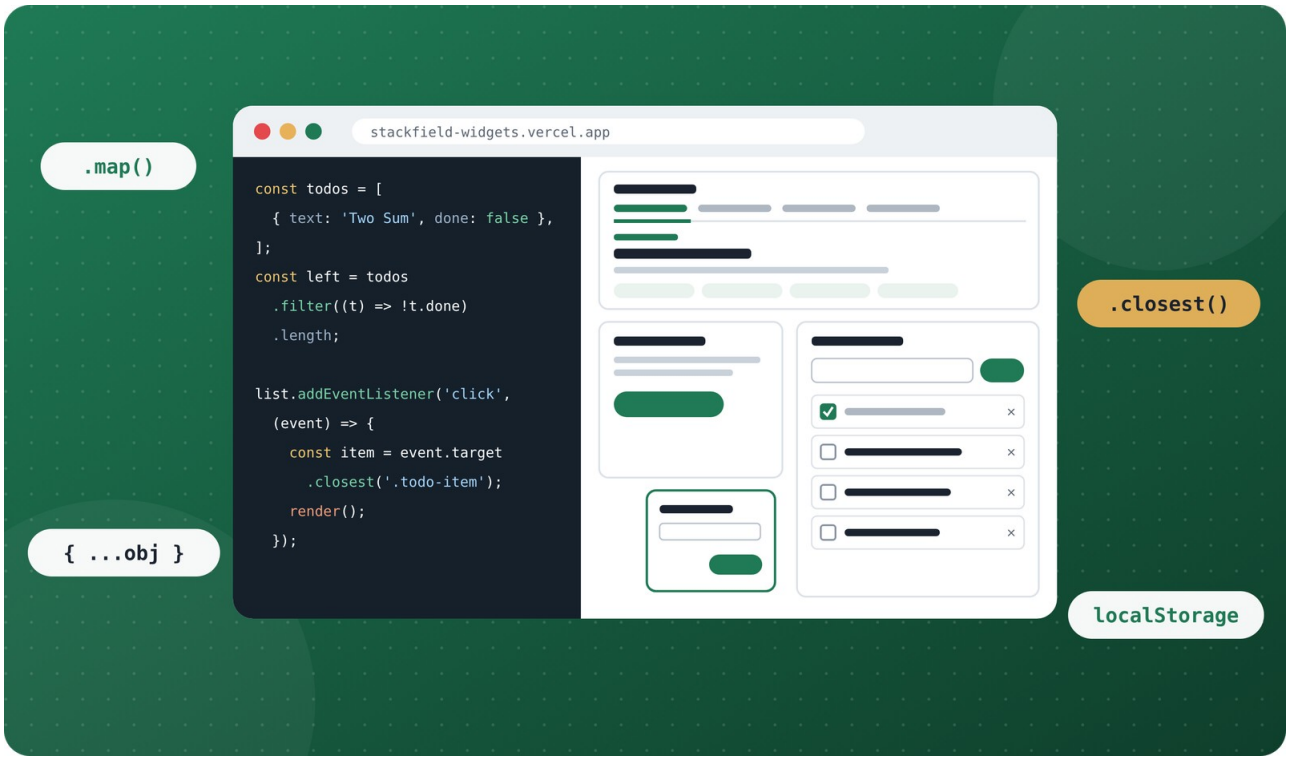


FULL-STACK DEVELOPER HANDBOOK

Week 3

Data, the DOM and Interactive Widgets

Arrays · Objects + JSON · The DOM · Events · Closures · localStorage · Modules

Week 3 of 26 · Days 15–21 · 10 – 16 October 2026

7 daily chapters	**39** diagrams and screenshots	**6** LeetCode problems	**3** working widgets

This book belongs to _____

START HERE

How to use this book

This handbook holds everything you need for Week 3, in the order you need it. Every lesson explains the idea in plain words first, then shows the code, exactly what it prints (every output in this book was run in Chrome), and other ways to do the same job. Read the day's chapter before you code, keep it open while you build, and use the review pages on Friday.

Every day follows the same shape

Part	What it gives you
Chapter opener	The date, time needed, what you build today and where the code is
Today you will	The skills you'll have by the end of the day
Key terms	The new words, defined in one line each
Lessons	Each idea explained in plain words, with worked examples, output, diagrams and tables
DSA problem	Days 15 to 20: one LeetCode problem a day, explained, then solved two to four ways
Build it, step by step	Exactly what to do today, in order
Must know	The few ideas that matter most. Revise these
Watch out	The mistakes almost everyone makes
More practice	Extra exercises for when you want more; answers at the back
Check yourself	Three questions; answers at the back
Resources	Hand-picked, free links for going deeper, checked before printing
Done when	The checklist that tells you the day is finished

The boxes you'll see

> IN PLAIN WORDS
>
> The idea explained with an everyday comparison, before any code.

JS · code you type

```
const city = 'Dhaka';
console.log(`Hello from ${city}`);
```

CONSOLE OUTPUT · what you should see

```
Hello from Dhaka
```

⇄ MORE WAYS **Other ways to do the same job**

WAY 1 **A different approach, with a note on when to choose it**

There is rarely only one right answer. Knowing two or three ways helps you read other people's code.

★ MUST KNOW *the ideas to remember from today*

✓ The most important ideas of the day. If you remember nothing else, remember these.

TIP

A shortcut, a better habit or a tool that makes the work easier.

WATCH OUT

A common mistake, and how to avoid it.

QUESTIONS LEARNERS ASK

Q **A question beginners often ask at this point?**

A short, direct answer.

How to study with it

1. **Read a lesson, then type the examples yourself.** Typing (not copying) is how syntax sticks. Predict the output before you run it.
2. **Change one thing and run it again.** What happens with a different number, an empty string, a narrower window? Most learning happens here.
3. **Try the task before opening the finished code.** Stuck for 20 minutes? Look at only the part you need, close it, then write it yourself.
4. **End each day with Check yourself and Done when.** Then commit, push and mark the day done in your Logbook.

WEEK 3

Contents

Week at a glance 5

The week in pictures 7

Day 15 — Arrays: lists of values, and the methods that transform them 9
Saturday 10 October

Day 16 — Objects and JSON: modelling real data 34
Sunday 11 October

Day 17 — The DOM: select, create and update the page 56
Monday 12 October

Day 18 — Events in depth: bubbling, delegation and an accessible modal 74
Tuesday 13 October

Day 19 — Closures, timers and localStorage 92
Wednesday 14 October

Day 20 — Build day: modules, the finished to-do list, and going live 110
Thursday 15 October

Day 21 — Week 3 review: theory stops here 126
Friday 16 October

Quick reference 133

Answers 138

Glossary 143

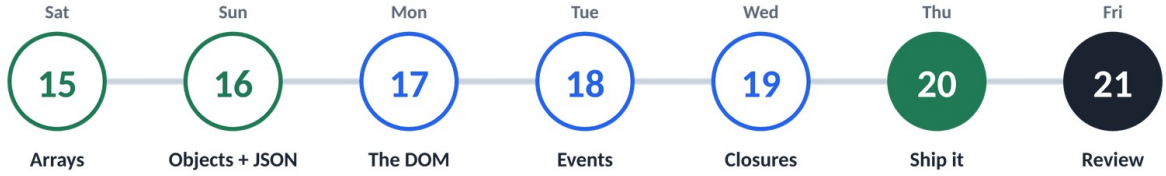
Notes 145

> **TIP**
>
> Every line above is a link. Click a chapter to jump straight to it, in Word and in the PDF. Links in the Resources tables open in your browser.

OVERVIEW

Week at a glance

The goal of Week 3: learn to work with **data** (arrays and objects) and to make pages **react** to people (the DOM and events), by building three widgets almost every web app has: tabs, a modal dialog and a to-do list that remembers your tasks. Plus six LeetCode problems, including the famous Two Sum.



Days 15–16: data (green) · Days 17–19: the page (blue) · Day 20 ships the widgets · Day 21: theory stops here

Day	Date	Topic	You build
15	10 October	Arrays	10 array exercises, the plan and static HTML/CSS for three widgets, and Running Sum
16	11 October	Objects + JSON	An objects demo, five expense functions, tabs built from data, and Richest Customer Wealth
17	12 October	The DOM	A DOM playground with five exercises, working tabs, and Contains Duplicate
18	13 October	Events	A character counter, a delegated list, an event lab, the modal dialog and keyboard tabs, and Valid Anagram
19	14 October	Closures	Closure exercises, a focus timer, a debounced search, a saved theme, the to-do list (part 1), and Two Sum
20	15 October	Ship it	The widgets as ES modules, a to-do list with filters, editing and saving, live on Vercel; and Best Time to Buy and Sell Stock
21	16 October	Review	A filterable list from scratch, one problem redone, and the Project 1 plan

By Friday you will be able to

- ✓ Store lists and records as arrays and objects, and reshape them with map, filter and reduce
- ✓ Copy and update data without surprises: spread, destructuring, references vs copies
- ✓ Save data as JSON in localStorage and load it back safely
- ✓ Find, create, change and remove any element on a page, safely (no XSS)
- ✓ Handle clicks, typing, forms and keys, with bubbling and event delegation
- ✓ Build accessible tabs, a modal dialog and a to-do list with the state → render pattern
- ✓ Explain closures, timers and debouncing
- ✓ Split code into ES modules and deploy it on Vercel

Your daily routine

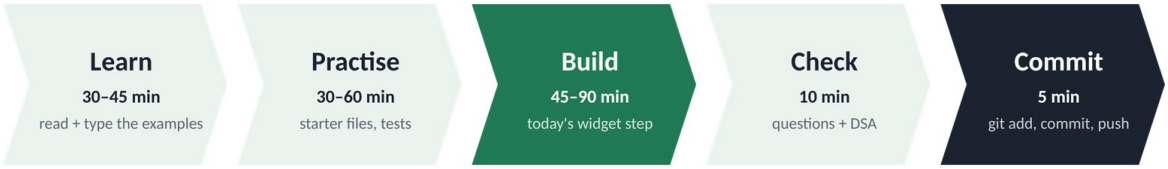


Figure 0.1 · The same five steps every day, about 3 hours in total

- **Before:** read the chapter opener, "Today you will" and the Key terms (5 minutes).
- **During:** read a lesson, type its examples in the Console, predict the output, then practise. Try each task before opening the finished code.
- **After:** answer "Check yourself" without looking, solve the day's LeetCode problem, tick "Done when", then commit and push.

What you need

Tool	Why
VS Code	With Live Server (essential from Day 20: modules need a server) and Prettier
Google Chrome	DevTools: Console, Elements (live DOM, `$0`), Sources (breakpoints), Application (Local Storage)
Git + GitHub	Commit every day; the widgets get their own repository on Day 20
Vercel	The same free account as Week 2, for the widgets page
LeetCode	One problem a day, Days 15–20
The code folder	`fullstack-journey-week03`: one finished folder per day, starter and solution files, and tests that print PASS / FAIL

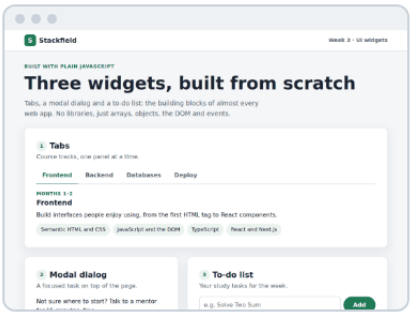
Last week → this week

From Week 2	How Week 3 builds on it
Functions + callbacks	Every array method takes a callback; closures are functions that remember
getElementById, textContent	The full DOM toolbox: select anything, create and remove elements
addEventListener	The event object, bubbling, delegation, keys, forms and a modal
Strings + numbers	Used inside map and filter; `toLocaleString` for taka amounts
The debugger	Breakpoints inside callbacks; the Application panel for saved data
Deploying on Vercel	The same flow, now for a site made of ES modules

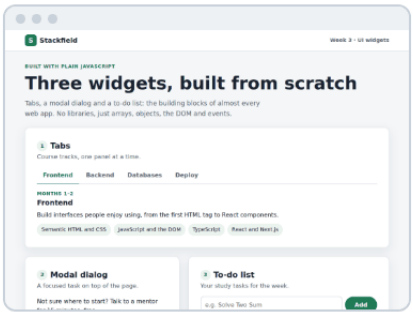
OVERVIEW

The week in pictures

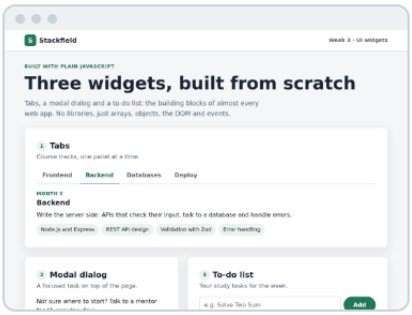
The widgets page grows every day. Each screenshot is the real code from that day's folder, so you can always see where you're heading.



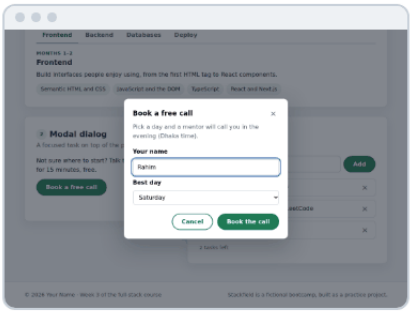
Day 15 static HTML and CSS



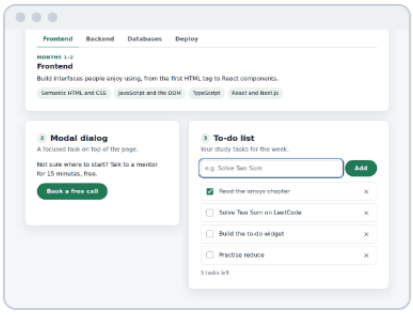
Day 16 tabs built from data



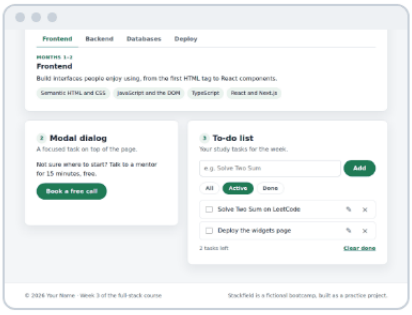
Day 17 tabs switch on click



Day 18 + an accessible modal



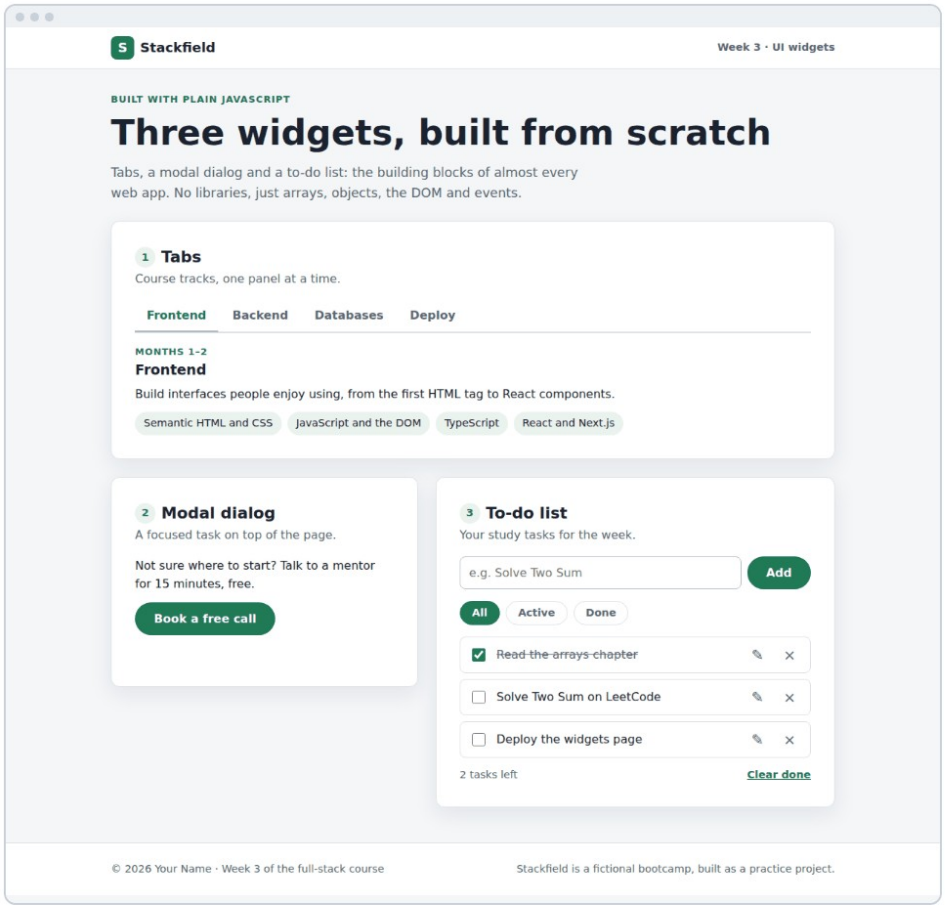
Day 19 + a to-do list (state + render)

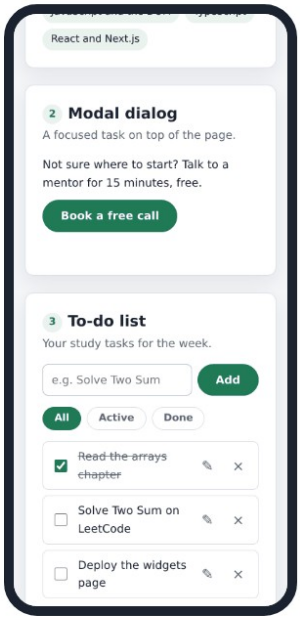


Day 20 filters, editing, saved: live

Figure 0.2 · The widgets page on Days 15 to 20

Day	What changes
15	The plan, then the HTML and CSS only: three widgets that look finished but do nothing yet
16	The tabs are built from an array of objects instead of hand-written HTML
17	Clicking a tab shows its panel
18	One delegated listener and the arrow keys for the tabs; the modal dialog works
19	The to-do list: add, tick and delete, drawn from an array by one render()
20	ES modules, filters, editing, empty states and saving; then it goes live on Vercel
21	Nothing new: you prove it stuck, and plan Project 1





Laptop the modal and to-do list side by side

Phone one column

Figure 0.3 · Where you're heading: the finished widgets on a laptop and a phone

> **TIP**
>
> Days 15 and 16 are mostly data work in the Console, so the page barely changes. That's normal. From Day 17 every day adds something you can click, and it all rests on those two days.

DAY

15

WEEK 3 · SATURDAY 10 OCTOBER

Arrays: lists of values, and the methods that transform them

Almost all data in an app is a list. Today you learn to store lists and reshape them.

TIME	YOU BUILD	CODE FOLDER
3 hours	**10 array exercises, the plan and static HTML/CSS for three widgets, and Running Sum**	`day15-arrays`

Today you will

- ✓ Create arrays, read items by their index, and use `length` and `at(-1)`
- ✓ Add and remove items with `push`, `pop`, `shift`, `unshift` and `splice`
- ✓ Tell `slice` (copy a part) from `splice` (cut out or insert)
- ✓ Know which methods change the original array and which give you a new one
- ✓ Loop with `for`, `for…of` and `forEach`
- ✓ Search with `includes`, `indexOf`, `find`, `findIndex`, `some` and `every`
- ✓ Transform lists with `map`, `filter` and `reduce`, and chain them
- ✓ Sort numbers, text and objects correctly with a compare function
- ✓ Remove duplicates with a `Set`, and copy or join arrays with spread (`...`)
- ✓ Plan the three widgets and write their static HTML and CSS
- ✓ Solve LeetCode 1480, Running Sum of 1d Array, three ways

Key terms

Term	Meaning
Array	An ordered list of values in one variable: `['Sat', 'Sun', 'Mon']`
Item (element)	One value inside an array
Index	An item's position, counted from **0**
length	How many items the array has
Method	A function that belongs to a value: `days.push('Tue')`
Mutate	Change the original array (instead of making a new one)
Callback	The function you give to `map`, `filter`, `find`…; it runs once per item
Predicate	A callback that answers yes or no (`true` / `false`) for each item
Accumulator	The running value that `reduce` carries from item to item
Compare function	`(a, b) => a - b`: tells `sort` which of two items comes first
Set	A collection that keeps only one of each value
Spread (`...`)	Unpacks an array's items: `[...a, ...b]`, `Math.max(...a)`

15.1 What an array is

IN PLAIN WORDS

An array is a row of numbered pigeonholes. Each hole holds one value, and the numbers start at **0**, not 1. The whole row has one name, so you can pass a list of 5 or 5,000 values around as easily as a single number.

```
const days = ['Sat', 'Sun', 'Mon', 'Tue'];
```

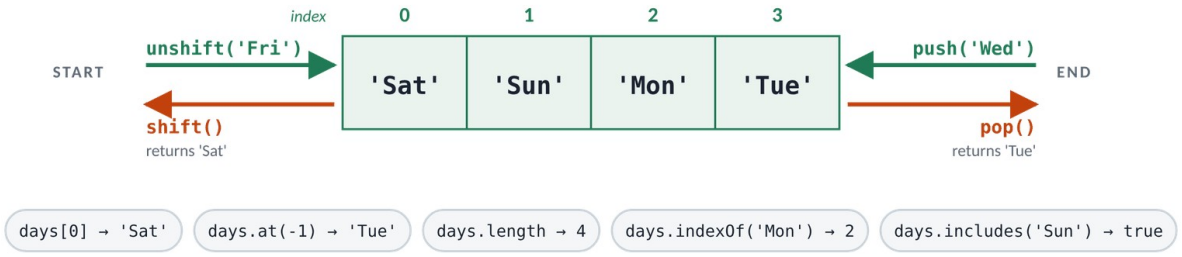


Figure 15.1 · An array: numbered positions, and the four methods that work on its two ends

JS · **creating an array and reading items**

```
const days = ['Sat', 'Sun', 'Mon'];   // square brackets, items separated by commas
console.log(days);
console.log(days[0]);                 // the FIRST item is at index 0
console.log(days[2]);
console.log(days.length);             // how many items
console.log(days[days.length - 1]);   // the last item, the classic way
console.log(days.at(-1));             // the last item, the modern way
console.log(days[5]);                 // nothing there: undefined, not an error
```

▶ CONSOLE OUTPUT

```
['Sat', 'Sun', 'Mon']
Sat
Mon
3
Mon
Mon
undefined
```

TIP

Chrome shows an array with its length in front, like `(3) ['Sat', 'Sun', 'Mon']`, and a small triangle (▶) you can click to open it. This book leaves out the `(3)` to keep the outputs short. Inside a bigger value Chrome shortens nested arrays to `Array(3)` and objects to `{…}`; click to open them.

The last index is always `length - 1`: an array of 3 items has indexes 0, 1 and 2. Forgetting this "off by one" is the most common array bug of all.

JS · any value can go in an array, even another array

```
const mixed = ['Rahim', 25, true, null];   // any types, even mixed
const grid = [[1, 2, 3], [4, 5, 6]];        // arrays inside an array: a table
console.log(mixed.length, grid.length);
console.log(grid[1]);                       // the second row
console.log(grid[1][2]);                    // row 1, then item 2 of that row
console.log(Array.isArray(grid), typeof grid);
```

CONSOLE OUTPUT

```
4 2
[4, 5, 6]
6
true object
```

Changing an item

JS · writing to an index

```
const colours = ['red', 'green', 'blue'];
colours[1] = 'yellow';     // replace the item at index 1
colours[3] = 'purple';     // the next free index adds an item at the end
console.log(colours);
console.log(colours.length);
```

CONSOLE OUTPUT

```
['red', 'yellow', 'blue', 'purple']
4
```

WATCH OUT

`const` does **not** freeze an array. `const colours` means the name `colours` always points to the same array, but the items inside can change. `colours[0] = 'pink'` works; `colours = ['pink']` is a TypeError.

⇄ MORE WAYS **Five ways to create an array**

WAY 1 Square brackets: the usual way

```
const empty = [];
const primes = [2, 3, 5, 7];
```

Use this 95% of the time.

WAY 2 Split a string

```
'html,css,js'.split(',')          // ['html', 'css', 'js']
```

Handy for text from a file, a form or a URL.

WAY 3 Array.from()

```
Array.from('abc')                    // ['a', 'b', 'c']
Array.from({ length: 5 }, (_, i) => i + 1)   // [1, 2, 3, 4, 5]
```

Turns "list-like" things into real arrays, and builds number ranges. `_` is a name for a parameter you don't use.

WAY 4 Array(n).fill()

```
Array(3).fill(0)                     // [0, 0, 0]
```

A list of the same starting value, e.g. scores for 3 players.

WAY 5 Spread something else

```
[...'hey']                           // ['h', 'e', 'y']
[...new Set([1, 1, 2])]              // [1, 2]
```

`...` unpacks anything you can loop over into a new array (15.12).

15.2 Adding and removing at the ends

IN PLAIN WORDS

Picture a queue at a bus stop. `push` adds a person at the back and `pop` removes the person at the back; `unshift` squeezes someone in at the front and `shift` lets the person at the front leave. All four **change the queue itself**.

JS · push, unshift, pop and shift

```
const queue = ['Asha', 'Bilal'];
queue.push('Chandni');            // add to the END
console.log(queue);
queue.unshift('Dipu');            // add to the START
console.log(queue);
const lastOut = queue.pop();      // remove from the END, and get it back
const firstOut = queue.shift();   // remove from the START, and get it back
console.log(lastOut, firstOut);
console.log(queue);
console.log(queue.push('Esha', 'Farid'));  // adds two; returns the new length
```

▶ CONSOLE OUTPUT

```
['Asha', 'Bilal', 'Chandni']
['Dipu', 'Asha', 'Bilal', 'Chandni']
Chandni Dipu
['Asha', 'Bilal']
4
```

Method	Where	Returns	Example
`push(x, …)`	adds at the end	the new length	`cart.push('pen')`
`pop()`	removes the last	the removed item	`const last = stack.pop()`
`unshift(x, …)`	adds at the start	the new length	`news.unshift(latest)`
`shift()`	removes the first	the removed item	`const next = queue.shift()`

⇄ MORE WAYS Two classic shapes built from these methods

WAY 1 A stack: last in, first out

```
const history = [];
history.push('page 1');
history.push('page 2');
history.pop();          // 'page 2': the newest leaves first
```

The browser's Back button and every Undo button work like this. You'll use a stack for LeetCode 20 in Week 4.

WAY 2 A queue: first in, first out

```
const printJobs = [];
printJobs.push('CV.pdf');
printJobs.push('notes.pdf');
printJobs.shift();      // 'CV.pdf': the oldest leaves first
```

Print queues, customer-support tickets, messages waiting to be sent.

TIP

`push` and `pop` are fast. `shift` and `unshift` must renumber every other item, so on arrays with hundreds of thousands of items they are slow. For the lists in this course, both are fine.

15.3 slice vs splice: copy a part, or cut it out

Two methods with nearly the same name and very different effects. `slice` **copies** a part of the array and leaves the original alone. `splice` **cuts** items out of the original (and can insert new ones in the gap).

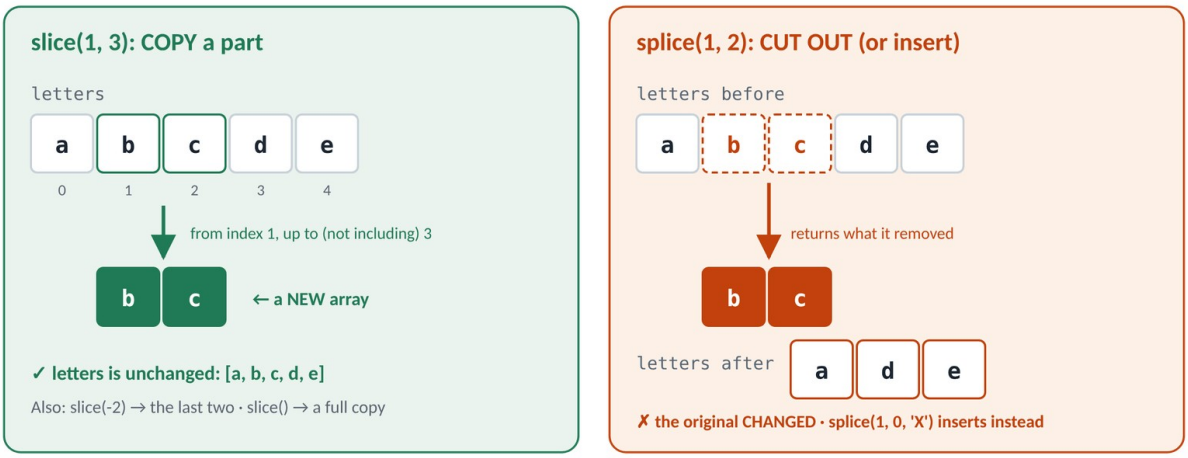


Figure 15.2 · slice(1, 3) copies b and c; splice(1, 2) removes them from the original

JS · slice copies; splice changes the original

```
const letters = ['a', 'b', 'c', 'd', 'e'];
console.log(letters.slice(1, 3));   // from index 1 up to (not including) 3
console.log(letters.slice(2));      // from index 2 to the end
console.log(letters.slice(-2));     // the last two
console.log(letters);               // unchanged

const removed = letters.splice(1, 2);   // at index 1, remove 2 items
console.log(removed);
console.log(letters);               // CHANGED
letters.splice(1, 0, 'X', 'Y');     // at index 1, remove 0, insert 'X' and 'Y'
console.log(letters);
letters.splice(2, 1, 'Z');          // at index 2, replace 1 item with 'Z'
console.log(letters);
```

CONSOLE OUTPUT

```
['b', 'c']
['c', 'd', 'e']
['d', 'e']
['a', 'b', 'c', 'd', 'e']
['b', 'c']
['a', 'd', 'e']
['a', 'X', 'Y', 'd', 'e']
['a', 'X', 'Z', 'd', 'e']
```

Call	Means	Changes the original?
`slice(start, end)`	copy from start up to (not incl.) end	No: returns a new array
`slice(start)` / `slice(-n)`	copy to the end / the last n	No
`slice()`	copy everything	No: a quick way to copy
`splice(start, count)`	remove count items at start	**Yes**: returns the removed items
`splice(start, 0, a, b)`	insert a and b at start	**Yes**
`splice(start, 1, x)`	replace one item with x	**Yes**

TIP

Memory trick: s**p**lice has an extra **p**, and it **p**ermanently changes the array.

⇄ MORE WAYS **Four ways to remove the item at index 2**

WAY 1 splice: change the array

```
list.splice(2, 1);
```

Fine for a list only you use. The original is changed.

WAY 2 filter by index: a new array

```
const shorter = list.filter((_, index) => index !== 2);
```

Keeps every item whose index isn't 2. The original is safe. You'll learn `filter` in 15.8.

WAY 3 Two slices and spread

```
const shorter = [...list.slice(0, 2), ...list.slice(3)];
```

Everything before, then everything after. Also a new array.

WAY 4 toSpliced(): splice without the damage

```
const shorter = list.toSpliced(2, 1);
```

Added to JavaScript in 2023 and works in all current browsers. Same arguments as `splice`, but it returns a new array.

15.4 Change the original, or get a new array?

Every array method either **mutates** (changes the array you call it on) or **returns a new array** and leaves the original alone. Knowing which is which prevents a whole family of bugs, and from Month 2 React will insist on the "new array" style.

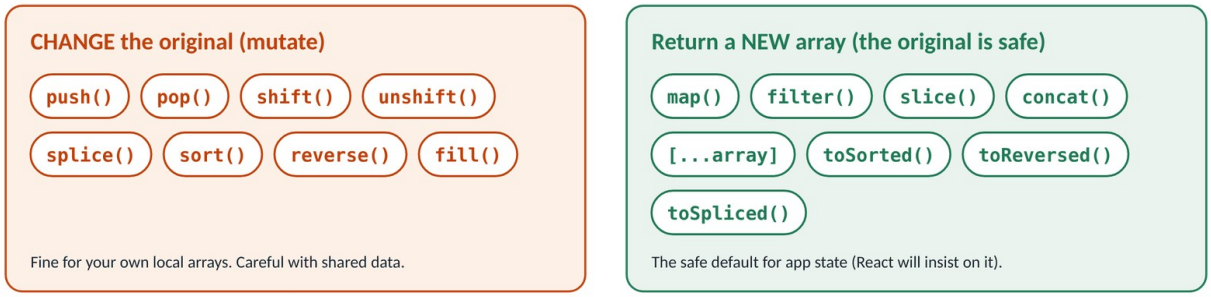


Figure 15.3 · Methods that change the original (left) and methods that give you a new array (right)

JS · = gives a second name; spread gives a copy

```
const original = [3, 1, 2];
const sameArray = original;        // NOT a copy: a second name for the same array
sameArray.push(4);
console.log(original);             // changed through the other name!

const copy = [...original];        // a real copy, made with spread
copy.push(5);
console.log(original, copy);
console.log(original === sameArray, original === copy);
```

▶ CONSOLE OUTPUT

```
[3, 1, 2, 4]
[3, 1, 2, 4] [3, 1, 2, 4, 5]
true false
```

IN PLAIN WORDS

A variable holding an array is like a note with a house address on it. `const sameArray = original` copies the **note**, not the house: both notes lead to the same house, so a change made through one is visible through the other. `[...original]` builds a new house with the same furniture.

⇄ MORE WAYS Four ways to copy an array

WAY 1 Spread: [...list]

```
const copy = [...list];
```

Short and clear. The most common way today.

WAY 2 slice() with no arguments

```
const copy = list.slice();
```

The classic way; you'll see it in older code.

WAY 3 Array.from(list)

```
const copy = Array.from(list);
```

Also converts list-like things (such as a NodeList, Day 17).

WAY 4 structuredClone(list): a deep copy

```
const copy = structuredClone(list);
```

Also copies the arrays and objects **inside** the array. The first three only copy the outer list (Day 16 explains "shallow" and "deep").

Mutating method	Safe twin (new array)	Notes
`sort()`	`toSorted()` or `[...a].sort()`	The twins arrived in 2023
`reverse()`	`toReversed()` or `[...a].reverse()`	
`splice()`	`toSpliced()`	Same arguments
`a[i] = x`	`a.with(i, x)`	Replace one item, get a new array
`push(x)`	`[...a, x]`	Add to the end
`unshift(x)`	`[x, ...a]`	Add to the start

15.5 Looping over an array

JS · four loops, same list

```
const topics = ['Arrays', 'Objects', 'DOM'];

for (let i = 0; i < topics.length; i++) {     // 1. classic for: you control the index
  console.log(i, topics[i]);
}
for (const topic of topics) {                    // 2. for...of: just the items
  console.log(topic);
}
topics.forEach((topic, index) => {               // 3. forEach: item AND index
  console.log(`${index + 1}. ${topic}`);
});
for (const [index, topic] of topics.entries()) {  // 4. for...of + entries()
  console.log(index, topic);
}
```

▶ CONSOLE OUTPUT

```
0 Arrays
1 Objects
2 DOM
Arrays
Objects
DOM
1. Arrays
2. Objects
3. DOM
0 Arrays
1 Objects
2 DOM
```

Loop	Choose it when...	Can stop early?
`for…of`	you want each item (your default)	Yes: `break`, `continue`, `return`
`forEach`	you want item and index and will visit all of them	**No**: it always visits every item
classic `for`	you need to skip, go backwards or step by 2	Yes
`for…of` + `entries()`	you want the index but also want `break`	Yes

WATCH OUT

- `return` inside a `forEach` callback only ends **that one call**, not the loop. To stop at the first match, use `for…of` with `break`, or better, `find` or `some` (15.6).
- Don't use `for…in` on arrays. It is for objects, gives the indexes as strings (`'0'`, `'1'`) and can pick up extra keys.

15.6 Searching: is it there, where is it, does anything match?

Most search methods take a **callback**: a small function that JavaScript calls once for each item. A callback that returns `true` or `false` is called a **predicate**; think of it as a yes/no question you ask about every item.

JS · the search toolbox

```
const scores = [72, 45, 91, 60, 88, 45];
console.log(scores.includes(91));                    // is it there?
console.log(scores.indexOf(45), scores.lastIndexOf(45));
console.log(scores.indexOf(100));                    // -1 means "not found"
console.log(scores.find((s) => s > 80));             // the first item that passes
console.log(scores.findIndex((s) => s > 80));        // …and its index
console.log(scores.findLast((s) => s < 50));         // search from the end
console.log(scores.some((s) => s < 50));             // does AT LEAST ONE pass?
console.log(scores.every((s) => s >= 50));           // do ALL pass?
console.log(scores.find((s) => s > 100));            // nothing passes
```

CONSOLE OUTPUT

```
true
1 5
-1
91
2
45
true
false
undefined
```

Your question	Method	Gives you
Is this exact value in the list?	`includes(value)`	`true` / `false`
Where is this exact value?	`indexOf(value)`	an index, or `-1`
Which is the first item that…?	`find(fn)`	the item, or `undefined`
Where is the first item that…?	`findIndex(fn)`	an index, or `-1`
Does any item…?	`some(fn)`	`true` / `false`
Does every item…?	`every(fn)`	`true` / `false`
Which items…? (all of them)	`filter(fn)` (15.8)	a new array

TIP

`some` and `every` stop as soon as they know the answer: `some` at the first `true`, `every` at the first `false`. On an empty array, `some` is `false` and `every` is `true` ("no item failed").

QUESTIONS LEARNERS ASK

Q Why not just `if (scores.indexOf(91))`?

Because `indexOf` returns `0` when the item is first, and `0` is falsy. Use `includes` for yes/no, or compare with `!== -1`.

Q `includes` can't find my object. Why?

`includes` and `indexOf` compare with `===`, and two separate objects are never `===` (Day 16). Use `find` or `some` with a callback that compares a key, e.g. `users.some((u) => u.id === 7)`.

15.7 map: change every item

`map`, `filter` and `reduce` are the three methods you'll use more than all the others together. Each takes a callback and returns something **new**, leaving the original alone.

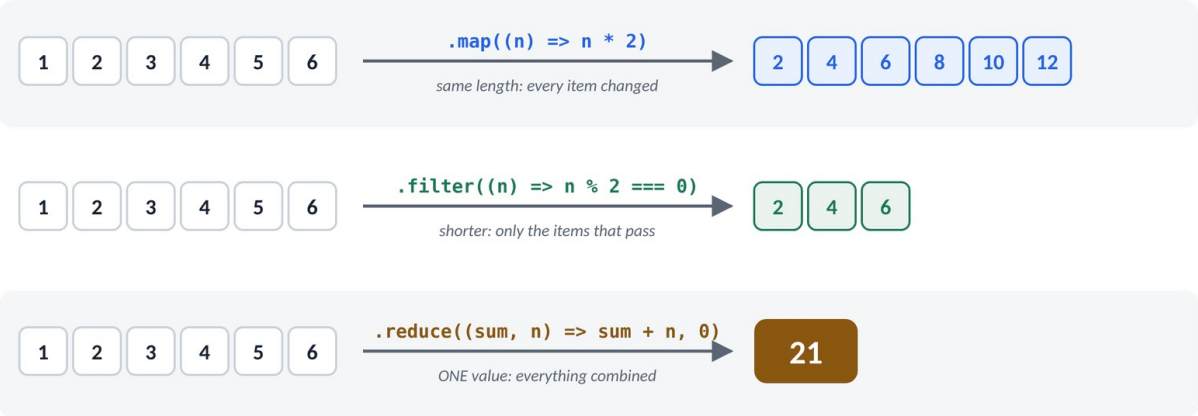


Figure 15.4 · The big three: map keeps the length, filter keeps some items, reduce makes one value

IN PLAIN WORDS

`map` is a machine on a conveyor belt. Every item goes in, a changed item comes out, and the belt never loses or adds an item: 3 in, 3 out.

JS · map with numbers and strings

```
const prices = [100, 250, 80];
const onSale = prices.map((price) => price - price / 10);    // 10% off
console.log(onSale);
console.log(prices);                                          // the original is unchanged

const names = ['rahim', 'nadia', 'tanvir'];
console.log(names.map((name) => name[0].toUpperCase() + name.slice(1)));
console.log(names.map((name, index) => `${index + 1}. ${name}`));
console.log(names.map((name) => name.length));
```

CONSOLE OUTPUT

```
[90, 225, 72]
[100, 250, 80]
['Rahim', 'Nadia', 'Tanvir']
['1. rahim', '2. nadia', '3. tanvir']
[5, 5, 6]
```

JS · map over a list of objects (more on Day 16)

```
const students = [
  { name: 'Rahim', score: 72 },
  { name: 'Nadia', score: 91 },
  { name: 'Tanvir', score: 45 },
];
console.log(students.map((s) => s.name));
console.log(students.map((s) => `${s.name}: ${s.score >= 50 ? 'pass' : 'fail'}`));
```

CONSOLE OUTPUT

```
['Rahim', 'Nadia', 'Tanvir']
['Rahim: pass', 'Nadia: pass', 'Tanvir: fail']
```

⇄ MORE WAYS **Three ways to double every number**

WAY 1 A for loop and push

```
const doubled = [];
for (const n of numbers) {
  doubled.push(n * 2);
}
```

Works everywhere; you manage the new array yourself.

WAY 2 forEach and push

```
const doubled = [];
numbers.forEach((n) => doubled.push(n * 2));
```

A little shorter, same idea.

WAY 3 map

```
const doubled = numbers.map((n) => n * 2);
```

One line, and the variable can be `const` from the start. This is what professionals write.

The missing return

JS · the most common map bug

```
const broken = [1, 2, 3].map((n) => { n * 2 });   // braces, but no return
console.log(broken);
const fixed = [1, 2, 3].map((n) => { return n * 2; });
console.log(fixed);
const short = [1, 2, 3].map((n) => n * 2);        // no braces: returned automatically
console.log(short);
```

▶ CONSOLE OUTPUT

```
[undefined, undefined, undefined]
[2, 4, 6]
[2, 4, 6]
```

WATCH OUT

`map` is for **making a new array**. If you only want to do something with each item (log it, add it to the page) and ignore the result, use `forEach` or `for…of`.

15.8 filter: keep some items

IN PLAIN WORDS

`filter` is a sieve. Each item is asked the callback's question; items that answer `true` fall through into the new array, the rest stay behind. The new array can be shorter, the same length, or empty, but its items are never changed.

JS · filter in five situations

```
const numbers = [1, 2, 3, 4, 5, 6];
console.log(numbers.filter((n) => n % 2 === 0));    // keep the evens
console.log(numbers.filter((n) => n > 10));         // nothing passes: an empty array

const scores = [72, 45, 91, 60, 88, 45];
console.log(scores.filter((s) => s >= 60).length);  // how many passed?

const tags = ['js', '', 'css', null, 'html', undefined];
console.log(tags.filter(Boolean));                  // drop empty/missing values

const cart = ['pen', 'book', 'lamp'];
console.log(cart.filter((item) => item !== 'book')); // remove by value
```

CONSOLE OUTPUT

```
[2, 4, 6]
[]
4
['js', 'css', 'html']
['pen', 'lamp']
```

`filter(Boolean)` passes each item to `Boolean()`, which returns `false` for the falsy values from Day 10 (`''`, `0`, `null`, `undefined`, `NaN`, `false`). It's a well-known shortcut for "remove the empty ones", but careful: it removes `0` too.

⇄ MORE WAYS **Three ways to find the passing scores**

WAY 1 filter

```
const passed = scores.filter((s) => s >= 50);
```

The clearest: it says exactly what it does.

WAY 2 A loop with if

```
const passed = [];
for (const s of scores) {
  if (s >= 50) passed.push(s);
}
```

Longer, but you can add extra steps (e.g. count the fails at the same time).

WAY 3 reduce

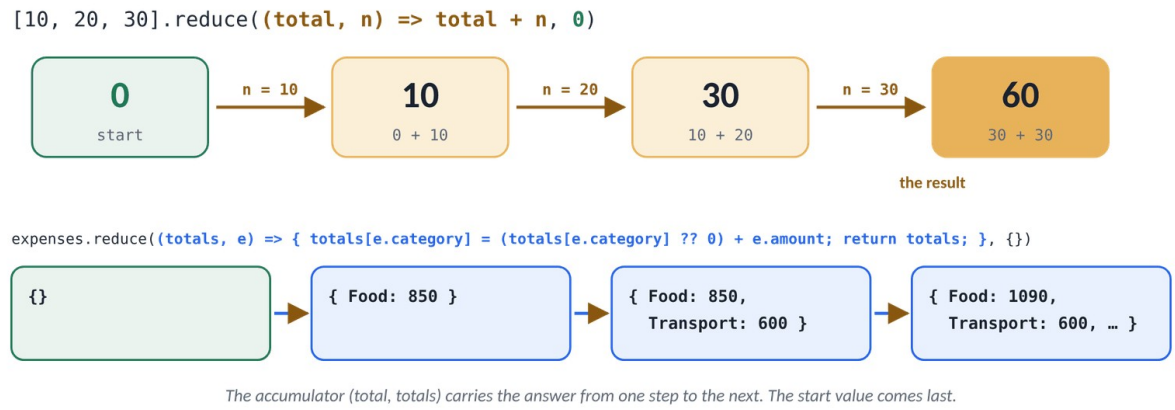
```
const passed = scores.reduce(
  (kept, s) => (s >= 50 ? [...kept, s] : kept), []);
```

Possible, but harder to read. Use `reduce` for **one** value, not for filtering.

15.9 reduce: combine everything into one value

IN PLAIN WORDS

`reduce` is a snowball rolling down a hill. It starts as the **start value** you give it, and at each item it picks up a little more. At the bottom you have one snowball: a total, a maximum, a count, or even a whole object.



The accumulator (total, totals) carries the answer from one step to the next. The start value comes last.

Figure 15.5 · reduce step by step: the accumulator carries the answer from one item to the next

Lap	sum (accumulator)	n (item)	The callback returns...
1	`0` (the start value)	`10`	`0 + 10` → `10`
2	`10`	`20`	`10 + 20` → `30`
3	`30`	`30`	`30 + 30` → `60`, the final result

JS · a sum, a maximum and a count

```
const numbers = [10, 20, 30];
const total = numbers.reduce((sum, n) => sum + n, 0);   // (callback, start value)
console.log(total);

const biggest = numbers.reduce((best, n) => (n > best ? n : best), numbers[0]);
console.log(biggest);

const words = ['js', 'css', 'js', 'html', 'js'];
const counts = words.reduce((tally, word) => {
  tally[word] = (tally[word] ?? 0) + 1;    // 0 the first time we meet a word
  return tally;                            // ALWAYS return the accumulator
}, {});                                    // start with an empty object
console.log(counts);

console.log([].reduce((sum, n) => sum + n, 0));   // empty array: the start value
```

CONSOLE OUTPUT

```
60
30
{js: 3, css: 1, html: 1}
0
```

JS · what happens without a start value

```
[].reduce((sum, n) => sum + n);   // no start value, nothing to start from
```

CONSOLE OUTPUT

```
Uncaught TypeError: Reduce of empty array with no initial value
```

⇄ MORE WAYS **Three ways to add up an array**

WAY 1 for...of and a running total

```
let total = 0;
for (const n of numbers) {
  total += n;
}
```

Easy to read and to debug with a breakpoint.

WAY 2 forEach

```
let total = 0;
numbers.forEach((n) => { total += n; });
```

The same idea as a callback.

WAY 3 reduce

```
const total = numbers.reduce((sum, n) => sum + n, 0);
```

One expression, and `total` can be `const`.

WATCH OUT

- Always give a **start value** (the second argument). Without it `reduce` uses the first item, which crashes on an empty array and gives wrong answers when you reduce objects.
- Always **return** the accumulator from the callback. Forget it and the next lap gets `undefined`.

15.10 Chaining: filter, then map, then reduce

Because `filter` and `map` return arrays, you can call the next method straight on the result. A chain reads like a recipe: one step per line, top to bottom.

JS · Exercise bonus: a chain, and a real-world one

```
const doubledEvensTotal = [1, 2, 3, 4, 5, 6]
  .filter((n) => n % 2 === 0)        // [2, 4, 6]
  .map((n) => n * 2)                 // [4, 8, 12]
  .reduce((sum, n) => sum + n, 0);   // 24
console.log(doubledEvensTotal);

const results = [
  { name: 'Rahim', score: 72 },
  { name: 'Nadia', score: 91 },
  { name: 'Tanvir', score: 45 },
  { name: 'Mitu', score: 88 },
];
const honours = results
  .filter((r) => r.score >= 80)      // the top scorers
  .map((r) => r.name)                // just their names
  .join(' and ');                    // one string
console.log(honours);
const average = results.reduce((sum, r) => sum + r.score, 0) / results.length;
console.log(average);
```

▶ CONSOLE OUTPUT

```
24
Nadia and Mitu
74
```

> **TIP**
>
> To debug a chain, split it into named steps (`const evens = …; const doubled = …;`) and log each one. Once it works, join it back into a chain if you like.

15.11 Sorting without surprises

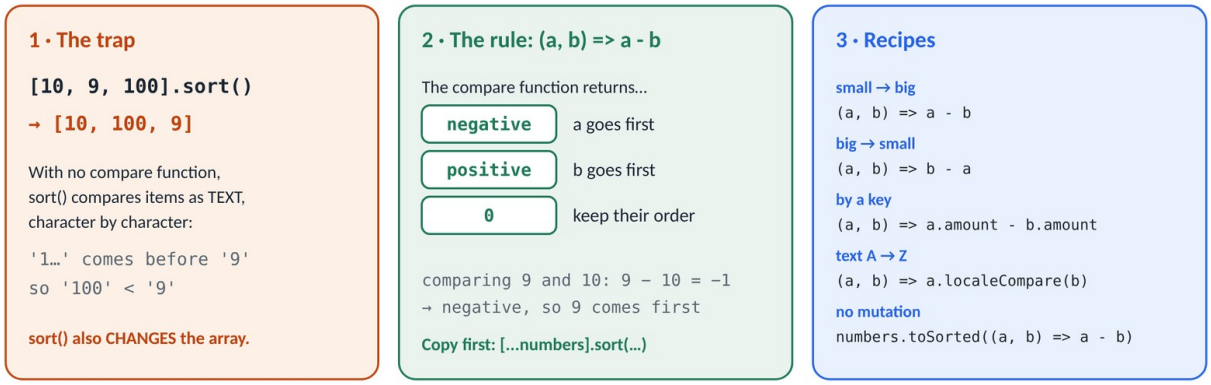


Figure 15.6 · Why [10, 9, 100].sort() goes wrong, the compare-function rule, and ready-made recipes

JS · numbers and text

```
console.log([10, 9, 100, 1].sort());                    // ✗ compared as TEXT
console.log([10, 9, 100, 1].sort((a, b) => a - b));     // small → big
console.log([10, 9, 100, 1].sort((a, b) => b - a));     // big → small

const names = ['nadia', 'Rahim', 'Élodie', 'arif'];
console.log([...names].sort());                          // capitals first, accents last
console.log([...names].sort((a, b) => a.localeCompare(b)));   // like a dictionary
```

CONSOLE OUTPUT

```
[1, 10, 100, 9]
[1, 9, 10, 100]
[100, 10, 9, 1]
['Rahim', 'arif', 'nadia', 'Élodie']
['arif', 'Élodie', 'nadia', 'Rahim']
```

With no compare function, `sort` turns every item into text and compares character by character, so `'100'` comes before `'9'` just as "apple" comes before "banana". The compare function fixes this: `sort` gives it two items, `a` and `b`, and looks only at the **sign** of the answer.

compare(a, b) returns	sort puts…	Example with (a, b) => a - b
a negative number	`a` first	`9 - 10 = -1` → 9 before 10
a positive number	`b` first	`100 - 9 = 91` → 9 before 100
`0`	keeps their order	`5 - 5 = 0`

JS · sorting objects by a key

```
const people = [
  { name: 'Nadia', age: 31 },
  { name: 'Arif', age: 24 },
  { name: 'Mitu', age: 28 },
];
const byAge = [...people].sort((a, b) => a.age - b.age);   // copy, then sort
console.log(byAge.map((p) => p.name));
// toSorted() returns a sorted copy
const byName = people.toSorted((a, b) => a.name.localeCompare(b.name));
console.log(byName.map((p) => p.name));
console.log(people.map((p) => p.name));                     // untouched
```

CONSOLE OUTPUT

```
['Arif', 'Mitu', 'Nadia']
['Arif', 'Mitu', 'Nadia']
['Nadia', 'Arif', 'Mitu']
```

WATCH OUT

`sort` **changes the original** and returns that same array. `const sorted = scores.sort(…)` sorts `scores` too. Copy first with `[...scores].sort(…)`, or use `toSorted(…)`.

15.12 Spread, Set and other handy tools

JS · spread, Set, join, flat and reverse

```
const scores = [72, 45, 91];
console.log(Math.max(...scores), Math.min(...scores));  // spread items into arguments
console.log(Math.max(scores));                          // ✗ the array is ONE argument

const morning = ['tea'];
const evening = ['rice', 'dal'];
console.log([...morning, 'lunch', ...evening]);         // arrays + single items

console.log([...new Set(['js', 'css', 'js'])]);         // remove duplicates
console.log(['2026', '10', '10'].join('-'));            // array → string
console.log([[1, 2], [3, 4]].flat());                   // flatten one level
console.log([1, 2, 3].reverse());                       // changes the original!
```

CONSOLE OUTPUT

```
91 45
NaN
['tea', 'lunch', 'rice', 'dal']
['js', 'css']
2026-10-10
[1, 2, 3, 4]
[3, 2, 1]
```

A Set keeps one of each value

JS · add, has and size

```
const seen = new Set();
seen.add('js');
seen.add('css');
seen.add('js');                  // already there: ignored
console.log(seen.size, seen.has('css'), seen.has('php'));
console.log(seen);
console.log([...seen]);          // back to an array
```

CONSOLE OUTPUT

```
2 true false
Set(2) {'js', 'css'}
['js', 'css']
```

A Set answers "have I seen this before?" instantly, however big it is. That makes it the key tool for two of this week's LeetCode problems (Day 17 and, as a Map, Day 19).

More methods you'll meet	Example → result
`join(sep)`	`['a', 'b'].join(', ')` → `'a, b'`
`concat()`	`[1].concat([2, 3])` → `[1, 2, 3]` (same as `[...a, ...b]`)
`flat()` / `flatMap()`	`[[1], [2]].flat()` → `[1, 2]`
`fill(value)`	`Array(3).fill('-')` → `['-', '-', '-']`
`reverse()` / `toReversed()`	reverse in place / a reversed copy
`Array.isArray(x)`	the reliable "is it an array?" test
`at(i)`	`at(0)` first, `at(-1)` last, `at(-2)` second to last

15.13 Practice: the 10 exercises

Open `practice/arrays.html` with Live Server and write each function in `arrays-starter.js` until every test prints PASS. Part 1 of `arrays-solutions.js` (A to E) repeats today's basics with tests; read it first and predict each result.

Function	Example	Use
1 · `double(numbers)`	`[1, 2, 3]` → `[2, 4, 6]`	`map`
2 · `keepEvens(numbers)`	`[1…6]` → `[2, 4, 6]`	`filter` and `%`
3 · `sum(numbers)`	`[10, 20, 30]` → `60`; `[]` → `0`	`reduce` with a start value
4 · `max(numbers)`	`scores` → `91`	`Math.max(...)`, `reduce` or a loop
5 · `sortNumbers(numbers)`	`[10, 9, 100, 1]` → `[1, 9, 10, 100]`	copy, then `sort((a, b) => a - b)`
6 · `unique(items)`	`[1, 2, 2, 3, 1]` → `[1, 2, 3]`	`new Set`, or `filter` + `indexOf`
7 · `findFirstAbove(n, limit)`	`(scores, 80)` → `91`	`find`
8 · `anyFailed` / `allPassed`	`true` / `false`	`some` / `every`
9 · `countAbove(n, limit)`	`(scores, 60)` → `3`	`filter` then `.length`
10 · `topThree(numbers)`	`scores` → `[91, 88, 72]`	copy, sort high → low, `slice(0, 3)`

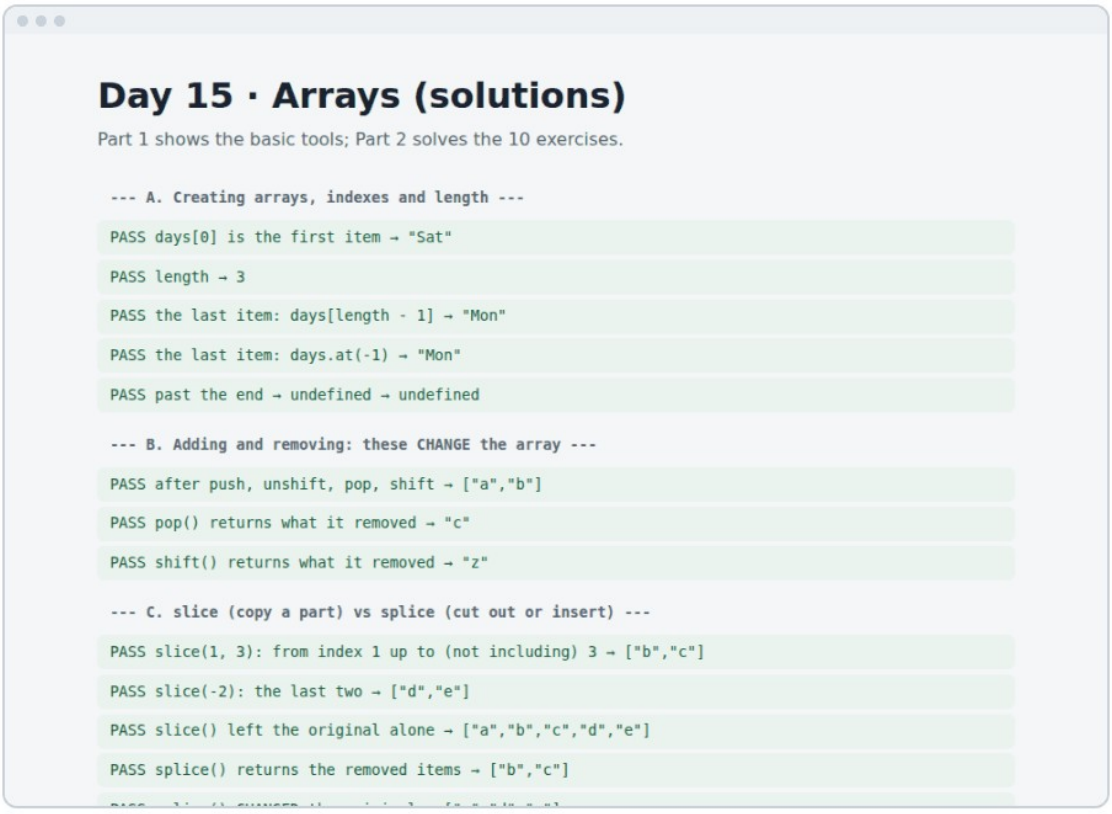


Figure 15.7 · practice/solutions.html: Part 1 of the solutions, every line a PASS

⇄ MORE WAYS **Exercise 6 · two ways to remove duplicates**

WAY 1 A Set

```
const unique = (items) => [...new Set(items)];
```

Short and fast. A Set keeps the first time it meets each value, so the order is kept too.

WAY 2 filter + indexOf

```
const unique = (items) =>
  items.filter((item, index) => items.indexOf(item) === index);
```

Keeps an item only where it appears for the first time (its first index equals the current index). Slower on long lists, but a good logic exercise.

▶ CONSOLE OUTPUT · the goal: 15 × PASS in arrays.html

```
PASS  double([1, 2, 3])  →  [2,4,6]
PASS  keepEvens([1…6])  →  [2,4,6]
PASS  sum([10, 20, 30])  →  60
PASS  sum([]) → 0  →  0
…
PASS  topThree(scores)  →  [91,88,72]
PASS  scores still in the original order  →  [72,45,91,60,88,45]
```

15.14 Plan the three widgets, then build them without JavaScript

This week's output is one page with three widgets that almost every web app has: **tabs**, a **modal dialog** and a **to-do list**. Professionals plan before they code: what each widget does, which **states** it can be in, and what **data** it needs. Read `widgets/plan.md`, then write the HTML and CSS yourself.

Widget	Behaviour	States the CSS must show	Data (from Day 16/19)
Tabs	Click a tab (or press ← →) to show its panel	selected tab: `aria-selected="true"`; other panels: `hidden`	an array of track objects
Modal	A button opens a dialog; ×, Cancel, Escape or the backdrop close it	open / closed (built into `<dialog>`); a thank-you message	the form's fields: `name`, `day`
To-do	Add, tick and delete tasks; later filter and edit	done: `.is-done` (line-through); empty-list message	an array of `{ id, text, done }` objects

HTML · widgets/index.html (Day 15): the tabs, written by hand

```
<div class="tabs" id="tracks">
  <div class="tab-list" role="tablist" aria-label="Course tracks">
    <button class="tab" type="button" role="tab" id="tab-frontend"
            aria-selected="true" aria-controls="panel-frontend">Frontend</button>
    <button class="tab" type="button" role="tab" id="tab-backend"
            aria-selected="false" aria-controls="panel-backend"
            tabindex="-1">Backend</button>
    …
  </div>
  <div class="tab-panel" role="tabpanel" id="panel-frontend"
       aria-labelledby="tab-frontend" tabindex="0">
    <p class="panel-meta">Months 1–2</p>
    <h3>Frontend</h3>
    …
  </div>
  <div class="tab-panel" role="tabpanel" id="panel-backend"
       aria-labelledby="tab-backend" tabindex="0" hidden>…</div>
</div>
```

Attribute	What it tells the browser and screen readers
`role="tablist"` / `"tab"` / `"tabpanel"`	These buttons are tabs, and these boxes are their panels
`aria-selected="true"`	This tab is the chosen one (the CSS styles it too)
`aria-controls="panel-…"`	Which panel this tab shows (JavaScript uses it on Day 17)
`aria-labelledby="tab-…"`	The panel's name is the text of its tab
`tabindex="-1"` on the other tabs	Only the selected tab is reached with Tab; arrows move between tabs (Day 18)
`hidden`	The panel is not shown and not read aloud

CSS · widgets/css/styles.css: every state has a style before any JavaScript exists

```
/* The selected tab: styled by the same attribute screen readers use */
.tab[aria-selected="true"] {
  border-bottom-color: var(--color-primary);
  color: var(--color-primary);
}

/* A finished task */
.todo-item.is-done .todo-text {
  color: var(--color-muted);
  text-decoration: line-through;
}

/* minmax(0, 1fr), not 1fr: stops the tab row from stretching the page on phones */
main { display: grid; grid-template-columns: minmax(0, 1fr); }
```

TIP

Test your states before writing JavaScript: in DevTools → Elements, double-click `aria-selected="false"` on a tab and change it to `"true"`, or add the class `is-done` to a task. If it looks right, the CSS is ready, and tomorrow's scripts only have to switch attributes and classes.

15.15 DSA: LeetCode 1480, Running Sum of 1d Array

The problem. Given an array `nums`, return a new array where each item is the sum of all items up to and including that position. `[1, 2, 3, 4]` → `[1, 3, 6, 10]` (1, 1+2, 1+2+3, 1+2+3+4).

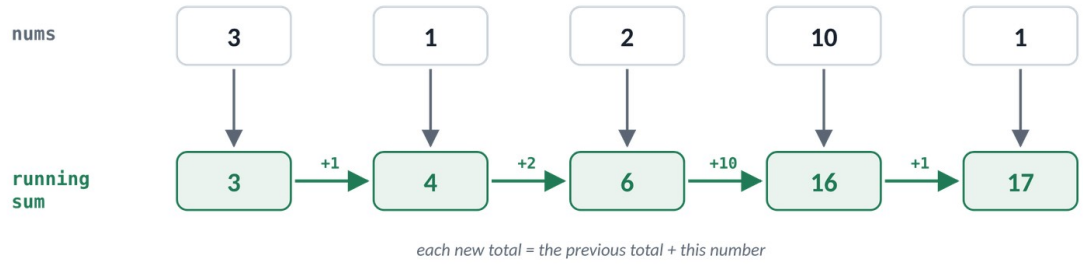


Figure 15.8 · Each new total is the previous total plus the current number

1. **Understand:** the output has the same length as the input, and each item depends on the one before it.
2. **The slow idea:** for each position, add up everything before it again. That repeats a lot of work.
3. **The key insight:** keep a **running total**. Each step is just `total += nums[i]`, then remember the total. You never re-add earlier numbers.
4. **Edge cases:** one number → the same number; negative numbers work the same way.

⇄ MORE WAYS Running Sum three ways (dsa/1480-running-sum.js)

WAY 1 A new array and a running total

```
function runningSum(nums) {
  const result = [];
  let total = 0;
  for (const n of nums) {
    total += n;
    result.push(total);
  }
  return result;
}
```

The clearest version. Time O(n), and O(n) extra space for the new array.

WAY 2 In place: each item adds the one before it

```
function runningSumInPlace(nums) {
  for (let i = 1; i < nums.length; i++) {
    nums[i] += nums[i - 1];   // nums[i - 1] already holds the sum up to i - 1
  }
  return nums;
}
```

No extra array: O(1) extra space. It **mutates** the input, which LeetCode allows but your own apps usually shouldn't.

WAY 3 map with a total outside the callback

```
function runningSumMap(nums) {
  let total = 0;
  return nums.map((n) => (total += n));
}
```

Short. It works because `total += n` is an expression whose value is the new total. Clever code is harder to read, so use it only when the team knows the trick.

RESULT

```
runningSum([1, 2, 3, 4])     → [1, 3, 6, 10]
runningSum([1, 1, 1, 1, 1])  → [1, 2, 3, 4, 5]
runningSum([3, 1, 2, 10, 1]) → [3, 4, 6, 16, 17]
```

How fast? Each way visits every number once: **O(n)** time. The running-total idea (also called a **prefix sum**) comes back often, e.g. "the sum of any range of the array" becomes one subtraction once you have the prefix sums.

> **TIP**
>
> On LeetCode, choose JavaScript, paste the body into `var runningSum = function(nums) { … };`, then **Run** and **Submit**. Locally, open `dsa/index.html` with Live Server to see every test pass.

Build it, step by step

1. Open `practice/solutions.html` with Live Server and the Console. Read Part 1 (A–E) of `arrays-solutions.js`, predicting each line before you look.
2. Open `practice/arrays.html` and write the 10 functions in `arrays-starter.js` until all 15 tests pass.
3. Stuck for 20 minutes on one? Read only that exercise in `arrays-solutions.js`, close it, and write it yourself.
4. Read `widgets/plan.md`. Create `widgets/index.html`: the header, the tabs (four buttons and four panels), the modal button and the to-do form with three sample tasks.
5. Write `widgets/css/styles.css` mobile-first. Style every state by hand in DevTools: a selected tab, a done task.
6. Solve **Running Sum of 1d Array** on LeetCode, then compare with `dsa/1480-running-sum.js`.
7. Commit: `git commit -m "Day 15: array exercises and static widget markup"` and push.

★ MUST KNOW *the ideas to remember from today*

- ✓ Indexes start at **0**; the last item is `length - 1` or `at(-1)`.
- ✓ `push`/`pop` work on the end, `unshift`/`shift` on the start. All four change the array.
- ✓ `slice` copies a part; `splice` changes the original.
- ✓ `const` stops reassignment, not changes inside the array.
- ✓ `map` → same length, changed items. `filter` → some items. `reduce` → one value (give it a start value).
- ✓ `find` gives the first match or `undefined`; `some`/`every` give `true`/`false`.
- ✓ Numbers need `sort((a, b) => a - b)`, and `sort` mutates: copy first or use `toSorted`.
- ✓ `[...array]` copies; `[...new Set(array)]` removes duplicates.

WATCH OUT

- Off-by-one: `list[list.length]` is always `undefined`.
- `[10, 9, 100].sort()` → `[10, 100, 9]`: sorted as text.
- A `map` callback with `{ }` but no `return` gives `[undefined, …]`.
- Using `map` when you don't use the new array (use `forEach`).
- `reduce` without a start value, or without returning the accumulator.
- `const copy = original` is not a copy.
- `if (list.indexOf(x))` fails when x is at index 0; use `includes`.

QUESTIONS LEARNERS ASK

Q Which should I learn first: loops or map/filter/reduce?

Both, in that order. Loops show what happens step by step; the methods say the same thing in one line. When a method confuses you, rewrite it as a `for…of` loop.

Q Are array methods slower than loops?

A tiny bit, and it almost never matters. Write the clearest code first. Chaining three methods on 10 items is instant.

Q Why do so many methods take a function?

Because the method knows how to walk through the array, and only you know what to do with each item. That's the callback idea from Day 11.

Q What is `_` in `(_, index) => …`?

A normal parameter name. By habit, `_` means "I have to accept this value to get to the next one, but I don't use it".

More practice

MORE PRACTICE · try each one, then check the Answers at the back

1. Given `const fruits = ['mango', 'banana', 'lychee']`, write code that prints `3 fruits: mango, banana, lychee`.
2. Make an array of the squares of 1 to 10 without typing them.
3. From `['  js', 'CSS ', ' Html']`, make `['js', 'css', 'html']`.
4. Count how many words in `'the cat and the hat'` have 3 letters.
5. Find the longest word in `['grid', 'flexbox', 'css']`.
6. Write `last(arr, n)` that returns the last n items, e.g. `last([1, 2, 3, 4], 2)` → `[3, 4]`.
7. Move the first item of an array to the end, without mutating it: `[1, 2, 3]` → `[2, 3, 1]`.
8. Why does `[1, 2, 3].includes('2')` return `false`?

Check yourself

1. What is the difference between `slice` and `splice`?
2. Why does `[10, 9, 100].sort()` give `[10, 100, 9]`, and how do you fix it?
3. You need the total price of a cart. Which method would you use, and what start value?

*Answer out loud before you turn to the **Answers** section at the back.*

DONE WHEN · tick these off, then mark the day done in your Logbook

- ☐ All 15 tests in `arrays.html` pass with your own code
- ☐ The widgets page shows the three widgets, styled, with no JavaScript yet
- ☐ Running Sum accepted on LeetCode
- ☐ Committed and pushed

Resources

All free. Every name is a link, in Word and in the PDF.

Resource (click to open)	Use it for
javascript.info: Arrays javascript.info/array	The basics, with tasks at the end
javascript.info: Array methods javascript.info/array-methods	map, filter, reduce, sort and more, with 13 tasks
MDN: Array developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array	The full list of methods; each page has live examples
MDN: Array.prototype.map() developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/map	map in detail
MDN: Array.prototype.reduce() developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/reduce	reduce, including grouping and counting
MDN: Array.prototype.sort() developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/sort	Compare functions, and sorting objects
MDN: Spread syntax (...) developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Spread_syntax	Copying, joining and passing arrays as arguments
MDN: Set developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Set	add, has, delete, size
Does it mutate? doesitmutate.xyz	Every array method, marked "mutates" or not
WAI-ARIA Authoring Practices: Tabs w3.org/WAI/ARIA/apg/patterns/tabs	How accessible tabs should behave (the plan follows it)
LeetCode 1480: Running Sum of 1d Array leetcode.com/problems/running-sum-of-1d-array	Today's problem

DAY
16

WEEK 3 · SUNDAY 11 OCTOBER

Objects and JSON: modelling real data

Arrays hold lists; objects describe things. Together they can model almost anything.

TIME	YOU BUILD	CODE FOLDER
3 hours	An objects demo, five expense functions, tabs built from data, and Richest Customer Wealth	day16-objects

Today you will

- ✓ Describe a real thing as an object, and read values with dot and bracket notation
- ✓ Add, change and delete keys, and check whether a key exists
- ✓ Write methods, and explain what `this` means inside them
- ✓ Unpack values with destructuring, and copy or merge objects with spread
- ✓ Explain the difference between a reference and a copy, and make deep copies
- ✓ Loop over objects with `Object.keys`, `Object.values` and `Object.entries`
- ✓ Answer questions about an array of objects: totals, groups, sorting
- ✓ Turn data into text and back with `JSON.stringify` and `JSON.parse`
- ✓ Read nested data safely with `?.` and `??`
- ✓ Build the tabs from an array of objects instead of hand-written HTML
- ✓ Solve LeetCode 1672, Richest Customer Wealth, three ways

Key terms

Term	Meaning
Object	A group of named values: `{ name: 'Rahim', city: 'Dhaka' }`
Key (property)	The name of a value in an object, like `name`
Value	What a key holds: any type, even an array, an object or a function
Method	A function stored as a value in an object: `counter.increment()`
this	Inside a method: the object the method was called on
Destructuring	Unpacking values into variables: `const { name } = user`
Spread / rest	`...` unpacks (spread) or collects (rest) keys or items
Reference	A variable holds a link to an object, not the object itself
Shallow / deep copy	Copies only the outer object / copies every level inside it
JSON	A text format for data; how objects travel and get saved
Optional chaining (`?.`)	Reads a key only if the thing before it isn't null or undefined

16.1 What an object is

> **IN PLAIN WORDS**
>
> An object is a filled-in form. Every box has a **label** (the key) and an **answer** (the value). An array answers "what is item number 3?"; an object answers "what is the city?"

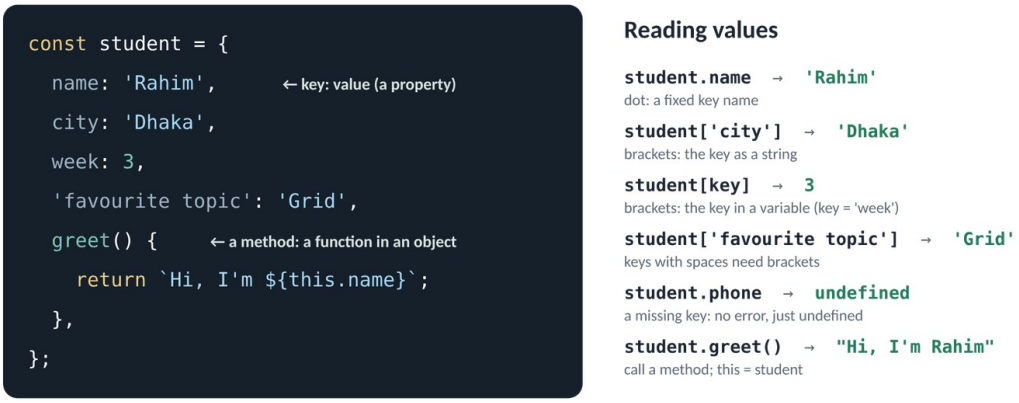


Figure 16.1 · An object literal (left) and the ways to read its values (right)

JS · an object with five keys

```
const student = {
  name: 'Rahim',                  // key: value, then a comma
  city: 'Dhaka',
  week: 3,
  skills: ['html', 'css'],        // a value can be an array…
  address: { area: 'Mirpur' },    // …or another object
};
console.log(student.name, student.week);
console.log(student.skills[1], student.address.area);
console.log(student);
```

▶ CONSOLE OUTPUT

```
Rahim 3
css Mirpur
{name: 'Rahim', city: 'Dhaka', week: 3, skills: Array(2), address: {…}}
```

The last line is how Chrome shows an object before you open it: arrays inside become `Array(2)` and objects inside become `{…}`. Click the small triangle (▶) to see everything.

	Array	Object
Holds	an ordered list of similar things	named facts about one thing
You ask	"what is the 3rd item?"	"what is the city?"
Brackets	`[ ]`	`{ }`
Example	`['Sat', 'Sun', 'Mon']`	`{ name: 'Rahim', city: 'Dhaka' }`
In an app	the list of expenses	one expense

> **TIP**
>
> You've used objects since Day 10 without knowing it: `console.log` is the `log` method of the `console` object, and `Math.max` is the `max` method of `Math`.

16.2 Reading values: dot or brackets

JS · Part A of objects-demo.js

```
const student = {
  name: 'Rahim', city: 'Dhaka', week: 3, 'favourite topic': 'CSS Grid',
};
console.log(student.name);                 // dot: you know the key
console.log(student['city']);              // brackets: the key as a string
const key = 'week';
console.log(student[key]);                 // brackets: the key is in a variable
console.log(student.key);                  // ✗ looks for a key called "key"
console.log(student['favourite topic']);   // brackets: a key with a space
console.log(student.phone);                // a missing key: undefined, no error
```

▶ CONSOLE OUTPUT

```
Rahim
Dhaka
3
undefined
CSS Grid
undefined
```

Use	When
`obj.key`	You know the key while writing the code, and it's a normal name (your default)
`obj['key']`	The key has spaces or dashes, or starts with a digit
`obj[variable]`	The key is only known when the code runs: from a loop, a click, a form field

A lookup object instead of if / else

JS · brackets with a variable

```
const dayNames = { sat: 'Saturday', sun: 'Sunday', mon: 'Monday' };
const short = 'sun';
console.log(dayNames[short]);

const prices = { tea: 20, coffee: 60 };
for (const drink of ['tea', 'coffee', 'juice']) {
  console.log(drink, prices[drink] ?? 'not on the menu');
}
```

▶ CONSOLE OUTPUT

```
Sunday
tea 20
coffee 60
juice not on the menu
```

A lookup object replaces a long `if / else if` or `switch`: the key is the question, the value is the answer. On Day 20 the tabs use one to map key names to moves, and the to-do list uses one for its filters.

16.3 Changing, adding and deleting keys

JS · Part B: objects can grow and shrink

```
const course = { title: 'Full-stack', weeks: 26 };
course.weeks = 27;                      // change
course.language = 'JavaScript';         // add: just assign a new key
course['start date'] = '2026-09-26';    // add, with brackets
delete course.language;                 // remove
console.log(course);
console.log('weeks' in course, 'language' in course);
console.log(Object.hasOwn(course, 'title'));
console.log(Object.keys(course).length);   // how many keys
```

▶ CONSOLE OUTPUT

```
{title: 'Full-stack', weeks: 27, 'start date': '2026-09-26'}
true false
true
3
```

⇄ MORE WAYS **Three ways to check whether a key exists**

WAY 1 The in operator

```
'email' in user
```

Short and clear. Also finds keys an object inherits (rarely matters for plain data).

WAY 2 Object.hasOwn()

```
Object.hasOwn(user, 'email')
```

Only the object's own keys. The strictest check.

WAY 3 Compare with undefined

```
user.email !== undefined
```

Common, but wrong if a key exists and its value is `undefined` on purpose.

WATCH OUT

A `const` object can still change inside, like a `const` array (15.1). `const` only stops you pointing the name at a different object. `Object.freeze(obj)` stops changes to the top level, if you ever need that.

16.4 Methods and this

JS · Part C: a method changes its own object

```
const counter = {
  count: 0,
  increment() {            // a method: a function stored in the object
    this.count++;          // this = the object before the dot
    return this.count;
  },
  describe() {
    return `The count is ${this.count}`;
  },
};
counter.increment();
counter.increment();
console.log(counter.count);
console.log(counter.describe());
```

▶ CONSOLE OUTPUT

```
2
The count is 2
```

When you call `counter.increment()`, JavaScript sets `this` to the object **before the dot**, here `counter`. That's how one method can work with its own object's data.

⇄ MORE WAYS **Three ways to write a method, and one that doesn't work**

WAY 1 Method shorthand (use this)

```
const user = {
  name: 'Nadia',
  greet() { return `Hi, I'm ${this.name}`; },
};
```

Short, and `this` works.

WAY 2 A function expression

```
const user = {
  name: 'Nadia',
  greet: function () { return `Hi, I'm ${this.name}`; },
};
```

The older spelling of the same thing.

WAY 3 ✗ An arrow function

```
const user = {
  name: 'Nadia',
  greet: () => `Hi, I'm ${this.name}`,   // this is NOT user here
};
```

Arrow functions don't get their own `this`, so `this.name` is not `'Nadia'`. Use arrows for callbacks, and shorthand for methods.

WATCH OUT

`this` depends on **how** a function is called, not where it was written. `const inc = counter.increment; inc();` loses the object before the dot, so `this` is no longer `counter`. When a method is passed as a callback, wrap it: `button.addEventListener('click', () => counter.increment())`.

16.5 Shorthand keys and computed keys

JS · Part D, and computed keys

```
const title = 'Laptop stand';
const price = 1800;
const product = { title, price };        // same as { title: title, price: price }
console.log(product);

const field = 'colour';
const choice = { [field]: 'black' };     // [ ] around a key: use the variable's VALUE
console.log(choice);
```

CONSOLE OUTPUT

```
{title: 'Laptop stand', price: 1800}
{colour: 'black'}
```

Shorthand is everywhere in modern code, e.g. `todos.push({ id, text, done: false })` on Day 19. Computed keys are handy for forms: `{ [input.name]: input.value }` turns any field into a key and value.

16.6 Destructuring: unpack values into variables

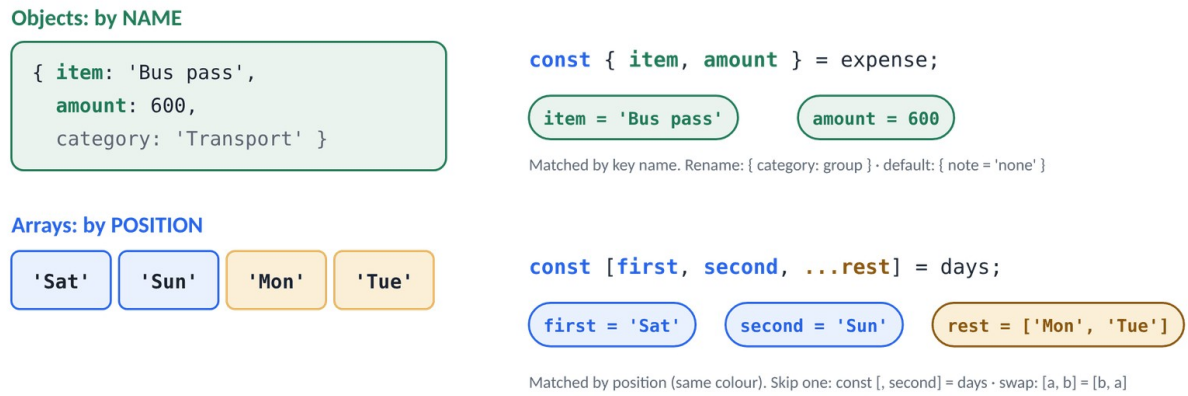


Figure 16.2 · Objects are unpacked by name; arrays are unpacked by position

JS · Part E: objects by name, arrays by position

```
const expense = { item: 'Bus pass', amount: 600, category: 'Transport' };
const { item, amount } = expense;           // pick keys by name
const { category: group } = expense;        // rename: category → group
const { note = 'no note' } = expense;       // a default for a missing key
console.log(item, amount, group, note);

const [first, second, ...others] = ['Sat', 'Sun', 'Mon', 'Tue'];   // by position
console.log(first, second, others);
const [, , third] = ['Sat', 'Sun', 'Mon'];  // skip items with commas
console.log(third);

let a = 1;
let b = 2;
[a, b] = [b, a];                            // swap two variables in one line
console.log(a, b);
```

CONSOLE OUTPUT

```
Bus pass 600 Transport no note
Sat Sun ['Mon', 'Tue']
Mon
2 1
```

JS · destructuring in parameters, nested objects and loops

```
function describe({ name, city = 'somewhere' }) {   // destructure the argument
  return `${name} from ${city}`;
}
console.log(describe({ name: 'Nadia', city: 'Sylhet' }));
console.log(describe({ name: 'Arif' }));

const user = { id: 7, profile: { name: 'Mitu', age: 28 } };
const { profile: { name, age } } = user;            // reach inside a nested object
console.log(name, age);

const bills = [{ title: 'Tea', cost: 20 }, { title: 'Bus', cost: 40 }];
for (const { title, cost } of bills) {              // destructure in a loop
  console.log(title, cost);
}
```

CONSOLE OUTPUT

```
Nadia from Sylhet
Arif from somewhere
Mitu 28
Tea 20
Bus 40
```

⇄ MORE WAYS Three ways to get values out of an object

WAY 1 Dots, one by one

```
const item = expense.item;
const amount = expense.amount;
```

Clear, but repetitive when you need several keys.

WAY 2 Destructuring

```
const { item, amount } = expense;
```

One line for many keys; the variable names match the keys.

WAY 3 Destructuring in the parameter

```
function show({ item, amount }) {
  return `${item}: ৳${amount}`;
}
show(expense);
```

The function says exactly which keys it needs. On Day 20 `setupModal({ openButton, dialog, status })` works like this, so the order of the arguments never matters.

16.7 Spread and rest: three dots, two jobs

JS · Part F: merge, copy-and-change, remove a key, collect arguments

```
const defaults = { theme: 'light', language: 'en', fontSize: 16 };
const saved = { theme: 'dark' };
const settings = { ...defaults, ...saved };      // merge: later keys win
console.log(settings);
console.log(defaults.theme);                     // untouched

const updated = { ...settings, fontSize: 18 };   // copy, then change one key
console.log(updated.fontSize, settings.fontSize);

const { language, ...withoutLanguage } = settings;   // rest: "all other keys"
console.log(withoutLanguage);

function total(...numbers) {          // rest parameter: any number of arguments
  return numbers.reduce((sum, n) => sum + n, 0);
}
console.log(total(1, 2, 3), total(10, 20), total());
```

▶ CONSOLE OUTPUT

```
{theme: 'dark', language: 'en', fontSize: 16}
light
18 16
{theme: 'dark', fontSize: 16}
6 30 0
```

	Spread: unpacks	Rest: collects
Where	on the right of `=`, or in a call	on the left of `=`, or in a parameter list
Objects	`{ ...a, key: 1 }` copies a's keys	`const { id, ...others } = a`
Arrays	`[...list, 4]`, `Math.max(...list)`	`const [first, ...rest] = list`
Functions	`add(...pair)`	`function add(...numbers) {}`

⇄ MORE WAYS Three ways to copy an object

WAY 1 Spread

```
const copy = { ...original };
```

The usual way. Add keys after it to change them: `{ ...original, done: true }`.

WAY 2 Object.assign()

```
const copy = Object.assign({}, original);
```

The older way; you'll see it in older code and libraries.

WAY 3 structuredClone(): a deep copy

```
const copy = structuredClone(original);
```

Copies arrays and objects inside it too (16.8). Built into every current browser.

16.8 References and copies

IN PLAIN WORDS

A variable doesn't hold an object; it holds a **remote control** for it. `const b = a` hands you a second remote for the same TV: press "volume up" on either and the one TV gets louder. A copy is a second TV.

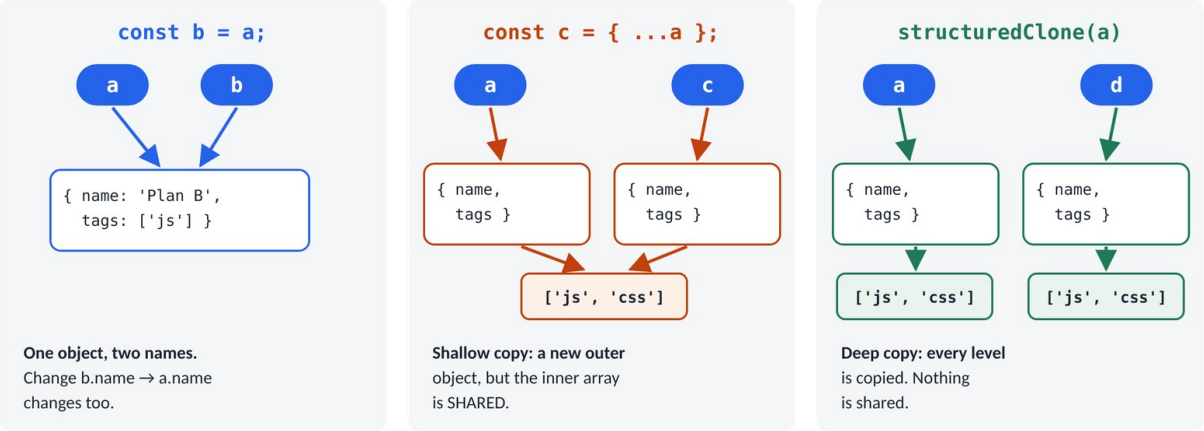


Figure 16.3 · Two names for one object, a shallow copy that still shares the inner array, and a deep copy

JS · **Part G: values are copied, objects are shared**

```
let x = 5;
let y = x;           // numbers, strings and booleans are COPIED
y = 10;
console.log(x, y);

const original = { name: 'Plan A', tags: ['js'] };
const sameObject = original;          // NOT a copy: two names, ONE object
sameObject.name = 'Plan B';
console.log(original.name);
console.log({ n: 1 } === { n: 1 });   // two objects, even if they look alike
console.log(original === sameObject); // the same object

const shallow = { ...original };      // a new outer object…
shallow.tags.push('css');             // …but tags is still SHARED
console.log(original.tags);

const deep = structuredClone(original);   // copies every level
deep.tags.push('html');
console.log(original.tags, deep.tags);
```

CONSOLE OUTPUT

```
5 10
Plan B
false
true
['js', 'css']
['js', 'css'] ['js', 'css', 'html']
```

Kind of value	`let b = a` gives…	Compared with `===`
number, string, boolean, null, undefined	an independent copy	by value: `'hi' === 'hi'` is true
object, array, function	a second name for the same thing	by identity: only the same object is `===`

> **TIP**
>
> To compare two objects by their **content**, compare their JSON: `JSON.stringify(a) === JSON.stringify(b)`. That's exactly what the `test()` helper in this week's practice files does. It works for simple data whose keys are in the same order.

16.9 Looping over an object

JS · Part H: keys, values and entries turn an object into arrays

```
const prices = { tea: 20, coffee: 60, juice: 45 };
console.log(Object.keys(prices));          // the keys, as an array
console.log(Object.values(prices));        // the values
console.log(Object.entries(prices)[0]);    // [key, value] pairs; the first one

for (const [drink, cost] of Object.entries(prices)) {   // destructure each pair
  console.log(`${drink} costs ৳${cost}`);
}
const total = Object.values(prices).reduce((sum, p) => sum + p, 0);
console.log(total);
const cheap = Object.keys(prices).filter((drink) => prices[drink] < 50);
console.log(cheap);
const doubled = Object.fromEntries(
  Object.entries(prices).map(([drink, cost]) => [drink, cost * 2]),
);
console.log(doubled);
```

CONSOLE OUTPUT

```
['tea', 'coffee', 'juice']
[20, 60, 45]
['tea', 20]
tea costs ৳20
coffee costs ৳60
juice costs ৳45
125
['tea', 'juice']
{tea: 40, coffee: 120, juice: 90}
```

The trick: `Object.keys`, `values` and `entries` turn an object into an array, and then every array method from Day 15 works. `Object.fromEntries` turns `[key, value]` pairs back into an object.

⇄ MORE WAYS **Three ways to loop over an object**

WAY 1 for...of + Object.entries (use this)

```
for (const [key, value] of Object.entries(obj)) {
  console.log(key, value);
}
```

Key and value together, and `break` works.

WAY 2 Object.keys + brackets

```
Object.keys(obj).forEach((key) => {
  console.log(key, obj[key]);
});
```

Fine when you mostly need the keys.

WAY 3 for...in (older)

```
for (const key in obj) {
  console.log(key, obj[key]);
}
```

Works on plain objects, but also visits inherited keys. You'll see it in older code.

16.10 Arrays of objects: the shape of real data

Real apps store lists of records: expenses, tasks, users, products. Each record is an object and every object has the **same shape** (the same keys). Think of it as a table: each object is a row, each key is a column. Open `practice/expenses.html`; these are the five expenses:

JS · practice/expenses-solutions.js: the data

```
const expenses = [
  { id: 1, title: 'Rice and lentils', amount: 850,
    category: 'Food', date: '2026-09-28' },
  { id: 2, title: 'Bus pass', amount: 600,
    category: 'Transport', date: '2026-10-01' },
  { id: 3, title: 'Internet bill', amount: 1200,
    category: 'Bills', date: '2026-10-03' },
  { id: 4, title: 'Tea and snacks', amount: 240,
    category: 'Food', date: '2026-10-05' },
  { id: 5, title: 'Online course', amount: 1500,
    category: 'Learning', date: '2026-09-15' },
];
console.table(expenses);   // the Console shows them as a table
```

(index)	id	title	amount	category	date
0	1	Rice and lentils	850	Food	2026-09-28
1	2	Bus pass	600	Transport	2026-10-01
2	3	Internet bill	1200	Bills	2026-10-03
3	4	Tea and snacks	240	Food	2026-10-05
4	5	Online course	1500	Learning	2026-09-15

JS · questions you can now ask the data

```
console.log(expenses.length, expenses[1].title);
console.log(expenses.find((e) => e.id === 3).title);                  // one by id
console.log(expenses.filter((e) => e.category === 'Food').map((e) => e.title));
console.log(expenses.some((e) => e.amount > 1000));
console.log(expenses.map((e) => e.amount));
```

CONSOLE OUTPUT

```
5 Bus pass
Internet bill
['Rice and lentils', 'Tea and snacks']
true
[850, 600, 1200, 240, 1500]
```

Now the five functions of today's practice, a preview of Project 1. Each is **pure**: it takes the list, returns an answer, and never changes the list.

JS · Exercise 1 · totalAmount(list)

```
const totalAmount = (list) => list.reduce((total, e) => total + e.amount, 0);
console.log(totalAmount(expenses));
```

CONSOLE OUTPUT

```
4390
```

JS · Exercise 2 · totalsByCategory(list): reduce into an object

```
function totalsByCategory(list) {
  return list.reduce((totals, { category, amount }) => {
    totals[category] = (totals[category] ?? 0) + amount;   // 0 the first time
    return totals;
  }, {});
}
console.log(totalsByCategory(expenses));
```

CONSOLE OUTPUT

```
{Food: 1090, Transport: 600, Bills: 1200, Learning: 1500}
```

The bottom half of Figure 15.5 shows this `reduce` step by step: the accumulator starts as `{}` and gains one key the first time each category appears.

JS · Exercise 3 · sortByAmount(list, direction)

```
function sortByAmount(list, direction = 'desc') {
  const sorted = [...list].sort((a, b) => a.amount - b.amount);   // copy, small → big
  return direction === 'desc' ? sorted.reverse() : sorted;
}
console.log(sortByAmount(expenses).map((e) => e.amount));
console.log(sortByAmount(expenses, 'asc')[0].title);
console.log(expenses[0].id);                    // the original order is untouched
```

CONSOLE OUTPUT

```
[1500, 1200, 850, 600, 240]
Tea and snacks
1
```

JS · Exercise 4 · groupByMonth(list)

```
function groupByMonth(list) {
  const groups = {};
  for (const expense of list) {
    const month = expense.date.slice(0, 7);    // '2026-09-28' → '2026-09'
    if (!groups[month]) groups[month] = [];    // the first one of that month
    groups[month].push(expense);
  }
  return groups;
}
const byMonth = groupByMonth(expenses);
console.log(Object.keys(byMonth));
console.log(byMonth['2026-09'].map((e) => e.id));
console.log(totalAmount(byMonth['2026-10']));
```

▶ CONSOLE OUTPUT

```
['2026-09', '2026-10']
[1, 5]
2040
```

⇄ MORE WAYS Three ways to group a list

WAY 1 A loop and an object (above)

```
const groups = {};
for (const e of list) {
  (groups[e.category] ??= []).push(e);
}
```

`??=` assigns only if the key is still empty: a shortcut for the `if` line above.

WAY 2 reduce

```
const groups = list.reduce((acc, e) => {
  (acc[e.category] ??= []).push(e);
  return acc;
}, {});
```

The same logic as one expression.

WAY 3 Object.groupBy() (2024)

```
const groups = Object.groupBy(list, (e) => e.category);
```

Built in to all current browsers since 2024. Knowing how to write it yourself still matters: it's a common interview task.

JS · Exercise 5 · formatTaka(amount) and a summary line

```
const formatTaka = (amount) => `৳${amount.toLocaleString('en-IN')}`;
console.log(formatTaka(4390));
console.log(formatTaka(125000));             // Indian grouping: 1,25,000
const [biggest] = sortByAmount(expenses);    // destructure the first item
console.log(`Biggest expense: ${biggest.title} (${formatTaka(biggest.amount)})`);
```

▶ CONSOLE OUTPUT

```
৳4,390
৳1,25,000
Biggest expense: Online course (৳1,500)
```

⇄ MORE WAYS **Two ways to format money**

WAY 1 toLocaleString

```
(125000).toLocaleString('en-IN')   // '1,25,000'
```

Groups the digits the way people in Bangladesh and India read them (lakh, crore). Add the ৳ sign yourself.

WAY 2 Intl.NumberFormat

```
new Intl.NumberFormat('en-IN',
  { style: 'currency', currency: 'BDT' }).format(4390)   // 'BDT 4,390.00'
```

Full currency formatting, including decimals. With the `bn-BD` locale it even uses Bengali digits. You'll use this in Project 1.

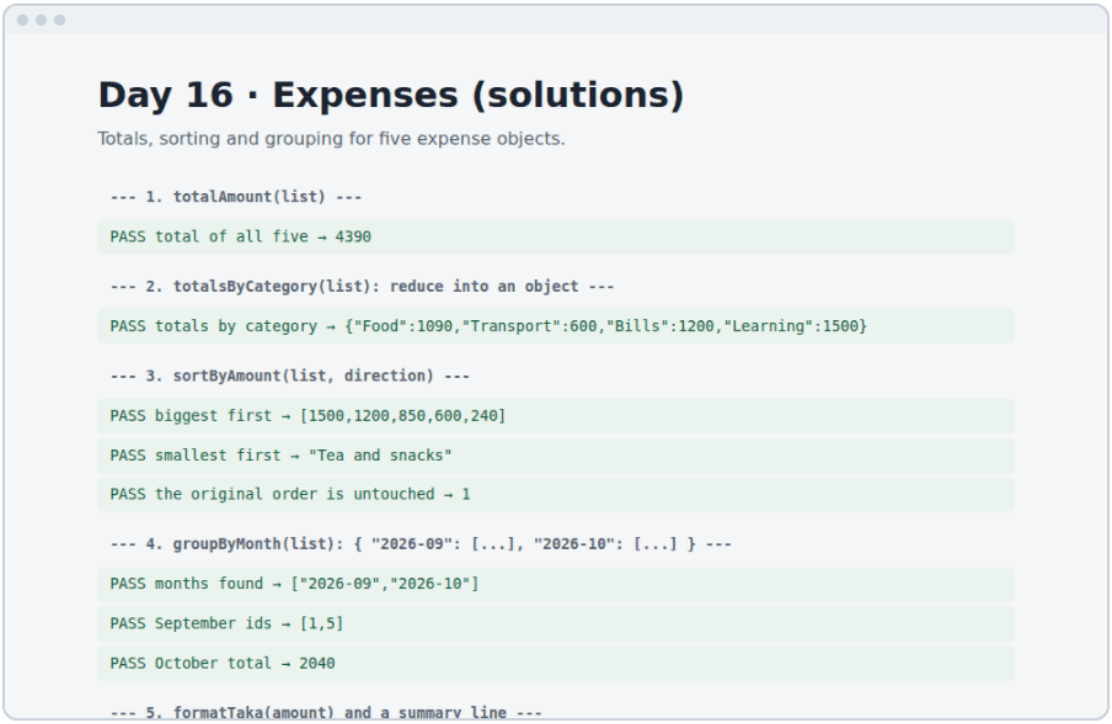


Figure 16.4 · practice/expenses-solutions.html: all five functions, tested

16.11 JSON: data as text

Objects live in your program's memory. To **save** them (in localStorage, Day 19) or **send** them (to a server, Month 3) they must become text. JSON (JavaScript Object Notation) is that text format, and every language can read it.

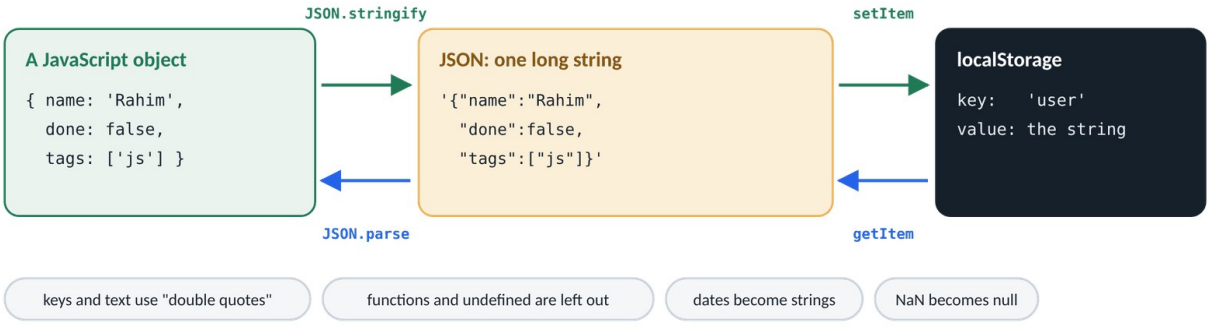


Figure 16.5 · stringify turns an object into a string; parse turns it back into a new object

JS · Part I: stringify and parse

```
const task = { name: 'Rahim', done: false, tags: ['js'] };
const text = JSON.stringify(task);           // object → string
console.log(text);
console.log(typeof text);
const back = JSON.parse(text);               // string → a NEW object
console.log(back.tags[0], back === task);
console.log(JSON.stringify(task, null, 2));  // pretty-printed, 2 spaces
```

CONSOLE OUTPUT

```
{"name":"Rahim","done":false,"tags":["js"]}
string
js false
{
  "name": "Rahim",
  "done": false,
  "tags": [
    "js"
  ]
}
```

JSON rule	Example
Keys and strings use **double** quotes	`{"name": "Rahim"}`, never `{name: 'Rahim'}`
Allowed values	strings, numbers, `true`/`false`, `null`, arrays, objects
No functions, no `undefined`	they are left out of objects
Dates become strings	`"1970-01-01T00:00:00.000Z"`: convert back yourself
`NaN` and `Infinity` become `null`	check numbers before saving
No comments, no trailing commas	`[1, 2,]` is not valid JSON

JS · what JSON can't hold, and reading broken JSON safely

```
const odd = { when: new Date(0), say() {}, missing: undefined, bad: NaN };
console.log(JSON.stringify(odd));

try {
  JSON.parse("{ name: 'Rahim' }");          // ✗ JSON needs "double quotes"
} catch (error) {
  console.log('Could not read it:', error.name);
}
```

CONSOLE OUTPUT

```
{"when":"1970-01-01T00:00:00.000Z","bad":null}
Could not read it: SyntaxError
```

`try { … } catch (error) { … }` runs the `try` part; if anything in it throws an error, JavaScript jumps to `catch` instead of crashing the whole script. Always wrap `JSON.parse` of saved data like this, because saved text can be broken. Day 20's storage module does exactly that.

> **TIP**
>
> In the Console, `copy(JSON.stringify(data, null, 2))` copies any data to your clipboard as nicely formatted JSON. Handy for bug reports and tests.

16.12 Reading nested data safely: ?. and ??

JS · optional chaining and the nullish operator

```
const user = { profile: { address: null }, settings: { volume: 0 } };
console.log(user.profile.address?.city);   // stops at null: undefined
console.log(user.billing?.plan);           // missing object: undefined
console.log(user.settings.volume ?? 50);   // ?? keeps 0
console.log(user.settings.volume || 50);   // || replaces 0
console.log(user.greet?.());               // call it only if it exists
console.log(user.billing.plan);            // ✗ without ?.
```

▶ CONSOLE OUTPUT

```
undefined
undefined
0
50
undefined
Uncaught TypeError: Cannot read properties of undefined (reading 'plan')
```

Write	Means	Gives, if the left side is null/undefined
`a?.b`	read key b, if a exists	`undefined` (no crash)
`a?.[key]`	the same, with brackets	`undefined`
`a.fn?.()`	call fn, if it exists	`undefined`
`a ?? b`	a, unless a is null/undefined	`b`
`a ??= b`	give a the value b only if a is null/undefined	(a becomes b)

WATCH OUT

Don't put `?.` everywhere "just in case". It hides real bugs: if `user` should always exist, a crash tells you something is wrong. Use `?.` where a value is **allowed** to be missing, like data from a form, a server or localStorage.

16.13 The tabs, built from data

Yesterday the four tabs were written by hand in the HTML. Today their content moves into an **array of objects**, and one function builds the buttons and panels from it. Want a fifth tab? Add one object. No HTML changes.

JS · widgets/js/main.js (Day 16), section 1: the data (shortened)

```
const tracks = [
  {
    id: 'frontend',
    title: 'Frontend',
    months: 'Months 1–2',
    summary: 'Build interfaces people enjoy using, from the first HTML tag to …',
    topics: ['Semantic HTML and CSS', 'JavaScript and the DOM', 'TypeScript', …],
  },
  { id: 'backend', title: 'Backend', months: 'Month 3', /* … */ },
  // … databases and deploy
];
```

JS · section 2: build one tab and one panel per object (shortened)

```
function renderTabs(root, items) {
  const list = root.querySelector('.tab-list');
  const panels = root.querySelector('.tab-panels');

  items.forEach((item, index) => {
    const isFirst = index === 0;

    const tab = document.createElement('button');   // a new <button>, off the page
    tab.className = 'tab';
    tab.id = `tab-${item.id}`;                       // the ids come from the data
    tab.textContent = item.title;
    tab.setAttribute('role', 'tab');
    tab.setAttribute('aria-selected', String(isFirst));
    tab.setAttribute('aria-controls', `panel-${item.id}`);
    if (!isFirst) tab.tabIndex = -1;
    list.append(tab);                                  // now it's on the page

    const panel = document.createElement('div');
    panel.id = `panel-${item.id}`;
    panel.hidden = !isFirst;                           // only the first panel shows
    // … the meta line, heading, summary and the topics (map → <li>)
    panels.append(panel);
  });
}

renderTabs(document.getElementById('tracks'), tracks);
```

Some of these lines are tomorrow's topic (the DOM). For today, focus on the **data side**: one object per tab, `forEach` over the array, the index decides which tab starts selected, and every id is built from `item.id` with a template literal.

	Day 15: HTML by hand	Day 16: built from data
Add a tab	copy 15 lines of HTML, change 8 ids	add one object to `tracks`
Typos in ids	easy to make, hard to spot	impossible: ids come from one field
Content from a server later	rewrite the HTML	replace the array; nothing else changes
Works without JavaScript	yes	no (fine for an app; a trade-off to know)

⇄ MORE WAYS **Three ways to turn the topics array into <li> items**

WAY 1 map to elements, then append them all

```
topicList.append(...item.topics.map((topic) => {
  const li = document.createElement('li');
  li.textContent = topic;
  return li;
}));
```

What the widget does: `map` makes the elements, spread passes them to `append`.

WAY 2 forEach and append one by one

```
item.topics.forEach((topic) => {
  const li = document.createElement('li');
  li.textContent = topic;
  topicList.append(li);
});
```

The same result; maybe easier to read at first.

WAY 3 innerHTML with map and join

```
topicList.innerHTML = item.topics
  .map((topic) => `<li>${topic}</li>`)
  .join('');
```

Short. Only safe for text **you** wrote, never for text a visitor typed (Day 17 shows why).

16.14 DSA: LeetCode 1672, Richest Customer Wealth

The problem. `accounts[i][j]` is the money customer `i` has in bank `j`. A customer's wealth is the sum of their row. Return the largest wealth. `[[1, 5], [7, 3], [3, 5]]` → `10` (the second customer: 7 + 3).

1. **Understand:** an array of arrays is a table. Each inner array is one customer (a row).
2. **Plan:** for each row, add up its numbers; keep the biggest total seen so far.
3. **Two loops:** an outer loop over customers, an inner loop over that customer's banks.
4. **Edge cases:** one customer; several customers with the same wealth (return the number, not who).

Customer (row)	Sum of the row	Wealth	Richest so far
`[1, 5]`	1 + 5	6	6
`[7, 3]`	7 + 3	10	**10**
`[3, 5]`	3 + 5	8	10

⇄ MORE WAYS **Richest Customer Wealth three ways (dsa/1672-richest-customer-wealth.js)**

WAY 1 **Two loops**

```
function maximumWealth(accounts) {
  let richest = 0;
  for (const customer of accounts) {
    let wealth = 0;
    for (const money of customer) {
      wealth += money;
    }
    if (wealth > richest) richest = wealth;
  }
  return richest;
}
```

The clearest. Time O(rows × columns): every number is visited once.

WAY 2 **map each row to its sum, then Math.max**

```
function maximumWealthMap(accounts) {
  const wealths = accounts.map((customer) =>
    customer.reduce((sum, money) => sum + money, 0));
  return Math.max(...wealths);
}
```

Two clear steps: "sum every row", then "take the biggest". Uses an extra array of sums.

WAY 3 **One reduce that keeps the maximum**

```
function maximumWealthReduce(accounts) {
  return accounts.reduce((richest, customer) => {
    const wealth = customer.reduce((sum, money) => sum + money, 0);
    return Math.max(richest, wealth);
  }, 0);
}
```

No extra array. A `reduce` inside a `reduce`: read the inner one first.

▶ RESULT

```
maximumWealth([[1, 2, 3], [3, 2, 1]])          → 6
maximumWealth([[1, 5], [7, 3], [3, 5]])        → 10
maximumWealth([[2, 8, 7], [7, 1, 3], [1, 9, 5]]) → 17
```

How fast? Every number is added once, so the work grows with the total number of cells: **O(m × n)** for m customers and n banks. You can't do better, because you must look at every number at least once.

Build it, step by step

1. Open `practice/objects.html` and read `objects-demo.js` part by part (A–I). Predict every PASS line before you read it.
2. Open `practice/expenses.html` and write the five functions in `expenses-starter.js` until every test passes.
3. In the widgets, move the tab content into a `tracks` array of objects and empty the tabs in the HTML.
4. Write `renderTabs(root, items)` and call it. The page should look exactly like yesterday's.
5. Add a fifth track object as a test, check it appears, then remove it.
6. Solve **Richest Customer Wealth**, then compare with `dsa/1672-richest-customer-wealth.js`.

7. Commit: `git commit -m "Day 16: objects, expense utilities and data-driven tabs"` and push.

★ MUST KNOW *the ideas to remember from today*

- ✓ Dot for known keys; brackets for keys in variables or with spaces.
- ✓ A missing key gives `undefined`; `in` and `Object.hasOwn` check whether a key exists.
- ✓ In a method, `this` is the object before the dot. Don't write methods as arrow functions.
- ✓ Destructuring: objects by name `{ a, b }`, arrays by position `[x, y]`.
- ✓ `{ ...obj, key: value }` copies and changes one key without touching the original.
- ✓ `const b = a` shares the object. Spread is a shallow copy; `structuredClone` is deep.
- ✓ `Object.keys / values / entries` turn objects into arrays; `fromEntries` goes back.
- ✓ `JSON.stringify` → string, `JSON.parse` → new object. Wrap `parse` in `try/catch`.

WATCH OUT

- `obj.key` when you meant `obj[key]` (a variable).
- Arrow functions as methods: `this` isn't the object.
- Thinking `{ ...a }` copies the arrays inside `a`: it doesn't.
- Comparing objects with `===` to check their content.
- Forgetting `return totals` in a `reduce` that builds an object.
- JSON with single quotes or trailing commas.
- Using `?.` everywhere and hiding real bugs.

QUESTIONS LEARNERS ASK

Q Can a key be a number?

You can write `{ 1: 'a' }`, but keys are always stored as strings: `Object.keys({ 1: 'a' })` is `['1']`. For keys of any type, JavaScript has `Map` (Day 19).

Q What order are keys in?

Normal text keys stay in the order you added them. Keys that look like whole numbers come first, sorted. Don't rely on order for logic; use an array when order matters.

Q Is JSON the same as a JavaScript object?

No. JSON is **text** that looks like an object literal, with stricter rules. `JSON.parse` turns the text into a real object.

Q When do I need a deep copy?

When the object has arrays or objects inside it and you will change those inner values in the copy. For flat objects, spread is enough.

More practice

MORE PRACTICE · try each one, then check the Answers at the back

1. Create an object `book` with a title, an author, a year and a method `age()` that returns how many years old it is in 2026.
2. Given `const scores = { math: 80, english: 72, physics: 91 }`, find the subject with the highest score.
3. Swap the keys and values of `{ a: 1, b: 2 }` to get `{ 1: 'a', 2: 'b' }`.
4. Remove the `password` key from a user object without changing the original.
5. Count how many expenses each category has: `{ Food: 2, Transport: 1, … }`.
6. What does `JSON.parse(JSON.stringify({ a: [1, 2] }))` give, and is it a deep copy?
7. Write `pick(obj, keys)` so that `pick({ a: 1, b: 2, c: 3 }, ['a', 'c'])` gives `{ a: 1, c: 3 }`.

Check yourself

1. When must you use brackets instead of a dot to read a key?
2. After `const copy = { ...original }`, why can changing `copy.tags` still change `original`?
3. Why do you need `JSON.stringify` before saving an object, and what does `JSON.parse` give back?

*Answer out loud before you turn to the **Answers** section at the back.*

DONE WHEN · tick these off, then mark the day done in your Logbook

- ☐ You can predict every line of `objects-demo.js`
- ☐ All expense tests pass with your own code
- ☐ The tabs are built from the `tracks` array and look the same as yesterday
- ☐ Richest Customer Wealth accepted on LeetCode; committed and pushed

Resources

All free. Every name is a link, in Word and in the PDF.

Resource (click to open)	Use it for
javascript.info: Objects javascript.info/object	The basics, with small tasks
javascript.info: Object references and copying javascript.info/object-copy	References, shallow and deep copies
javascript.info: Object methods, "this" javascript.info/object-methods	Methods and this, step by step
javascript.info: Destructuring assignment javascript.info/destructuring-assignment	Every destructuring form, with tasks

Resource (click to open)	Use it for
javascript.info: Object.keys, values, entries javascript.info/keys-values-entries	Turning objects into arrays and back
javascript.info: JSON methods, toJSON javascript.info/json	stringify, parse and their options
javascript.info: Optional chaining javascript.info/optional-chaining	?. in all its forms
MDN: Working with objects developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Working_with_objects	The official guide
MDN: structuredClone() developer.mozilla.org/en-US/docs/Web/API/Window/structuredClone	Deep copies, and what can't be copied
MDN: Nullish coalescing operator (??) developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Nullish_coalescing	?? vs ||, with examples
LeetCode 1672: Richest Customer Wealth leetcode.com/problems/richest-customer-wealth	Today's problem

DAY
17

WEEK 3 · MONDAY 12 OCTOBER

The DOM: select, create and update the page

The page is a tree of objects. Today you learn to find any branch and change it.

TIME	YOU BUILD	CODE FOLDER
3 hours	**A DOM playground with five exercises, working tabs, and Contains Duplicate**	`day17-dom`

Today you will

- ✓ Explain the DOM as a tree of objects the browser builds from your HTML
- ✓ Find elements with `querySelector`, `querySelectorAll`, `getElementById` and `closest`
- ✓ Read and change text, HTML, form values, attributes and `data-*` attributes
- ✓ Add, remove and toggle classes, and set inline styles when you must
- ✓ Create, insert, move and remove elements
- ✓ Choose `textContent` over `innerHTML` for anything a person typed, and explain XSS
- ✓ Render a list from an array, and re-render when the array changes
- ✓ Make the tabs switch with `aria-selected`, `tabIndex` and `hidden`
- ✓ Solve LeetCode 217, Contains Duplicate, three ways

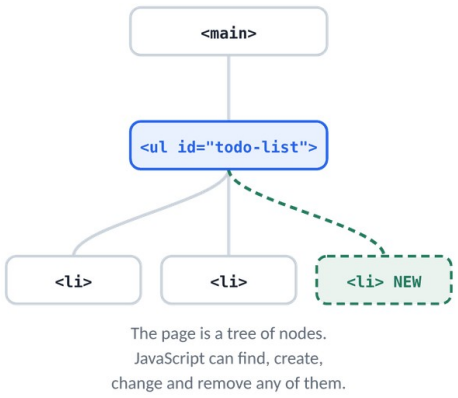
Key terms

Term	Meaning
DOM	Document Object Model: the page as a tree of objects that JavaScript can change
Node / element	One point in the tree; an element is a node made from a tag
Parent / child / sibling	The element around it / inside it / next to it
Selector	A CSS pattern such as `#save`, `.item` or `ul > li`
NodeList	The list `querySelectorAll` returns: has `forEach`, not `map`
textContent	An element's text, read or written as plain text
innerHTML	An element's content as HTML: writing it creates elements
Attribute	`id`, `class`, `aria-*`, `data-*`... written in the HTML tag
classList	The object for adding, removing and toggling classes
XSS	Cross-site scripting: a visitor's text becomes code that runs on your page
Render	Build (or rebuild) part of the page from data

17.1 The DOM is a tree

IN PLAIN WORDS

When the browser reads your HTML, it builds a **family tree** of objects: `<html>` is the great-grandparent, `<body>` is its child, and every tag inside is a child, grandchild or sibling. JavaScript can't change your HTML file, but it can change this living tree, and the browser redraws the page straight away.



1. `const list = document.querySelector('#todo-list');` — find it (the blue node)
2. `const li = document.createElement('li');` — create a node (not on the page yet)
3. `li.textContent = 'Solve Two Sum';` — set its text (safe for any text)
4. `li.classList.add('todo-item');  li.dataset.id = 't4';` — add a class and a data-id attribute
5. `list.append(li);` — attach it: now it appears on the page
6. `li.remove();` — take it off the page again

Figure 17.1 · The page as a tree, and the six operations you'll use every day

All the examples in this chapter run on this small page. Type it into `practice/dom-lesson.html` (or paste it into any page's `<body>` in DevTools) and try each example in the Console.

HTML · the HTML used by the examples in this chapter

```
<main id="app">
  <h1 class="title">Stackfield</h1>
  <ul id="todo-list" class="list">
    <li class="item" data-id="t1">Read the arrays chapter</li>
    <li class="item done" data-id="t2">Solve Two Sum</li>
    <li class="item" data-id="t3">Build the widgets</li>
  </ul>
  <button id="save" class="button" type="button" aria-pressed="false">Save</button>
  <input id="name" value="Rahim">
</main>
```

JS · walking the tree

```
const list = document.querySelector('#todo-list');
console.log(list.tagName, list.id, list.className);
console.log(list.children.length);                  // the three <li> inside
console.log(list.firstElementChild.textContent);    // go down
console.log(list.parentElement.id);                 // go up
console.log(list.children[1].nextElementSibling.textContent);   // go sideways
```

▶ CONSOLE OUTPUT

```
UL todo-list list
3
Read the arrays chapter
app
Build the widgets
```

Family word	Gives you
`el.parentElement`	the element around it
`el.children`	the elements directly inside it (a live list)
`el.firstElementChild` / `lastElementChild`	the first / last child element
`el.nextElementSibling` / `previousElementSibling`	the neighbour after / before it
`el.closest('.card')`	itself or the nearest ancestor that matches (used a lot on Day 18)

> **TIP**
>
> In DevTools → **Elements**, click any element, then type `$0` in the Console: it's that element. `$0.textContent = 'Hi'` changes it on the spot. It's the fastest way to experiment.

17.2 Finding elements

JS · selecting elements

```
const title = document.querySelector('.title');              // the first match
const items = document.querySelectorAll('#todo-list .item');  // ALL matches
const save = document.getElementById('save');                // by id (no #)
console.log(title.textContent, items.length, save.textContent);
console.log(document.querySelector('.done').dataset.id);
console.log(document.querySelector('.missing'));             // no match: null

items.forEach((item) => console.log(item.dataset.id));       // NodeList has forEach…
console.log(items.map);                                       // …but not map
const texts = [...items].map((item) => item.textContent);    // spread → a real array
console.log(texts);
```

▶ CONSOLE OUTPUT

```
Stackfield 3 Save
t2
null
t1
t2
t3
undefined
['Read the arrays chapter', 'Solve Two Sum', 'Build the widgets']
```

Selector	Finds
`'#save'`	the element with `id="save"`
`'.item'`	elements with the class `item`
`'li'`	every `<li>`
`'#todo-list .item'`	`.item` anywhere inside `#todo-list`
`'ul > li'`	`<li>` that are direct children of a `<ul>`
`'[data-id="t2"]'`	the element whose `data-id` is `t2`
`'.item:not(.done)'`	`.item` elements without the class `done`
`'button[aria-pressed="true"]'`	pressed toggle buttons

⇄ MORE WAYS Four ways to find the same button

WAY 1 getElementById

```
document.getElementById('save')
```

The fastest, and clear. Only for ids, and no `#`.

WAY 2 querySelector with an id

```
document.querySelector('#save')
```

Any CSS selector works, so one method covers everything.

WAY 3 querySelector with an attribute

```
document.querySelector('[aria-pressed]')
```

Useful when elements have no id or class to hook onto.

WAY 4 Search inside a parent

```
const app = document.getElementById('app');
app.querySelector('button')
```

`el.querySelector` only looks **inside** `el`. Widgets use this so two copies on one page don't find each other's parts.

JS · the error you'll meet most this week

```
const heading = document.querySelector('#titel');   // a typo in the selector
heading.textContent = 'Hello';
```

▶ CONSOLE OUTPUT

```
Uncaught TypeError: Cannot set properties of null (setting 'textContent')
```

WATCH OUT

"Cannot read / set properties of **null**" means the selector found nothing. Check three things: a typo in the selector, a missing `#` or `.`, and whether the script ran before the HTML existed (use `defer`, as on Day 10).

17.3 Reading and changing content

JS · text, HTML and form values

```
const title = document.querySelector('.title');
console.log(title.textContent);
title.textContent = 'Stackfield widgets';        // replace the text
console.log(title.textContent);

const first = document.querySelector('.item');
first.innerHTML = 'Read the <strong>arrays</strong> chapter';   // parsed as HTML
console.log(first.textContent);                  // the text only, tags removed
console.log(first.children.length);              // one <strong> was created

const nameInput = document.getElementById('name');
console.log(nameInput.value);                    // form fields: .value
nameInput.value = 'Nadia';
console.log(nameInput.value);
```

▶ CONSOLE OUTPUT

```
Stackfield
Stackfield widgets
Read the arrays chapter
1
Rahim
Nadia
```

Property	Reads / writes	Use it for
`textContent`	plain text; writing replaces everything inside	showing any text (your default)
`innerHTML`	HTML; writing creates elements	HTML **you** wrote, never visitor text
`innerText`	only the text you can see (respects CSS)	rarely; it's slower
`value`	what's in an `<input>`, `<select>` or `<textarea>`	every form field

17.4 Attributes, properties and data-*

JS · attributes, properties and data-*

```
const save = document.getElementById('save');
console.log(save.getAttribute('aria-pressed'));   // attributes are always strings
save.setAttribute('aria-pressed', 'true');
console.log(save.getAttribute('aria-pressed'));
console.log(save.id, save.type);                  // common ones are properties too
save.disabled = true;                             // a true/false attribute
console.log(save.hasAttribute('disabled'));
save.hidden = true;
console.log(save.hidden, save.hasAttribute('hidden'));

const item = document.querySelector('[data-id="t2"]');
console.log(item.dataset.id);                     // data-id → dataset.id
item.dataset.priority = 'high';                   // adds data-priority="high"
console.log(item.getAttribute('data-priority'));
```

▶ CONSOLE OUTPUT

```
false
true
save button
true
true true
t2
high
```

In the HTML	In JavaScript	Value
`data-id="t2"`	`el.dataset.id`	`'t2'` (a string)
`data-user-name="Mitu"`	`el.dataset.userName`	dashes become camelCase
`data-count="3"`	`Number(el.dataset.count)`	convert numbers yourself
`hidden`, `disabled`, `checked`	`el.hidden = true`	a real boolean
`aria-selected="true"`	`el.getAttribute('aria-selected')`	`'true'` (a string)

WATCH OUT

`getAttribute('aria-pressed')` returns the **string** `'false'`, and every non-empty string is truthy. `if (save.getAttribute('aria-pressed'))` is always true. Compare instead: `=== 'true'`.

`data-*` attributes let you store your own small facts on an element: which task a row shows (`data-id`), which filter a button sets (`data-filter`), which size a button picks (`data-size`). Tomorrow's events read them to know **what** was clicked.

17.5 Classes and styles

JS · classList and style

```
const item = document.querySelector('.item');
item.classList.add('done');                    // add a class
item.classList.remove('item');                 // remove one
console.log(item.className);
console.log(item.classList.contains('done'));
console.log(item.classList.toggle('done'));    // was on → now off: false
console.log(item.classList.toggle('done'));    // on again: true
item.classList.toggle('urgent', 3 > 5);        // on only if the 2nd argument is true
console.log(item.classList.contains('urgent'));

item.style.color = 'crimson';                  // an inline style: camelCase names
item.style.backgroundColor = '#fff4e5';
console.log(item.getAttribute('style'));
```

CONSOLE OUTPUT

```
done
true
false
true
false
color: crimson; background-color: rgb(255, 244, 229);
```

classList method	Does
`add('a', 'b')`	adds one or more classes (no duplicates)
`remove('a')`	removes a class (no error if it isn't there)
`toggle('a')`	adds if missing, removes if present; returns `true` if it's now on
`toggle('a', condition)`	on if `condition` is true, off if false: perfect for state
`contains('a')`	`true` / `false`
`replace('a', 'b')`	swaps one class for another

⇄ MORE WAYS Three ways to change how an element looks

WAY 1 A class (use this most)

```
row.classList.toggle('is-done', todo.done);
```

The look stays in the CSS; JavaScript only switches it on or off. Easy to change the design later.

WAY 2 An attribute the CSS also reads

```
tab.setAttribute('aria-selected', 'true');
/* CSS: .tab[aria-selected="true"] { … } */
```

One attribute does two jobs: screen readers hear the state, and the CSS styles it. The tabs work like this.

WAY 3 An inline style (for values made in JS)

```
bar.style.width = `${percent}%`;
card.style.setProperty('--accent', colour);
```

For numbers JavaScript calculates: a progress bar, a position, a colour the user picked. Not for on/off states.

JS · dom-playground.js, section 2: dark mode is one class on <body>

```
const themeButton = document.querySelector('#theme-button');
themeButton.addEventListener('click', () => {
  const isDark = document.body.classList.toggle('dark');   // true if it's now on
  themeButton.setAttribute('aria-pressed', String(isDark));
  themeButton.textContent = `Dark mode: ${isDark ? 'on' : 'off'}`;
});
```

17.6 Creating, inserting and removing elements

JS · create, insert and remove

```
const list = document.getElementById('todo-list');
const li = document.createElement('li');      // 1. create: not on the page yet
li.className = 'item';
li.textContent = 'Deploy on Vercel';          // 2. fill it in
li.dataset.id = 't4';
list.append(li);                              // 3. attach it: now it's visible
console.log(list.children.length, list.lastElementChild.textContent);

const heading = document.createElement('li');
heading.textContent = 'This week';
list.prepend(heading);                        // at the start instead
console.log(list.firstElementChild.textContent);

list.querySelector('[data-id="t2"]').remove();   // take one off the page
console.log([...list.children].map((el) => el.dataset.id ?? '-'));
list.replaceChildren();                       // remove EVERYTHING inside
console.log(list.children.length);
```

CONSOLE OUTPUT

```
4 Deploy on Vercel
This week
['-', 't1', 't3', 't4']
0
```

Method	Puts the new element…
`parent.append(a, b, 'text')`	at the end, inside the parent (several at once)
`parent.prepend(a)`	at the start, inside the parent
`el.before(a)` / `el.after(a)`	just before / after `el`, as its sibling
`el.replaceWith(a)`	in place of `el`
`el.remove()`	(takes `el` off the page)
`parent.replaceChildren(a, b)`	removes everything inside, then adds a and b

Appending an element that is **already** on the page moves it rather than copying it. To copy an element, use `el.cloneNode(true)` (`true` = with everything inside).

⇄ MORE WAYS **Three ways to add a new list item**

WAY 1 createElement + textContent + append

```
const li = document.createElement('li');
li.textContent = task;
list.append(li);
```

Safe for any text, including what a visitor typed. The widgets use this.

WAY 2 insertAdjacentHTML

```
list.insertAdjacentHTML('beforeend', '<li class="item">Fixed text</li>');
```

Quick for HTML you wrote. Positions: `beforebegin`, `afterbegin`, `beforeend`, `afterend`. Same XSS danger as `innerHTML`.

WAY 3 A <template> and cloneNode

```
// HTML: <template id="row"><li class="item"><span></span></li></template>
const row = document.getElementById('row').content.cloneNode(true);
row.querySelector('span').textContent = task;
list.append(row);
```

The structure stays in the HTML where it's easy to read; JavaScript fills in the text. Nice for rows with many parts.

17.7 textContent vs innerHTML: why it matters

JS · textContent shows tags; innerHTML builds them

```
const title = document.querySelector('.title');
title.textContent = '<b>Rahim</b>';
console.log(title.children.length, title.textContent);
title.innerHTML = '<b>Rahim</b>';
console.log(title.children.length, title.textContent);
```

▶ CONSOLE OUTPUT

```
0 <b>Rahim</b>
1 Rahim
```



Figure 17.2 · The same visitor text, shown safely with textContent and dangerously with innerHTML

Now imagine a comment box that shows comments with `innerHTML`. A visitor types `<img src="x" onerror="…">`. The browser creates a real image; it fails to load; its `onerror` code **runs on your page**, for everyone who views the comment. It could read login tokens, change the page or send data elsewhere. This attack is called **XSS** (cross-site scripting), and it's one of the most common security holes on the web.

★ **MUST KNOW** *the ideas to remember from today*

- ✓ Text from a person (a form, a URL, a database, an API) → `textContent`.
- ✓ `innerHTML` only with HTML **you** wrote in your own code.
- ✓ If you ever must show visitor HTML (a rich-text editor), clean it first with a tested library such as DOMPurify.

TRY IT

Open `practice/dom-playground.html`, section 4. The box already contains `<b>Rahim</b>`. Compare the two outputs, then type `<i>hi</i>` and watch the innerHTML line turn italic. That's the first step of an XSS attack, safely in your own page.

17.8 Render a list from an array

IN PLAIN WORDS

The array is the **recipe** and the list on the page is the **cake**. You don't fix a cake by sticking bits on; you change the recipe and bake again. In code: change the array, then call one `render()` function that rebuilds the list from it.

JS · render(): draw the list from the array

```
const topics = ['Arrays', 'Objects'];
const list = document.getElementById('todo-list');

function render() {
  const items = topics.map((topic) => {
    const li = document.createElement('li');
    li.textContent = topic;
    return li;
  });
  list.replaceChildren(...items);        // swap the old <li>s for the new ones
}

render();
console.log(list.children.length);
topics.push('The DOM');                  // 1. change the data
render();                                // 2. draw again from the data
console.log([...list.children].map((li) => li.textContent));
```

▶ CONSOLE OUTPUT

```
2
['Arrays', 'Objects', 'The DOM']
```

This "change the data, then render" loop is the most important pattern of the week. The to-do list (Day 19) and Project 1 are built on it, and in Month 2 React does exactly this for you, automatically.

⇄ MORE WAYS **Three ways to render a list**

WAY 1 map to elements + replaceChildren (use this)

```
list.replaceChildren(...topics.map(createItem));
```

Safe text, and one call replaces everything. `createItem` is a function that returns one `<li>`.

WAY 2 Empty it, then append in a loop

```
list.textContent = '';          // quick way to empty an element
for (const topic of topics) {
  list.append(createItem(topic));
}
```

The same result in steps; fine and common.

WAY 3 innerHTML from map + join

```
list.innerHTML = topics.map((t) => `<li>${t}</li>`).join('');
```

Shortest, but only for your own fixed data (XSS again). You'll see it in many tutorials.

The counter: the same idea with a number

JS · dom-playground.js, section 3

```
let count = 0;                                    // the state lives in a variable…
const countOutput = document.querySelector('#count');
const minusButton = document.querySelector('#minus');

function renderCount() {                          // …and the page shows it
  countOutput.textContent = count;
  minusButton.disabled = count === 0;             // no negative numbers
}
const plus = document.querySelector('#plus');
const reset = document.querySelector('#reset');
plus.addEventListener('click', () => { count++; renderCount(); });
minusButton.addEventListener('click', () => { count--; renderCount(); });
reset.addEventListener('click', () => { count = 0; renderCount(); });
renderCount();
```

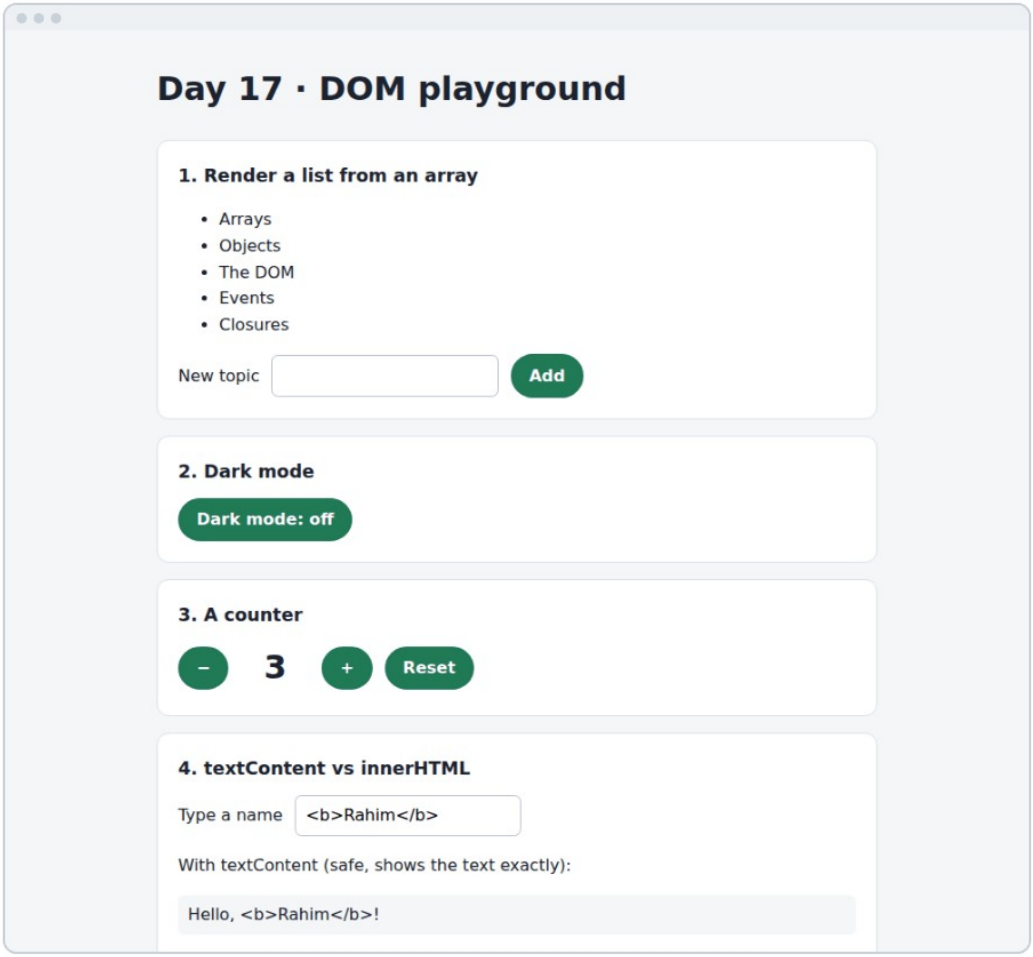


Figure 17.3 · practice/dom-playground.html: a list from an array, dark mode, a counter and the textContent test

17.9 Making the tabs work

The tabs were built from data yesterday. Today one function, `selectTab`, switches them: the chosen tab gets `aria-selected="true"` and its panel is shown; every other tab and panel is switched off. The CSS already styles `[aria-selected="true"]`, so no styling code is needed.

JS · selectTab() and one listener per tab (Day 17)

```
function selectTab(root, selectedTab) {
  const tabs = root.querySelectorAll('[role="tab"]');
  tabs.forEach((tab) => {
    const isSelected = tab === selectedTab;          // is this the clicked one?
    tab.setAttribute('aria-selected', String(isSelected));
    tab.tabIndex = isSelected ? 0 : -1;              // only it is in the Tab order
    // aria-controls holds the id of this tab's panel
    const panel = document.getElementById(tab.getAttribute('aria-controls'));
    panel.hidden = !isSelected;
  });
}

const root = document.getElementById('tracks');
root.querySelectorAll('[role="tab"]').forEach((tab) => {
  tab.addEventListener('click', () => selectTab(root, tab));   // one listener per tab
});

document.getElementById('tab-backend').click();      // .click() = a pretend click
const panels = root.querySelectorAll('[role="tabpanel"]');
console.log([...panels].map((panel) => panel.hidden));
```

CONSOLE OUTPUT

```
[true, false, true, true]
```

On every tab	Selected tab	Other tabs
`aria-selected`	`"true"`: styled, and announced as "selected"	`"false"`
`tabIndex`	`0`: reached with the Tab key	`-1`: skipped by Tab (arrows, Day 18)
its panel's `hidden`	`false`: shown	`true`: hidden from eyes and screen readers

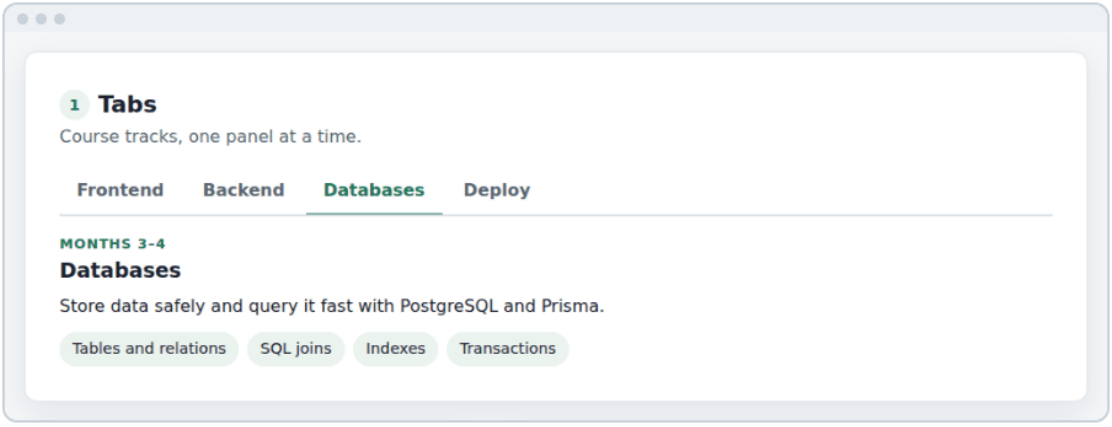


Figure 17.4 · The tabs widget with "Databases" selected

TIP

Buttons already work with the keyboard: Tab to the selected tab and press **Enter** or **Space**, and the same click listener runs. That's one reason every clickable thing should be a real `<button>`, not a `<div>`.

Four tabs means four listeners here. Tomorrow you'll replace them with **one** listener on the tab list (event delegation), and add the arrow keys.

17.10 DSA: LeetCode 217, Contains Duplicate

The problem. Return `true` if any value appears at least twice in the array, `false` if every value is different. `[1, 2, 3, 1]` → `true`; `[1, 2, 3, 4]` → `false`.



Figure 17.5 · The Set remembers every number it has passed; the second 1 is found at step 4

1. **The obvious way:** compare every pair of numbers. Correct, but slow: 10,000 numbers means about 50 million comparisons.
2. **Sorting helps:** after sorting, equal numbers sit next to each other, so you only compare neighbours.
3. **The fast way:** walk once and keep a Set of the numbers you've seen. Before adding a number, ask the Set: "seen it?" A Set answers instantly.
4. **Stop early:** return `true` the moment you find a repeat; only return `false` after the whole array.

⇄ MORE WAYS Contains Duplicate, four ways (dsa/217-contains-duplicate.js)

WAY 1 Compare every pair

```
function containsDuplicatePairs(nums) {
  for (let i = 0; i < nums.length; i++) {
    for (let j = i + 1; j < nums.length; j++) {   // j starts after i
      if (nums[i] === nums[j]) return true;
    }
  }
  return false;
}
```

Time **O(n²)**: double the input, four times the work. LeetCode's big tests time out.

WAY 2 Sort a copy, then check neighbours

```
function containsDuplicateSort(nums) {
  const sorted = [...nums].sort((a, b) => a - b);
  for (let i = 1; i < sorted.length; i++) {
    if (sorted[i] === sorted[i - 1]) return true;
  }
  return false;
}
```

Time **O(n log n)**, the cost of sorting. No extra memory besides the copy.

WAY 3 A Set of the values seen so far

```
function containsDuplicate(nums) {
  const seen = new Set();
  for (const n of nums) {
    if (seen.has(n)) return true;   // met this value before
    seen.add(n);
  }
  return false;
}
```

Time **O(n)**: one pass. Uses O(n) extra memory for the Set. The answer interviewers hope for.

WAY 4 One line

```
const containsDuplicateShort = (nums) => new Set(nums).size !== nums.length;
```

A Set drops duplicates, so it's smaller than the array if there were any. Neat, but it can't stop early.

Way	Time	For 10,000 numbers	Extra memory
Every pair	O(n²)	~50,000,000 checks	none
Sort first	O(n log n)	~130,000 steps	the sorted copy
Set	O(n)	~10,000 steps	the Set

▶ RESULT

```
containsDuplicate([1, 2, 3, 1])                → true
containsDuplicate([1, 2, 3, 4])                → false
containsDuplicate([1, 1, 1, 3, 3, 4, 3, 2, 4, 2]) → true
```

This is your first **time vs memory trade-off**: the Set makes the code much faster by using extra memory to remember things. You'll make this trade again in Valid Anagram (Day 18) and Two Sum (Day 19).

Build it, step by step

1. Make `practice/dom-lesson.html` with the HTML from 17.1 and try every example in this chapter in the Console.
2. Open `practice/dom-playground.html`. Try each section; in section 4, compare the two outputs.
3. Close the files and rebuild the counter from memory in a new page: state variable, `renderCount()`, three listeners.
4. In the widgets, write `selectTab(root, tab)` and add a click listener to each tab.
5. Test with the mouse, then the keyboard (Tab to a tab, Enter). Check `aria-selected` change in the Elements panel.
6. Solve **Contains Duplicate**, then compare with `dsa/217-contains-duplicate.js`.
7. Commit: `git commit -m "Day 17: DOM playground and working tabs"` and push.

★ MUST KNOW *the ideas to remember from today*

- ✓ The DOM is the live tree the browser builds from your HTML; changing it changes the page.
- ✓ `querySelector` → first match or `null`; `querySelectorAll` → a NodeList (`forEach` yes, `map` no).
- ✓ `textContent` for text, `value` for form fields, `innerHTML` only for your own HTML.
- ✓ `classList.add / remove / toggle(name, force) / contains`.
- ✓ `dataset.someName` reads and writes `data-some-name`.
- ✓ Create → fill → `append`. `remove()` takes it off; `replaceChildren()` clears.
- ✓ Change the data, then `render()` from the data.
- ✓ Attributes are strings: compare `getAttribute(…) === 'true'`.

WATCH OUT

- A selector typo gives `null`, then "Cannot read properties of null".
- Forgetting `#` or `.` in `querySelector`.
- Calling `map` on a NodeList (spread it first).
- `innerHTML` with text a person typed (XSS).
- Setting styles one by one in JavaScript instead of toggling a class.
- Script in `<head>` without `defer`: the elements don't exist yet.

QUESTIONS LEARNERS ASK

Q `getElementById` or `querySelector`?

Both are fine. `getElementById` is a little faster and only for ids; `querySelector` takes any CSS selector. Pick one style for a project.

Q Is `document.body.innerHTML = …` a fast way to build a page?

It works, but it throws away every element (and every event listener) inside and rebuilds them. Change only the part that changed.

Q Why does `el.style.color` return `''` when my CSS makes it red?

`el.style` only shows **inline** styles. To read the final style from all your CSS, use `getComputedStyle(el).color`.

Q Is changing the DOM slow?

Each change can make the browser recalculate the layout. Building elements first and adding them in one `append(...items)` or `replaceChildren(...)` call is kinder than 100 separate appends.

More practice

MORE PRACTICE · try each one, then check the Answers at the back

1. Log the text of every `<li>` on a page in one line.
2. Add the class `highlight` to every even item in a list (2nd, 4th, …).
3. Write `el(tag, text, className)` that creates an element in one call.
4. Build a `<table>` from `expenses` (Day 16) with a row per expense and cells for title and amount.
5. Make the counter never go above 10 and disable + at 10.
6. Move the first item of a list to the end with one line.

Check yourself

1. What does `querySelectorAll` return, and why can't you call `map` on it straight away?
2. Why is `element.innerHTML = userInput` dangerous, and what should you use instead?
3. What three things does `selectTab` change on each tab and panel?

*Answer out loud before you turn to the **Answers** section at the back.*

DONE WHEN · tick these off, then mark the day done in your Logbook

- ☐ Every example in this chapter runs in your Console
- ☐ All five playground sections work, and you rebuilt the counter from memory
- ☐ The tabs switch with the mouse and with Enter
- ☐ Contains Duplicate accepted; committed and pushed

Resources

All free. Every name is a link, in Word and in the PDF.

Resource (click to open)	Use it for
javascript.info: Walking the DOM javascript.info/dom-navigation	parents, children and siblings
javascript.info: Searching: getElement*, querySelector* javascript.info/searching-elements-dom	Every way to find elements
javascript.info: Modifying the document javascript.info/modifying-document	create, append, remove, insertAdjacentHTML
javascript.info: Styles and classes javascript.info/styles-and-classes	classList, style and getComputedStyle
MDN: Document Object Model (DOM) developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model	What the DOM is, and every part of it
MDN: Element.classList developer.mozilla.org/en-US/docs/Web/API/Element/classList	All the classList methods
MDN: HTMLElement.dataset developer.mozilla.org/en-US/docs/Web/API/HTMLElement/dataset	data-* attributes from JavaScript
MDN: Node.textContent developer.mozilla.org/en-US/docs/Web/API/Node/textContent	textContent vs innerText vs innerHTML
OWASP: Cross Site Scripting (XSS) owasp.org/www-community/attacks/xss	How XSS attacks work, from the security community
JavaScript30 javascript30.com	30 free small projects with the DOM and events
LeetCode 217: Contains Duplicate leetcode.com/problems/contains-duplicate	Today's problem

DAY
18

WEEK 3 · TUESDAY 13 OCTOBER

Events in depth: bubbling, delegation and an accessible modal

Pages come alive when they react. Today you learn how an event travels, and how to catch it well.

TIME	YOU BUILD	CODE FOLDER
3 hours	**A character counter, a delegated list, an event lab, the modal dialog and keyboard tabs, and Valid Anagram**	`day18-events`

Today you will

- ✓ Attach and remove listeners, and run a listener only once
- ✓ Read the event object: `type`, `target`, `currentTarget`, `key`
- ✓ Choose the right event: `click`, `input`, `change`, `submit`, `keydown`, `focusout`
- ✓ Explain capture and bubbling, and why you should rarely stop an event
- ✓ Handle a whole list with one delegated listener and `closest()`
- ✓ Stop the browser's own action with `preventDefault()`
- ✓ Read a form with `FormData`, and let HTML check the input first
- ✓ Build an accessible modal with `<dialog>` and `showModal()`
- ✓ Move between tabs with the arrow keys
- ✓ Solve LeetCode 242, Valid Anagram, three ways

Key terms

Term	Meaning
Event	Something that happens: a click, a key press, a form submit
Listener (handler)	The function the browser calls when the event happens
Event object	The details of one event, passed to your listener as `event`
target	The element where the event started (what was actually clicked)
currentTarget	The element whose listener is running right now
Bubbling	After the target, the event travels up through every ancestor
Capture	The trip down from `document` to the target, before bubbling
Delegation	One listener on a parent handles events from all its children
Default action	What the browser does by itself: follow a link, submit a form
Modal	A dialog that blocks the rest of the page until it closes
Focus	Which element receives the keyboard; shown by the focus ring

18.1 Events and listeners

IN PLAIN WORDS

You don't stand at the front door all day waiting for guests. You fit a **doorbell** and decide: "when it rings, open the door". `addEventListener` fits the doorbell: it tells the browser which event to wait for and which function to run when it happens.

JS · adding, removing and once

```
const save = document.getElementById('save');
function handleSave(event) {
  console.log('clicked:', event.type, event.target.id);
}
save.addEventListener('click', handleSave);          // the function, not a call
save.addEventListener('click', () => console.log('a second listener'));
save.click();                                        // .click() = a pretend click

save.removeEventListener('click', handleSave);       // needs the SAME function
save.click();

save.addEventListener('click', () => console.log('only once'), { once: true });
save.click();
save.click();
```

▶ CONSOLE OUTPUT

```
clicked: click save
a second listener
a second listener
a second listener
only once
a second listener
```

Event	Fires when...	Typical use
`click`	a button or link is activated (mouse, Enter, Space, touch)	buttons, tabs, menus
`input`	the value of a field changes, on every key	live search, character counters
`change`	a field is changed and "committed" (leave the box, tick, pick)	checkboxes, selects
`submit`	a form is sent (Enter or a submit button)	every form
`keydown` / `keyup`	a key goes down / up	shortcuts, arrow keys, Escape
`focusin` / `focusout`	an element gains / loses focus (these bubble)	saving an edit, validation
`pointerenter` / `pointerleave`	the mouse or finger enters / leaves	hover effects CSS can't do
`DOMContentLoaded`	the HTML has been read	not needed with `defer` or modules

⇄ MORE WAYS **Three ways to run code on a click**

WAY 1 addEventListener (use this)

```
button.addEventListener('click', handleClick);
```

Any number of listeners per event; can be removed; options such as `{ once: true }`.

WAY 2 The onclick property

```
button.onclick = handleClick;
```

Only **one** handler: a second assignment silently replaces the first. Seen in older code.

WAY 3 An onclick attribute in the HTML (avoid)

```
<button onclick="handleClick()">Save</button>
```

Mixes JavaScript into HTML, needs a global function, and is blocked by strict security settings (CSP) on many real sites.

WATCH OUT

`save.addEventListener('click', handleSave())` with brackets **calls** `handleSave` straight away and passes its result (`undefined`) as the listener. Pass the function itself: `handleSave`, or wrap it: `() => handleSave(42)`.

18.2 The event object

Every listener receives one argument: an object describing what happened. Name it `event` (you'll also see `e` or `evt`).

Property / method	Tells you / does
`event.type`	`'click'`, `'keydown'`, `'submit'`…
`event.target`	the element where it started (the thing actually clicked)
`event.currentTarget`	the element this listener is attached to
`event.key` / `event.code`	the key's meaning (`'a'`, `'Enter'`) / its physical position (`'KeyA'`)
`event.shiftKey`, `ctrlKey`, `altKey`, `metaKey`	`true` if that key was held down
`event.clientX` / `clientY`	where the pointer was, in pixels
`event.preventDefault()`	stop the browser's own action (18.7)
`event.stopPropagation()`	stop the event travelling further (18.4; rarely needed)

18.3 input or change?

JS · input vs change

```
const city = document.getElementById('city');
city.addEventListener('input', () => console.log('input:', city.value));
city.addEventListener('change', () => console.log('change:', city.value));
```

▶ CONSOLE OUTPUT · after typing "Dha" and pressing Tab

```
input: D
input: Dh
input: Dha
change: Dha
```

Field	`input` fires…	`change` fires…
text box, textarea	on every character	when you leave the box, if the value changed
checkbox, radio	when it's ticked	when it's ticked (use `change`)
select	when an option is picked	when an option is picked
range slider	continuously while dragging	when you let go

EXAMPLE A live character counter (practice/char-counter.js)

JS · the limit lives in the HTML; the script reads it

```
const bio = document.getElementById('bio');
const counter = document.getElementById('bio-counter');
const max = Number(bio.getAttribute('maxlength'));   // the limit in the HTML: 160

function updateCounter() {
  const used = bio.value.length;
  const left = max - used;
  counter.textContent = `${used} / ${max}`;
  counter.classList.toggle('is-near', left <= 20 && left > 0);   // amber
  counter.classList.toggle('is-full', left === 0);               // red
}

bio.addEventListener('input', updateCounter);   // every change, as you type
updateCounter();                                // and once at the start
```

TIP

Call the update function once at the start as well as in the listener. Then the counter is right even before the first key press, and when the browser refills the field after a reload.

18.4 How an event travels: capture and bubbling

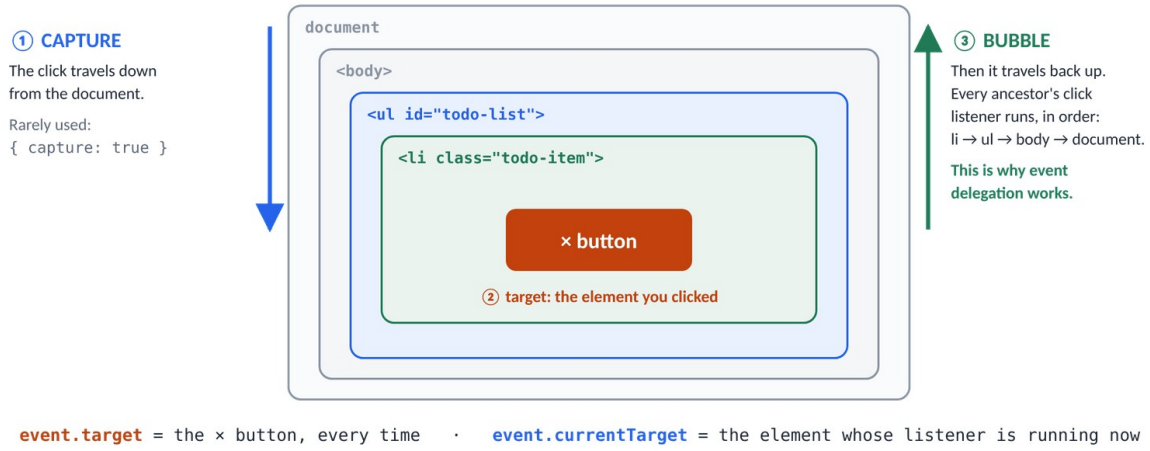


Figure 18.1 · A click travels down to its target (capture), then back up through every ancestor (bubble)

JS · bubbling: one click, three listeners

```
function logClick(event) {
  console.log(`listener on #${event.currentTarget.id} · target: #${event.target.id}`);
}
for (const id of ['outer', 'middle', 'inner']) {
  document.getElementById(id).addEventListener('click', logClick);
}
document.getElementById('inner').click();
```

CONSOLE OUTPUT

```
listener on #inner · target: #inner
listener on #middle · target: #inner
listener on #outer · target: #inner
```

One click, three listeners. `target` stays `#inner` the whole way (that's where the click started); `currentTarget` changes as the event bubbles up through `#middle` and `#outer`. This is exactly what the event lab shows on the page.

JS · capture first, then stopPropagation

```
const outer = document.getElementById('outer');
const middle = document.getElementById('middle');
outer.addEventListener('click', () => {
  console.log('outer, capture');
}, { capture: true });
outer.addEventListener('click', () => console.log('outer, bubble'));
middle.addEventListener('click', (event) => {
  console.log('middle: stopping here');
  event.stopPropagation();                  // #outer's bubble listener never hears it
});
document.getElementById('inner').click();
```

CONSOLE OUTPUT

```
outer, capture
middle: stopping here
```

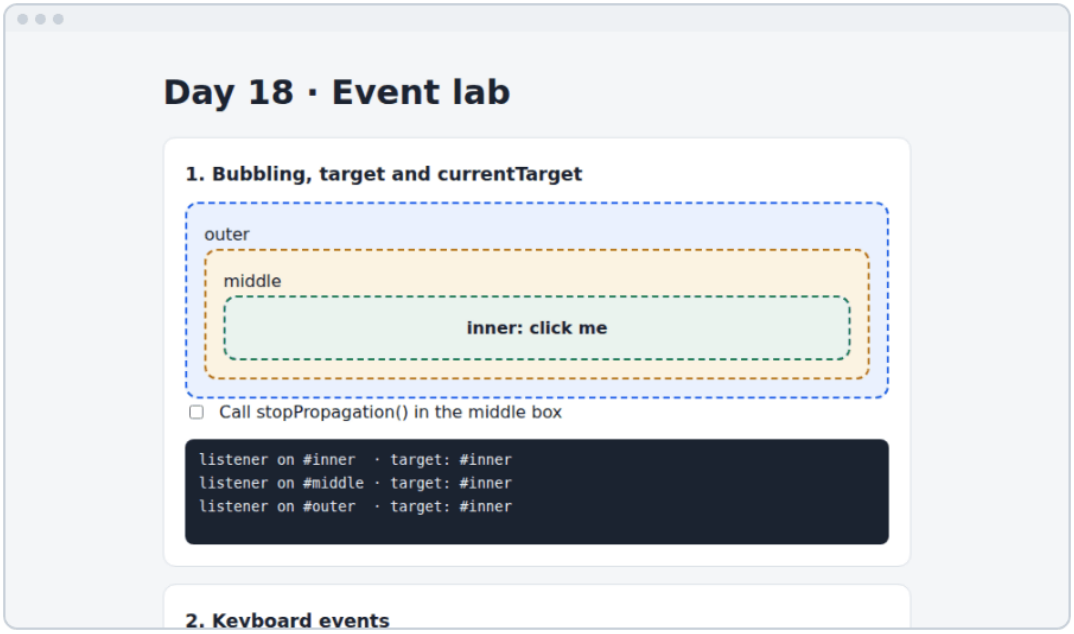


Figure 18.2 · practice/event-lab.html after a click on the inner box

WATCH OUT

Avoid `stopPropagation()`. Other code may rely on the event reaching the top: delegated listeners, "close the menu when you click outside", analytics. Almost every time, checking `event.target` in the parent's listener is the better fix.

18.5 Event delegation: one listener for a whole list

IN PLAIN WORDS

A big office doesn't put a guard at every door. One **receptionist** at the front desk asks each visitor "who are you here to see?". Because clicks bubble up, one listener on a list can handle every item in it, even items that are added later.

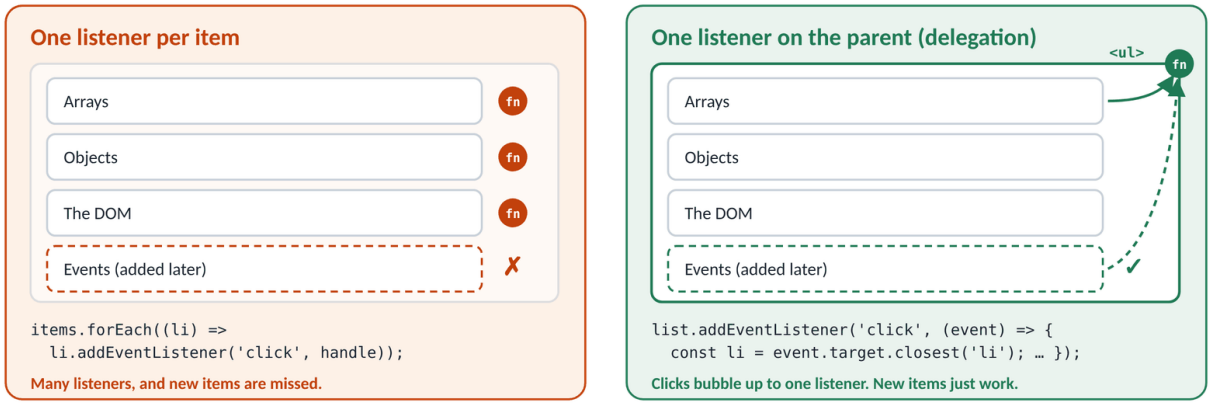


Figure 18.3 · One listener per item misses new items; one listener on the parent catches them all

JS · one listener for the whole list

```
const list = document.getElementById('lessons');
list.addEventListener('click', (event) => {
  const button = event.target.closest('button');   // the clicked button, or null
  if (!button) return;                              // a click on the gap between items
  console.log('clicked', button.textContent);
});

const added = document.createElement('li');       // added AFTER the listener
added.innerHTML = '<button type="button"><b>Events</b></button>';
list.append(added);
list.querySelector('b').click();       // a click on the <b> INSIDE the new button
list.querySelector('button').click();
```

CONSOLE OUTPUT

```
clicked Events
clicked Arrays
```

Two details make delegation work. First, `event.target` might be something **inside** the button (the `<b>`, an icon, a `<span>`), so we walk up with `closest('button')` to the element we care about. Second, if the click wasn't on a button at all, `closest` returns `null` and we stop.

Method	Answers
`el.closest('.todo-item')`	which `.todo-item` is this inside? (itself counts; `null` if none)
`el.matches('.todo-toggle')`	is this element itself a `.todo-toggle`? (`true` / `false`)
`parent.contains(el)`	is `el` somewhere inside `parent`?

⇄ MORE WAYS **Two ways to handle clicks on every list item**

WAY 1 **One listener per item**

```
list.querySelectorAll('button').forEach((button) => {
  button.addEventListener('click', handleClick);
});
```

Easy to understand, but items added later have no listener, and 500 items means 500 listeners.

WAY 2 **One listener on the parent (delegation)**

```
list.addEventListener('click', (event) => {
  const button = event.target.closest('button');
  if (button) handleClick(button);
});
```

One listener, and new items just work. The to-do list, the tabs and the filters all use this.

JS · widgets/js/main.js, section 3: the tabs, delegated

```
// Before (Day 17): four listeners, one per tab
tracksRoot.querySelectorAll('[role="tab"]').forEach((tab) => {
  tab.addEventListener('click', () => selectTab(tracksRoot, tab));
});

// After (Day 18): ONE listener on the tab list
list.addEventListener('click', (event) => {
  const tab = event.target.closest('[role="tab"]');
  if (!tab) return;                       // the click was on the list, not a tab
  selectTab(root, tab);
});
```

TRY IT

Open `practice/delegation.html`. Add three lessons with the form, then click them and their × buttons. None of them got a listener of their own, and all of them work.

18.6 Keyboard events

JS · key and code

```
const city = document.getElementById('city');
city.addEventListener('keydown', (event) => {
  console.log(event.key, event.code, event.shiftKey);
});
```

▶ CONSOLE OUTPUT · after pressing a, Shift+A, Enter, ←, Escape

```
a KeyA false
Shift ShiftLeft true
A KeyA true
Enter Enter false
ArrowLeft ArrowLeft false
Escape Escape false
```

	`event.key`	`event.code`
Means	what the key produces	where the key is on the keyboard
Shift + a	`'A'`	`'KeyA'`
On a French keyboard, the key left of Z	`'q'`	`'KeyA'`
Use it for	text, Enter, Escape, arrows (almost always)	games (WASD keys by position)

Arrow keys for the tabs

Accessible tabs follow a standard pattern: **Tab** moves into the tab list and out again; ← → move between tabs; **Home** / **End** jump to the first / last. Only the selected tab has `tabIndex = 0`, so Tab never has to walk through every tab. This is called a **roving tabindex**.

JS · widgets/js/main.js, section 3: ← → Home End

```
list.addEventListener('keydown', (event) => {
  const tabs = [...list.querySelectorAll('[role="tab"]')];   // a real array
  const current = tabs.indexOf(document.activeElement);    // the focused tab
  if (current === -1) return;
  const total = tabs.length;

  let next;
  if (event.key === 'ArrowRight') next = (current + 1) % total;
  else if (event.key === 'ArrowLeft') next = (current - 1 + total) % total;
  else if (event.key === 'Home') next = 0;
  else if (event.key === 'End') next = total - 1;
  else return;                              // any other key: do nothing

  event.preventDefault();                   // stop Home/End from scrolling the page
  tabs[next].focus();
  selectTab(root, tabs[next]);
});
```

Focused tab (of 4)	Key	next = ...
2 (Databases)	→	`(2 + 1) % 4` = **3** (Deploy)
3 (Deploy)	→	`(3 + 1) % 4` = **0**: wraps to Frontend
0 (Frontend)	←	`(0 - 1 + 4) % 4` = **3**: wraps to Deploy

`total` is the number of tabs. The `% total` makes the index **wrap around** like a clock: after the last tab comes the first. The `+ total` in the ← line stops the number going negative, because `-1 % 4` is `-1` in JavaScript, not 3.

⇄ MORE WAYS Two ways to map keys to moves

WAY 1 if / else if (Day 18)

```
if (event.key === 'ArrowRight') next = …;
else if (event.key === 'ArrowLeft') next = …;
```

Clear, and easy to add special cases to.

WAY 2 A lookup object (Day 20)

```
const moves = {
  ArrowRight: (current + 1) % tabs.length,
  ArrowLeft: (current - 1 + tabs.length) % tabs.length,
  Home: 0,
  End: tabs.length - 1,
};
const next = moves[event.key];
if (next === undefined) return;
```

The table of keys is data, so adding a key is one line. Day 20's `tabs.js` uses this.

18.7 preventDefault: stop the browser's own action

JS · preventDefault on a link

```
const link = document.getElementById('docs');
link.addEventListener('click', (event) => {
  event.preventDefault();                  // don't follow the link
  console.log('stopped:', event.defaultPrevented);
});
link.click();
console.log(location.pathname);           // still on the same page
```

CONSOLE OUTPUT

```
stopped: true
/
```

Default action	Happens on	Stop it when...
Follow the link	`click` on `<a>`	JavaScript shows the content instead
Send the form and reload	`submit` on `<form>`	JavaScript handles the data (always, this week)
Scroll the page	`keydown` Space, Home, End, arrows	those keys drive your widget
Tick the box	`click` on a checkbox	almost never (it confuses people)

WATCH OUT

`preventDefault()` and `stopPropagation()` are unrelated. The first stops **the browser's action**; the second stops **the event travelling** to other listeners. A form handler needs the first, almost never the second.

18.8 Forms: let HTML check first, then read the data

JS · reading a form with FormData

```
const form = document.getElementById('call-form');
form.addEventListener('submit', (event) => {
  event.preventDefault();                          // no page reload
  const data = new FormData(form);                 // every field that has a name
  console.log(data.get('name'), data.get('day'));
  console.log(Object.fromEntries(data));           // all fields as one object
});
form.requestSubmit();                              // like pressing the submit button
```

CONSOLE OUTPUT

```
Rahim Sunday
{name: 'Rahim', day: 'Sunday'}
```

⇄ MORE WAYS **Four ways to read a form**

WAY 1 Each field's value

```
const name = document.getElementById('call-name').value;
```

Fine for one or two fields.

WAY 2 FormData.get()

```
const data = new FormData(form);
const name = data.get('name');
```

Reads by the `name` attribute, so the ids don't matter.

WAY 3 Object.fromEntries(new FormData(form))

```
const { name, day } = Object.fromEntries(new FormData(form));
```

Every field in one object, then destructure. Day 20's modal uses this.

WAY 4 form.elements

```
const name = form.elements.name.value;
```

The form knows its own fields by name. Handy, but `name` can clash with built-in properties.

HTML attribute	The browser checks, before `submit` fires
`required`	not empty
`minlength="3"` / `maxlength="80"`	the length of the text
`type="email"` / `type="url"`	the right shape
`min` / `max` (numbers, dates)	the range
`pattern=".*\S.*"`	matches a pattern: here, at least one character that isn't a space

If any check fails, the browser shows its own message, focuses the field and **doesn't fire `submit` at all**. So by the time your listener runs, the basic checks have passed. Style the states with `:invalid` / `:user-invalid` in CSS. `required` alone accepts " " (only spaces), which is why the modal's name field adds `pattern=".*\S.*"` and the script still calls `trim()`.

18.9 The modal dialog

A modal puts one focused task on top of the page: here, "Book a free call". Building one by hand used to take 100+ lines (trapping focus, the Escape key, the dark background). The HTML `<dialog>` element does most of it for you when you open it with `showModal()`.

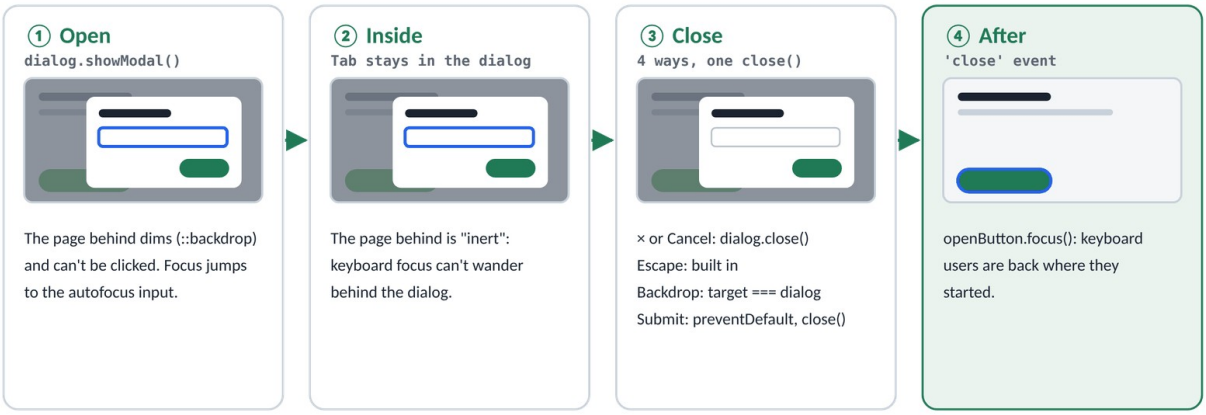


Figure 18.4 · Open, use, close, and give focus back: the four stages of the modal

HTML · widgets/index.html: the modal's HTML

```
<button class="button" type="button" id="open-call">Book a free call</button>
<p class="form-message" id="call-status" role="status" aria-live="polite"></p>

<dialog class="modal" id="call-dialog" aria-labelledby="call-dialog-title">
  <form class="modal-body" id="call-form">
    <div class="modal-head">
      <h3 id="call-dialog-title">Book a free call</h3>
      <button class="icon-button" type="button" id="close-call"
              aria-label="Close">×</button>
    </div>
    <label for="call-name">Your name</label>
    <input id="call-name" name="name" required pattern=".*\S.*" autofocus>
    <label for="call-day">Best day</label>
    <select id="call-day" name="day"><option>Saturday</option> … </select>
    <div class="modal-actions">
      <button class="button button-ghost" type="button"
              id="cancel-call">Cancel</button>
      <button class="button" type="submit">Book the call</button>
    </div>
  </form>
</dialog>
```

JS · widgets/js/main.js, section 4: the modal

```
const openButton = document.getElementById('open-call');
const dialog = document.getElementById('call-dialog');
const callForm = document.getElementById('call-form');
const callStatus = document.getElementById('call-status');

openButton.addEventListener('click', () => {
  callStatus.textContent = '';
  dialog.showModal();            // on top, page behind dimmed and inert, focus inside
});

// × and Cancel
const closeButton = document.getElementById('close-call');
const cancelButton = document.getElementById('cancel-call');
closeButton.addEventListener('click', () => dialog.close());
cancelButton.addEventListener('click', () => dialog.close());

// The dark backdrop: a click there has the <dialog> itself as its target
dialog.addEventListener('click', (event) => {
  if (event.target === dialog) dialog.close();
});

// However it closes (buttons, backdrop, Escape, submit): focus back to the opener
dialog.addEventListener('close', () => openButton.focus());

callForm.addEventListener('submit', (event) => {
  event.preventDefault();                   // the browser already checked required
  const data = new FormData(callForm);
  const name = data.get('name').trim();
  const day = data.get('day');
  dialog.close();
  callStatus.textContent =
    `Thanks, ${name}! A mentor will call you on ${day} evening.`;
  callForm.reset();
});
```

It closes when...	Handled by	Why it matters
× or Cancel is clicked	your click listeners → `close()`	a visible way out
Escape is pressed	the browser (built into modal dialogs)	keyboard users expect it
the dark backdrop is clicked	`event.target === dialog`	mouse users expect it
the form is submitted	your submit listener → `close()`	the task is done
(any of the above)	the `close` event → `openButton.focus()`	keyboard users continue where they were

Why does the backdrop test work? The dialog has `padding: 0` and the form fills it, so any click **inside** the box lands on the form or something in it. Only a click on the dark area outside the box reports the `<dialog>` element itself as `event.target`.

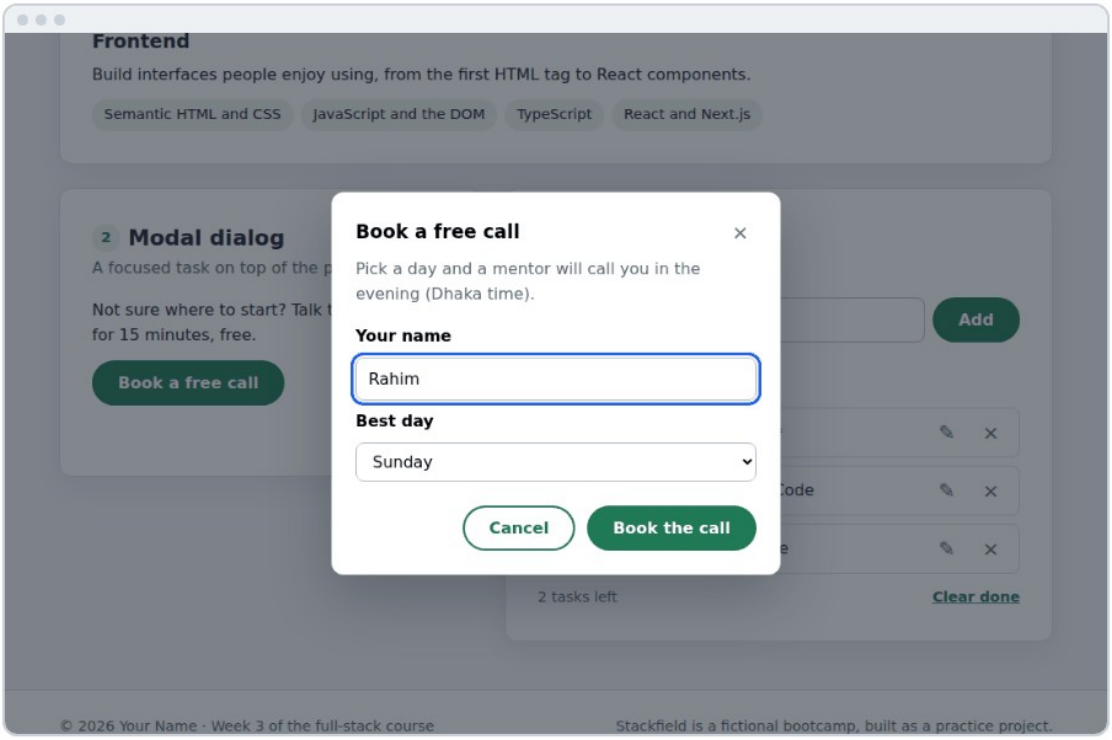


Figure 18.5 · The open modal: the page behind is dimmed and can't be clicked

- ☐ Tab to "Book a free call", press Enter: the dialog opens and the name field has focus
- ☐ Tab and Shift+Tab stay inside the dialog; the page behind can't be reached
- ☐ Escape closes it, and focus is back on "Book a free call"
- ☐ Submitting with an empty name (or only spaces) shows the browser's message instead
- ☐ A screen reader announces "Book a free call, dialog" when it opens, and the thank-you message after

⇄ MORE WAYS Two ways to close a dialog from a button

WAY 1 A click listener that calls close() (the widget)

```
cancel.addEventListener('click', () => dialog.close());
```

Full control, e.g. to ask "discard your changes?" first.

WAY 2 A form with method="dialog"

```
<form method="dialog">
  <button value="cancel">Cancel</button>
  <button value="ok">OK</button>
</form>
```

Any submit button closes the dialog with **no JavaScript**, and `dialog.returnValue` tells you which button it was. Great for simple confirm boxes.

WATCH OUT

Use `showModal()`, not `show()`. `show()` opens a **non-modal** dialog: no backdrop, the page behind stays clickable, and Escape doesn't close it.

18.10 DSA: LeetCode 242, Valid Anagram

The problem. Return `true` if `t` uses exactly the same letters as `s`, the same number of times. `"anagram"` / `"nagaram"` → `true`; `"rat"` / `"car"` → `false`.

1. **Quick exit:** different lengths can never be anagrams. Return `false` straight away.
2. **Idea 1, sort:** sort the letters of both words; anagrams become the same word (`"aaagmnr"`).
3. **Idea 2, count:** count each letter of `s` (+1), then take away each letter of `t` (−1). If a count would go below zero, `t` has a letter `s` doesn't have (or has too many of it).
4. **Why counting is faster:** it looks at each letter once, with no sorting.

Step	counts	Note
count "rat"	`{ r: 1, a: 1, t: 1 }`	+1 for each letter of s
take "c"	`counts.c` is missing (0)	**return false**: t has a letter s doesn't

⇄ MORE WAYS Valid Anagram three ways (dsa/242-valid-anagram.js)

WAY 1 Sort both and compare

```
function isAnagramSort(s, t) {
  if (s.length !== t.length) return false;
  const sortLetters = (word) => word.split('').sort().join('');
  return sortLetters(s) === sortLetters(t);
}
```

Short and easy to trust. Time O(n log n) for the sorting.

WAY 2 Count with an object

```
function isAnagram(s, t) {
  if (s.length !== t.length) return false;
  const counts = {};
  for (const letter of s) counts[letter] = (counts[letter] ?? 0) + 1;
  for (const letter of t) {
    if (!counts[letter]) return false;   // missing, or already used up
    counts[letter]--;
  }
  return true;              // same length + nothing went below 0 = same letters
}
```

Time **O(n)**. The object's size is limited by the alphabet, so the extra memory is small.

WAY 3 Count with a Map

```
function isAnagramMap(s, t) {
  if (s.length !== t.length) return false;
  const counts = new Map();
  for (const letter of s) counts.set(letter, (counts.get(letter) ?? 0) + 1);
  for (const letter of t) {
    if (!counts.get(letter)) return false;
    counts.set(letter, counts.get(letter) - 1);
  }
  return true;
}
```

The same logic with a **Map**: a built-in object made for counting and lookups. `set`, `get` and `has` are its main methods; you'll use it tomorrow for Two Sum.

RESULT

```
isAnagram('anagram', 'nagaram') → true
isAnagram('rat', 'car')         → false
isAnagram('listen', 'silent')   → true
isAnagram('aacc', 'ccac')       → false
```

Counting patterns like this one (an object or Map of "how many of each") solve a huge family of problems: most frequent word, first unique character, ransom note, group anagrams. When a problem says "same letters", "how many times" or "duplicates", think counting.

Build it, step by step

1. Open `practice/event-lab.html`. Click the inner box; then tick "stopPropagation" and click again. Explain the log out loud.
2. Build the character counter and the delegated list yourself, then compare with `char-counter.js` and `delegation.js`.
3. Replace the four tab listeners with one delegated listener, then add ← → Home End.
4. Build the modal: `showModal()`, ×, Cancel, the backdrop, and focus back to the button on `close`.
5. Test the modal and the tabs with the keyboard only, using the checklist in 18.9.
6. Solve **Valid Anagram**, then compare with `dsa/242-valid-anagram.js`.
7. Commit: `git commit -m "Day 18: events practice, delegated tabs and an accessible modal"` and push.

★ MUST KNOW *the ideas to remember from today*

- ✓ `addEventListener(type, fn)`: give the function, don't call it.
- ✓ `target` = where it started; `currentTarget` = whose listener is running.
- ✓ Events go down (capture) then up (bubble). Listeners run in the bubble phase by default.
- ✓ Delegation: one listener on the parent + `event.target.closest(selector)`.
- ✓ `input` for every keystroke; `change` for committed changes; `submit` for forms.
- ✓ `preventDefault()` stops the browser's action; `stopPropagation()` stops the journey (rarely needed).
- ✓ `new FormData(form)` reads every named field; the browser checks `required` and `pattern` first.
- ✓ `<dialog>` + `showModal()` gives a backdrop, focus trap and Escape; return focus on `close`.

WATCH OUT

- `addEventListener('click', handler())`: called immediately.
- Using `event.target` when the click landed on a child element (use `closest`).
- Forgetting `preventDefault()` on submit: the page reloads and your log vanishes.
- `removeEventListener` with a new arrow function: it removes nothing.
- Using `stopPropagation` to fix a problem delegation should solve.
- Opening the dialog with `show()` instead of `showModal()`.

QUESTIONS LEARNERS ASK

Q Why did my console message disappear when I submitted the form?

The page reloaded: submitting a form sends it and loads a new page. `event.preventDefault()` in the submit listener stops that. In DevTools, the Console setting "Preserve log" keeps messages across reloads.

Q Should I listen for `click` or `keydown` on buttons?

`click`. On a real `<button>`, Enter and Space already fire `click`. Use `keydown` only for extra keys like arrows or Escape.

Q Is it bad to have many listeners?

A few dozen is nothing. Delegation matters for lists that are long or change often, and it makes code simpler because new items need no setup.

Q What does `this` mean inside a listener?

In a normal `function` listener, `this` is `event.currentTarget`. In an arrow function it isn't. Use `event.currentTarget`: it's clearer and works with both.

More practice

MORE PRACTICE · try each one, then check the Answers at the back

1. Log the x and y position of every click on the page.
2. Make a text box that shows its value in upper case in a `<p>` as you type.
3. Press "/" anywhere on the page to focus the search box (like GitHub and YouTube).
4. In a delegated list, make a click on the text toggle `.done`, and a click on × delete the item.
5. Add a keyboard shortcut: Ctrl+Enter submits the modal's form.
6. Why does `-1 % 4` give `-1` in JavaScript, and how do you get 3?

Check yourself

1. What is the difference between `event.target` and `event.currentTarget`?
2. What is event delegation, and why is it useful for a list?
3. Name three ways the modal can close, and what happens to keyboard focus afterwards.

*Answer out loud before you turn to the **Answers** section at the back.*

DONE WHEN · tick these off, then mark the day done in your Logbook

- ☐ You can explain the event lab's log, with and without stopPropagation
- ☐ The character counter and the delegated list work, built by you
- ☐ The tabs use one listener and the arrow keys; the modal passes the keyboard checklist
- ☐ Valid Anagram accepted; committed and pushed

Resources

All free. Every name is a link, in Word and in the PDF.

Resource (click to open)	Use it for
javascript.info: Introduction to browser events javascript.info/introduction-browser-events	Listeners, the event object, and more
javascript.info: Bubbling and capturing javascript.info/bubbling-and-capturing	target vs currentTarget, with a demo
javascript.info: Event delegation javascript.info/event-delegation	Delegation patterns, with tasks
javascript.info: Browser default actions javascript.info/default-browser-action	preventDefault in depth
javascript.info: Keyboard: keydown and keyup javascript.info/keyboard-events	key vs code, with a key tester
MDN: <dialog>: The Dialog element developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/dialog	showModal, close, ::backdrop and method="dialog"
MDN: FormData developer.mozilla.org/en-US/docs/Web/API/FormData	Reading forms from JavaScript
MDN: Client-side form validation developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Forms/Form_validation	required, pattern and the :invalid styles
WAI-ARIA Authoring Practices: Dialog (Modal) w3.org/WAI/ARIA/apg/patterns/dialog-modal	How an accessible modal should behave
LeetCode 242: Valid Anagram leetcode.com/problems/valid-anagram	Today's problem

DAY
19

WEEK 3 · WEDNESDAY 14 OCTOBER

Closures, timers and localStorage

Functions that remember, code that runs later, and data that survives a reload.

TIME	YOU BUILD	CODE FOLDER
3 hours	**Closure exercises, a focus timer, a debounced search, a saved theme, the to-do list (part 1), and Two Sum**	`day19-closures`

Today you will

- ✓ Explain a closure in plain words, and use one to keep private state
- ✓ Write `createCounter`, `once`, `makeMultiplier` and an id generator
- ✓ Answer the classic `var` vs `let` loop question
- ✓ Schedule code with `setTimeout` and `setInterval`, and always cancel it
- ✓ Explain why timer callbacks run after the rest of your code
- ✓ Debounce a search box, and know when to throttle instead
- ✓ Save and load data with `localStorage` and JSON, safely
- ✓ Build the to-do list with one state array and one `render()`
- ✓ Solve LeetCode 1, Two Sum, with a Map

Key terms

Term	Meaning
Closure	A function together with the variables it remembers from where it was created
Lexical scope	A function can use the variables around the place it was **written**
Factory function	A function that makes and returns other functions or objects
Private state	Data only certain functions can reach, like `count` in `createCounter`
Timer id	The number `setTimeout` / `setInterval` return; pass it to `clear…` to cancel
Event loop	How JavaScript runs one task at a time, taking the next from a queue
Debounce	Wait until calls stop for a moment, then run once
Throttle	Run at most once every so many milliseconds
localStorage	A small key–value store in the browser that survives reloads
State	The data that describes what the page should show right now
Map	A built-in key–value collection with `set`, `get`, `has`

19.1 Functions that remember

IN PLAIN WORDS

When a function is created, it packs a **backpack** with the variables around it. Wherever the function goes later, and however much later it runs, it carries the backpack and can use (and change) what's inside. That backpack is the **closure**.

JS · Part A of closures-solutions.js: a function that makes functions

```
function makeGreeter(greeting) {
  return function (name) {            // this inner function uses greeting…
    return `${greeting}, ${name}!`;
  };
}
const hello = makeGreeter('Hello');   // makeGreeter has finished here…
const salaam = makeGreeter('Assalamu alaikum');
console.log(hello('Rahim'));           // …but greeting is still in the backpack
console.log(salaam('Nadia'));
```

▶ CONSOLE OUTPUT

```
Hello, Rahim!
Assalamu alaikum, Nadia!
```

1. `makeGreeter('Hello')` runs. Its variable `greeting` is `'Hello'`.
2. It creates the inner function and **returns** it. `makeGreeter` is now finished.
3. Normally `greeting` would be thrown away, but the inner function still uses it, so JavaScript keeps it alive in the backpack.
4. Calling `hello('Rahim')` later uses that remembered `greeting`. `salaam` has its **own** backpack with its own `greeting`.

You've already used closures without the name: every event listener that uses a variable from outside it (`openButton.focus()` inside the dialog's `close` listener) is a closure. So is every `map` callback that uses a variable like `limit`.

19.2 Private state: createCounter

```
function createCounter() {
  let count = 0;          // private
  return {
    increment() { count++; return count; },
    value() { return count; },
  };
}

const a = createCounter();
const b = createCounter();
a.increment(); a.increment();
b.increment();
a.value(); // 2   b.value(); // 1
```

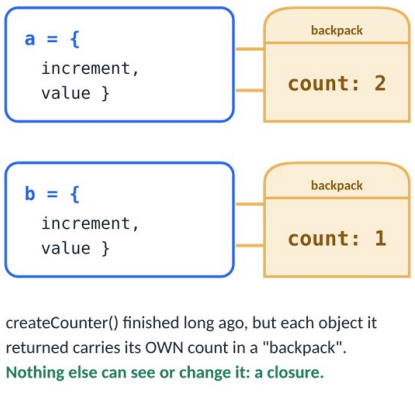


Figure 19.1 · Each counter carries its own backpack with its own count; nothing else can reach it

JS · Exercise 1: createCounter(start)

```
function createCounter(start = 0) {
  let count = start;                  // only the four functions below can reach this
  return {
    increment() { count++; return count; },
    decrement() { count--; return count; },
    reset() { count = start; return count; },
    value() { return count; },
  };
}
const first = createCounter();
first.increment();
first.increment();
const second = createCounter(10);
second.decrement();
console.log(first.value(), second.value());   // each has its OWN count
console.log(first.count);                     // private: not a property
console.log(first.reset());
```

▶ CONSOLE OUTPUT

```
2 9
undefined
0
```

⇄ MORE WAYS Three ways to keep a count, from worst to best

WAY 1 A global variable

```
let count = 0;
function increment() { count++; }
```

Any code anywhere can change or clash with `count`, and you can only have one counter.

WAY 2 An object property

```
const counter = { count: 0, increment() { this.count++; } };
```

Grouped, but public: `counter.count = -500` is allowed from anywhere.

WAY 3 A closure

```
const counter = createCounter();
counter.increment();
```

Private (only its own methods can change `count`), and as many independent counters as you like.

19.3 More closures you'll really use

JS · Exercise 2: once(fn), for "pay" or "submit" buttons

```
function once(fn) {
  let called = false;
  let result;
  return function (...args) {
    if (!called) {
      called = true;
      result = fn(...args);
    }
    return result;                    // later calls return the first result
  };
}
let launches = 0;
const launch = once(() => {
  launches++;
  return 'launched';
});
launch();
launch();
console.log(launches, launch());
```

CONSOLE OUTPUT

```
1 launched
```

JS · Exercises 3 and 4: makeMultiplier and createIdGenerator

```
const makeMultiplier = (factor) => (n) => n * factor;   // an arrow returning an arrow
const triple = makeMultiplier(3);
console.log(triple(5));
console.log([1, 2, 3].map(makeMultiplier(10)));          // as a callback

function createIdGenerator(prefix) {
  let next = 1;
  return () => `${prefix}-${next++}`;       // next++ gives the value, THEN adds 1
}
const taskId = createIdGenerator('task');
console.log(taskId(), taskId(), taskId());
console.log(createIdGenerator('note')());     // a new generator starts at 1
```

CONSOLE OUTPUT

```
15
[10, 20, 30]
task-1 task-2 task-3
note-1
```

Pattern	Remembers	Real use
`createCounter`	a number	likes, cart quantity, steps in a wizard
`once`	whether it ran, and the result	set things up only once; stop double payments
`makeMultiplier`	a setting (the factor)	pre-configured helpers: `withTax = makeTaxer(0.15)`
`createIdGenerator`	the next number	unique ids for new items
`debounce` (19.7)	the waiting timer	search boxes, autosave

19.4 var vs let in a loop: a classic interview question

JS · Exercise 5: what do the functions remember?

```
const withVar = [];
for (var i = 0; i < 3; i++) withVar.push(() => i);   // ONE i, shared by all three
const withLet = [];
for (let j = 0; j < 3; j++) withLet.push(() => j);   // a NEW j for every lap

console.log(withVar.map((fn) => fn()));
console.log(withLet.map((fn) => fn()));
```

▶ CONSOLE OUTPUT

```
[3, 3, 3]
[0, 1, 2]
```

Each arrow function remembers the **variable**, not the value it had at the time. With `var` there is only one `i` for the whole loop; by the time the functions run, the loop has finished and `i` is 3, so all three say 3. With `let`, JavaScript creates a fresh `j` for every lap, so each function remembers its own. One more reason to never use `var`.

19.5 Timers: code that runs later

IN PLAIN WORDS

A timer is a kitchen timer. You set it and **carry on cooking**; when it rings you deal with it. JavaScript doesn't stop and wait: the rest of your code keeps running, and the timer's function runs later.

JS · setTimeout: the order surprises everyone once

```
console.log('1. start');
setTimeout(() => console.log('3. after 500 ms'), 500);
setTimeout(() => console.log('2. after 0 ms'), 0);
console.log('1b. end of the script');
```

▶ CONSOLE OUTPUT

```
1. start
1b. end of the script
2. after 0 ms
3. after 500 ms
```

Even a 0 ms timer runs **after** the rest of the script. JavaScript does one thing at a time: it finishes the current job completely, and only then takes the next waiting job (a timer that's due, a click, a key press) from a queue. This is called the **event loop**. It's why a slow loop freezes the page: clicks queue up but can't run until the loop ends. Week 4 builds on this for `fetch` and `async/await`.

JS · setInterval, clearInterval and clearTimeout

```
let ticks = 0;
const intervalId = setInterval(() => {
  ticks++;
  console.log('tick', ticks);
  if (ticks === 3) {
    clearInterval(intervalId);            // always stop what you start
    console.log('stopped');
  }
}, 100);
console.log('this prints first');

const timerId = setTimeout(() => console.log('never shown'), 1000);
clearTimeout(timerId);                    // cancelled before it ran
```

CONSOLE OUTPUT

```
this prints first
tick 1
tick 2
tick 3
stopped
```

Call	Does
`setTimeout(fn, ms)`	runs `fn` once, after at least `ms` milliseconds; returns an id
`clearTimeout(id)`	cancels it if it hasn't run yet
`setInterval(fn, ms)`	runs `fn` every `ms` milliseconds until cancelled; returns an id
`clearInterval(id)`	stops it

WATCH OUT

- The delay is a **minimum**, not a promise: if the page is busy, the timer runs later. Browsers also slow timers down in background tabs.
- An interval you never clear keeps running (and using the battery) until the page closes. Keep the id and clear it.
- `setTimeout(doThing(), 500)` calls `doThing` now. Pass the function: `setTimeout(doThing, 500)` or `setTimeout(() => doThing(42), 500)`.

TIP

To see the event loop in action, paste the setTimeout example into **Loupe** (in the Resources), a free visualiser that animates the call stack and the queue.

19.6 A focus timer with setInterval

JS · a pure helper, tested first

```
function formatTime(totalSeconds) {
  const minutes = String(Math.floor(totalSeconds / 60)).padStart(2, '0');
  const seconds = String(totalSeconds % 60).padStart(2, '0');
  return `${minutes}:${seconds}`;
}
console.log(formatTime(65), formatTime(600), formatTime(0));
```

▶ CONSOLE OUTPUT

```
01:05 10:00 00:00
```

JS · practice/timers.js, section 1 (shortened)

```
let secondsLeft = 60;
let intervalId = null;                      // null = not running

function stop() {
  clearInterval(intervalId);
  intervalId = null;
}

startButton.addEventListener('click', () => {
  intervalId = setInterval(() => {          // every 1000 ms
    secondsLeft--;
    if (secondsLeft === 0) stop();          // time's up
    renderClock();
    if (secondsLeft === 0) clock.textContent = 'Done!';
  }, 1000);
  renderClock();                            // disables Start while running
});
pauseButton.addEventListener('click', () => { stop(); renderClock(); });
```

`intervalId` doubles as the state "is it running?": `null` means stopped. `renderClock()` uses it to disable Start while running (so a second click can't start a second interval, a classic bug) and to disable Pause while stopped.

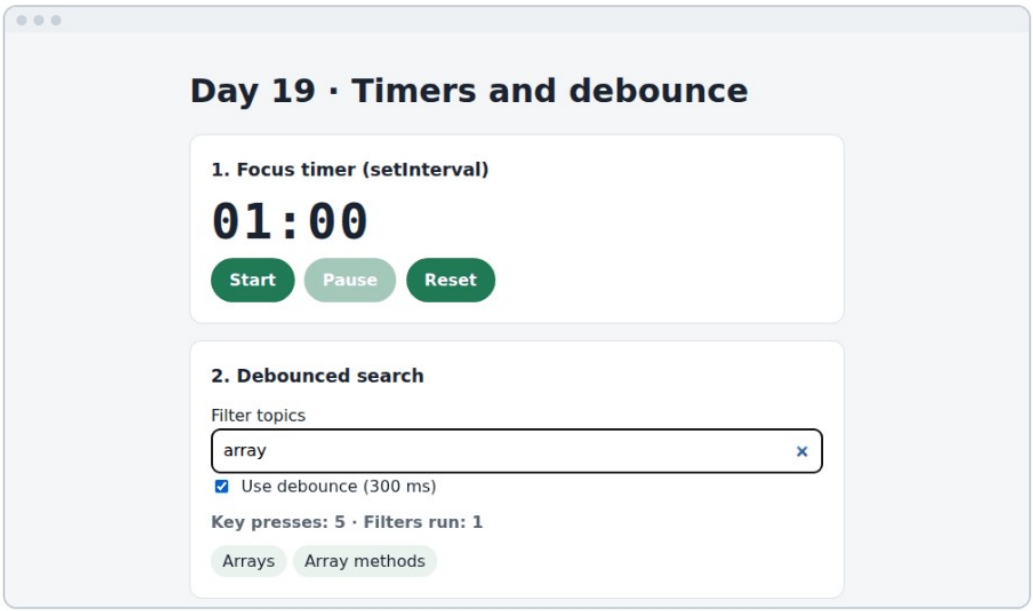


Figure 19.2 · practice/timers.html: the focus timer, and the search box after typing "array"

⇄ MORE WAYS **Two ways to count down**

WAY 1 Count the ticks (the practice page)

```
secondsLeft--;   // once a second
```

Simple, and fine for a study timer. Each tick can be a little late, so over an hour it can drift by a few seconds.

WAY 2 Compare with the clock

```
const endsAt = Date.now() + 60 * 1000;
// in the interval:
const secondsLeft = Math.max(0, Math.ceil((endsAt - Date.now()) / 1000));
```

Always exact, even if the tab was in the background. Use this for anything important (exams, auctions).

19.7 Debounce: wait until the typing stops

IN PLAIN WORDS

A lift door doesn't close the moment one person walks in. Every new person **resets** its wait, and it closes only when nobody has come for a few seconds. Debounce does the same for events: every call restarts a short timer, and the real work runs once, after the calls stop.

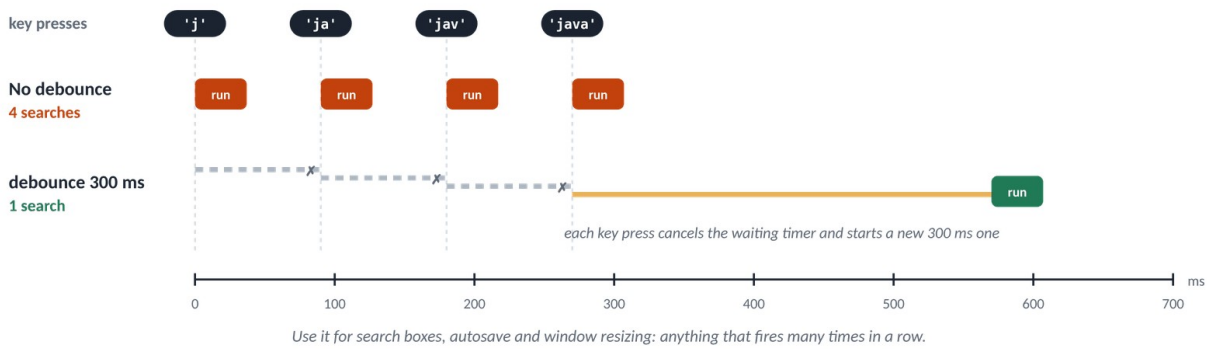


Figure 19.3 · Four quick key presses: four searches without debounce, one search with it

JS · **Exercise 6: debounce(fn, ms)**

```
function debounce(fn, ms) {
  let timerId;                                  // kept between calls: a closure
  return (...args) => {
    clearTimeout(timerId);                      // cancel the previous wait…
    timerId = setTimeout(() => fn(...args), ms); // …and start a new one
  };
}
const search = debounce((query) => console.log('searching for', query), 300);
search('j');
search('ja');
search('jav');
search('java');                                 // four calls, no pause between
console.log('typed 4 letters');
```

▶ CONSOLE OUTPUT

```
typed 4 letters
searching for java
```

JS · practice/timers.js, section 2: the same box, with and without debounce

```
const debouncedFilter = debounce(runFilter, 300);

searchInput.addEventListener('input', () => {
  keyPresses++;
  if (useDebounce.checked) debouncedFilter(searchInput.value);   // waits for a pause
  else runFilter(searchInput.value);                             // runs on every key
});
```

Typing "array" quickly	Without debounce	With debounce (300 ms)
Key presses	5	5
Times the filter runs	5	**1**
If each run asked a server	5 requests, 4 wasted	1 request

⇄ MORE WAYS **Three ways to handle events that fire very often**

WAY 1 Run every time

```
input.addEventListener('input', runFilter);
```

Fine when the work is cheap, like filtering 20 topics.

WAY 2 Debounce: after a pause

```
input.addEventListener('input', debounce(runFilter, 300));
```

Search boxes, autosave, "check if the username is free", window resizing.

WAY 3 Throttle: at most every X ms

```
function throttle(fn, ms) {
  let last = 0;
  return (...args) => {
    const now = Date.now();
    if (now - last >= ms) {
      last = now;
      fn(...args);
    }
  };
}
```

For continuous actions where you want regular updates while they happen: scroll position, dragging, a progress bar.

19.8 localStorage: data that survives a reload

Everything in your variables disappears when the page reloads. `localStorage` is a small store in the browser, one per website, that keeps **strings** under **keys** until the user or your code removes them, even after the computer restarts.

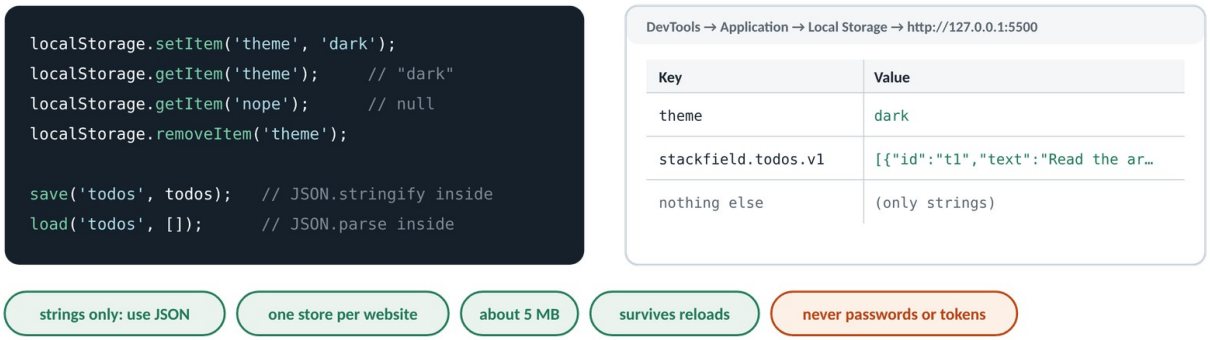


Figure 19.4 · The four localStorage calls, and the same data in DevTools → Application → Local Storage

JS · **setItem, getItem, removeItem**

```
localStorage.setItem('theme', 'dark');             // save (key, value)
console.log(localStorage.getItem('theme'));
console.log(localStorage.getItem('nope'));          // never saved: null
localStorage.setItem('visits', 3);                  // numbers become strings!
console.log(localStorage.getItem('visits'), typeof localStorage.getItem('visits'));
localStorage.removeItem('theme');
console.log(localStorage.length);                   // only 'visits' is left
```

▶ CONSOLE OUTPUT

```
dark
null
3 string
1
```

JS · **arrays and objects need JSON**

```
const todos = [{ id: 't1', text: 'Read', done: true }];
localStorage.setItem('todos-bad', todos);                // ✗ no JSON
console.log(localStorage.getItem('todos-bad'));
localStorage.setItem('todos', JSON.stringify(todos));    // ✓ save as JSON text
const loaded = JSON.parse(localStorage.getItem('todos'));
console.log(loaded[0].text, loaded.length);
```

▶ CONSOLE OUTPUT

```
[object Object]
Read 1
```

Fact	What it means for you
Strings only	`JSON.stringify` to save, `JSON.parse` to load
One store per website (origin)	`localhost:5500` and your Vercel site have separate stores
About 5 MB	plenty for settings and to-do lists, not for photos
Readable by any script on the page	**never** store passwords, tokens or personal data
Can be blocked or full	wrap it in `try / catch` (Day 20's `storage.js`)
The user can clear it	always have a default: `getItem(…) ?? 'light'`

⇄ MORE WAYS Three places to keep data in the browser

WAY 1 A variable

```
let theme = 'dark';
```

Gone on reload. Perfect for things that don't need to last.

WAY 2 sessionStorage

```
sessionStorage.setItem('step', '2');
```

Same methods as localStorage, but only for **this tab**, until it closes. Good for a half-filled form.

WAY 3 localStorage

```
localStorage.setItem('theme', 'dark');
```

Stays until cleared. Settings, drafts, offline lists. (For big or shared data: a server and database, from Month 3.)

JS · practice/theme.js (shortened): save a choice, read it back

```
const KEY = 'theme';

function applyTheme(theme) {
  document.documentElement.dataset.theme = theme;       // <html data-theme="dark">
  buttons.forEach((button) => {
    button.setAttribute('aria-pressed', String(button.dataset.theme === theme));
  });
}

themeButtons.addEventListener('click', (event) => {    // delegation again
  const button = event.target.closest('button');
  if (!button) return;
  localStorage.setItem(KEY, button.dataset.theme);      // 1. save
  applyTheme(button.dataset.theme);                     // 2. show
});

// On page load: the saved choice, or the operating system's setting
const prefersDark = matchMedia('(prefers-color-scheme: dark)').matches;
const systemTheme = prefersDark ? 'dark' : 'light';
applyTheme(localStorage.getItem(KEY) ?? systemTheme);
```

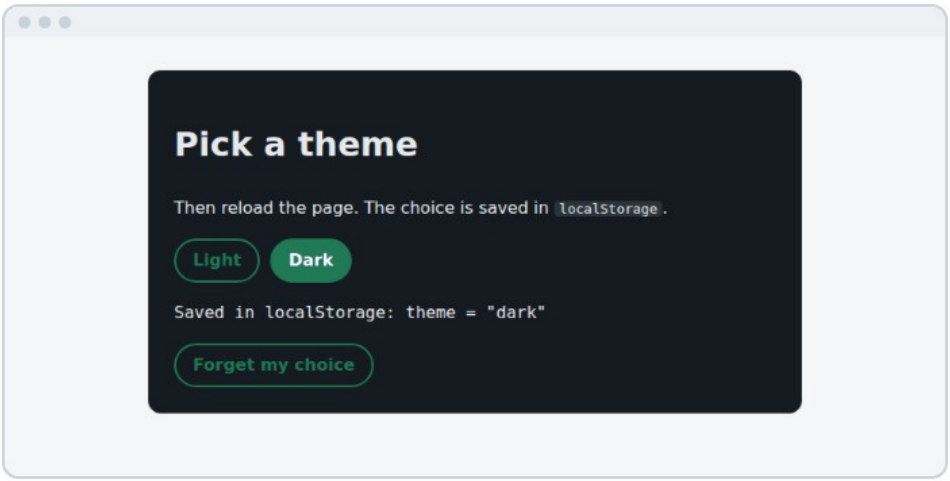


Figure 19.5 · practice/theme.html: choose Dark, reload, and it's still dark

> **TIP**
>
> DevTools → **Application** → **Local Storage** → your site shows every saved key. You can edit or delete values there, which is the fastest way to test "what if the saved data is missing or broken?"

19.9 State and render: the to-do list, part 1



Figure 19.6 · Data flows one way: an event changes the state, render() redraws the page from it

★ MUST KNOW *the ideas to remember from today*

- ✓ The truth lives in the `todos` **array** (the state), not in the page.
- ✓ Event handlers only **change the array**…
- ✓ …then call `render()`, which rebuilds the list from the array.

Because the page is always redrawn from the array, the two can never disagree. You never ask the page "is this task done?"; you ask the array. It's the same idea as 17.8, now with objects and three kinds of change.

JS · the three changes, on the data alone

```
let todos = [
  { id: 't1', text: 'Read the arrays chapter', done: true },
  { id: 't2', text: 'Solve Two Sum', done: false },
];
todos.push({ id: 't3', text: 'Build the to-do widget', done: false });   // add
todos.find((t) => t.id === 't2').done = true;                           // toggle
todos = todos.filter((t) => t.id !== 't1');                              // delete
console.log(todos.map((t) => `${t.done ? '☑' : '□'} ${t.text}`));
console.log(todos.filter((t) => !t.done).length, 'left');
```

▶ CONSOLE OUTPUT

```
['☑ Solve Two Sum', '□ Build the to-do widget']
1 left
```

JS · widgets/js/main.js, section 5: one row, and render()

```
function createTodoItem(todo) {
  const item = document.createElement('li');
  item.className = 'todo-item';
  item.classList.toggle('is-done', todo.done);
  item.dataset.id = todo.id;                 // which task this row shows

  const checkbox = document.createElement('input');
  checkbox.type = 'checkbox';
  checkbox.className = 'todo-toggle';
  checkbox.id = `todo-${todo.id}`;
  checkbox.checked = todo.done;

  const label = document.createElement('label');
  label.className = 'todo-text';
  label.htmlFor = checkbox.id;               // clicking the text ticks the box
  label.textContent = todo.text;             // safe for any text

  const remove = document.createElement('button');
  remove.type = 'button';
  remove.className = 'icon-button todo-delete';
  remove.textContent = '×';
  remove.setAttribute('aria-label', `Delete: ${todo.text}`);

  item.append(checkbox, label, remove);
  return item;
}

function render() {
  todoList.replaceChildren(...todos.map(createTodoItem));
  const left = todos.filter((todo) => !todo.done).length;
  todoCount.textContent = `${left} ${left === 1 ? 'task' : 'tasks'} left`;
}
```

JS · section 5: three handlers, each "change the array, then render"

```
// ADD
todoForm.addEventListener('submit', (event) => {
  event.preventDefault();
  const text = todoInput.value.trim();
  if (text === '') return;                           // ignore empty tasks
  todos.push({ id: createId(), text, done: false });
  todoInput.value = '';
  render();
});

// TOGGLE: one delegated 'change' listener for every checkbox
todoList.addEventListener('change', (event) => {
  if (!event.target.matches('.todo-toggle')) return;
  const id = event.target.closest('.todo-item').dataset.id;
  todos.find((t) => t.id === id).done = event.target.checked;
  render();
  document.getElementById(`todo-${id}`).focus();    // the old checkbox was replaced
});

// DELETE: one delegated 'click' listener for every × button
todoList.addEventListener('click', (event) => {
  const button = event.target.closest('.todo-delete');
  if (!button) return;
  const id = button.closest('.todo-item').dataset.id;
  todos = todos.filter((t) => t.id !== id);
  render();
  todoInput.focus();                                 // the row that had focus is gone
});

render();
```

Event	Changes the array with	Then
`submit`	`todos.push({ id, text, done: false })`	`render()`, clear the input
`change` (checkbox)	`find(…)` then `.done = checked`	`render()`, focus the new checkbox
`click` (×)	`todos = todos.filter(…)`	`render()`, focus the input

Why the focus lines? `render()` throws the old elements away and builds new ones. If the element that had keyboard focus is replaced, focus falls back to the top of the page, and a keyboard user loses their place. Putting focus back on the matching new element (or the input) fixes it.

Where do ids come from? `createId()` joins the time and a few random characters: `Date.now().toString(36) + Math.random().toString(36).slice(2, 6)`, e.g. `'mgaxv3k0q7zp'`. Unique enough for one person's list. (Browsers also have `crypto.randomUUID()`, but it only works on HTTPS and `localhost`.)

19.10 DSA: LeetCode 1, Two Sum

The problem. Given `nums` and a `target`, return the **indexes** of the two numbers that add up to `target`. There is exactly one answer, and you can't use the same item twice. `nums = [2, 7, 11, 15]`, `target = 9` → `[0, 1]` (2 + 7). This is LeetCode's problem number 1 and one of the most-asked interview questions anywhere.

```
twoSum([3, 2, 4], target = 6)
```

i	nums[i]	partner = 6 − nums[i]	seen before this step	what happens
0	3	3	{ }	not seen → seen.set(3, 0)
1	2	4	{ 3 → 0 }	not seen → seen.set(2, 1)
2	**4**	**2**	**{ 3 → 0, 2 → 1 }**	**seen.has(2)! → return [1, 2]**

The Map remembers every number already passed (value → index), so each partner check is instant.
One pass: O(n). Checking every pair instead would be O(n²).

Figure 19.7 · One pass with a Map: at index 2 the partner (2) has already been seen

1. **Flip the question:** for each number, the partner you need is `target - number`. For 7 and target 9, the partner is 2.
2. **Slow way:** for each number, search the rest of the array for its partner: every pair, O(n²).
3. **Fast way:** walk once, and **remember** every number you pass in a Map (number → its index). At each number, ask the Map: "have I seen my partner?" If yes, you're done.
4. **Order matters:** check the Map **before** adding the current number, so a number is never paired with itself.

⇄ MORE WAYS Two Sum three ways (dsa/1-two-sum.js)

WAY 1 Try every pair

```
function twoSumPairs(nums, target) {
  for (let i = 0; i < nums.length; i++) {
    for (let j = i + 1; j < nums.length; j++) {
      if (nums[i] + nums[j] === target) return [i, j];
    }
  }
  return [];
}
```

Correct and easy to explain first in an interview. Time $O(n^2)$.

WAY 2 One pass with a Map

```
function twoSum(nums, target) {
  const seen = new Map();                 // value → its index
  for (let i = 0; i < nums.length; i++) {
    const partner = target - nums[i];
    if (seen.has(partner)) {
      return [seen.get(partner), i];      // the partner came earlier
    }
    seen.set(nums[i], i);
  }
  return [];
}
```

Time **O(n)**, extra space O(n). The answer interviewers are waiting for, and the same "remember what you've seen" idea as Contains Duplicate.

WAY 3 The same with a plain object

```
function twoSumObject(nums, target) {
  const seen = {};
  for (let i = 0; i < nums.length; i++) {
    const partner = target - nums[i];
    if (partner in seen) return [seen[partner], i];
    seen[nums[i]] = i;
  }
  return [];
}
```

Works because the keys are numbers turned into strings. A Map is cleaner: it keeps any type of key as it is.

▶ RESULT

```
twoSum([2, 7, 11, 15], 9)       → [0, 1]
twoSum([3, 2, 4], 6)            → [1, 2]
twoSum([3, 3], 6)               → [0, 1]
twoSum([-1, -2, -3, -4, -5], -8) → [2, 4]
```

JS · **Map in one minute**

```
const seen = new Map();
seen.set('apple', 3);
seen.set(42, 'a number key');           // any type of key, kept as it is
console.log(seen.get('apple'), seen.has(42), seen.size);
seen.delete('apple');
console.log(seen);
```

▶ CONSOLE OUTPUT

```
3 true 2
Map(1) {42 => 'a number key'}
```

	Plain object	Map
Keys	strings (numbers become strings)	any value: numbers, objects…
Size	`Object.keys(obj).length`	`map.size`
Read / write	`obj[key]` / `obj[key] = v`	`map.get(key)` / `map.set(key, v)`
Best for	records with known keys (an expense)	lookups and counting (Two Sum, anagrams)

Build it, step by step

1. Open `practice/closures.html` and write the five functions in `closures-starter.js` until every test passes.
2. Open `practice/timers.html`. Untick "Use debounce" and type quickly; tick it and compare the counters.
3. Open `practice/theme.html`, choose Dark, reload. Find the value in DevTools → Application → Local Storage, then delete it.
4. In the widgets, write the to-do list: the `todos` array, `createTodoItem`, `render`, and the three handlers.
5. Test: add (and try an empty task), tick, untick, delete, all with the keyboard too.
6. Solve **Two Sum**, then compare with `dsa/1-two-sum.js`. Explain the Map version out loud.
7. Commit: `git commit -m "Day 19: closures, timers, localStorage and the to-do list"` and push.

★ MUST KNOW *the ideas to remember from today*

- ✓ A closure = a function + the variables it remembers from where it was created.
- ✓ Closures give private state (`createCounter`) and configured helpers (`makeMultiplier`, `debounce`).
- ✓ In loops, `let` makes a new variable per lap; `var` shares one.
- ✓ Timer callbacks run after the current code finishes, even with 0 ms.
- ✓ Keep timer ids; `clearTimeout` / `clearInterval` what you start.
- ✓ Debounce = wait for a pause, then run once.
- ✓ `localStorage` stores strings: `JSON.stringify` to save, `JSON.parse` to load; never secrets.
- ✓ State → `render()`: handlers change the data, `render()` redraws the page.

WATCH OUT

- `setTimeout(fn(), 500)`: `fn` runs immediately.
- Starting a second interval because the Start button wasn't disabled.
- Saving an array without JSON: you get `"[object Object]"`.
- Forgetting that `getItem` gives `null` when nothing is saved.
- Changing the page directly in a handler instead of changing the array and rendering.
- Losing keyboard focus after a re-render.

QUESTIONS LEARNERS ASK

Q Do closures use a lot of memory?

Only for the variables they actually keep alive, which is tiny in normal code. It becomes a problem only if you keep huge data alive by accident, e.g. an interval you never clear.

Q Why not just use a global variable instead of a closure?

Globals can be changed by any script on the page and clash with other names. A closure keeps the data private and lets you create as many independent copies as you need.

Q Is rebuilding the whole list on every change slow?

For tens or hundreds of items, no: it's instant. For thousands, you'd update only what changed, which is what React does for you in Month 2.

Q Is localStorage safe?

For preferences and to-do items, yes. For passwords, tokens or personal data, no: any script on the page (including an XSS attack) can read it.

More practice

MORE PRACTICE · try each one, then check the Answers at the back

1. Write `createToggle()` returning a function that returns `true`, `false`, `true`, … on each call.
2. Write `rateLimit(fn, max)` that lets `fn` run at most `max` times, then ignores calls.
3. Print "3", "2", "1", "Go!" one second apart.
4. Count how many times a page has been opened, and show "Visit number N".
5. Add "Mark all done" to the to-do list.
6. Solve Two Sum when the array is already **sorted**, without a Map.

Check yourself

1. What is a closure? Give an example from this week.
2. Why do you need `JSON.stringify` before saving to localStorage?
3. What does debouncing do, and where would you use it?

*Answer out loud before you turn to the **Answers** section at the back.*

DONE WHEN · tick these off, then mark the day done in your Logbook

- ☐ All closure tests pass, and you can explain `createCounter` and `debounce` out loud
- ☐ You've seen the debounce difference and found the theme in Local Storage
- ☐ The to-do list adds, toggles and deletes, with the mouse and the keyboard
- ☐ Two Sum accepted; committed and pushed

Resources

All free. Every name is a link, in Word and in the PDF.

Resource (click to open)	Use it for
javascript.info: Variable scope, closure javascript.info/closure	The clearest explanation of closures, with pictures
MDN: Closures developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Closures	The official guide, with practical examples
javascript.info: Scheduling: setTimeout and setInterval javascript.info/settimeout-setinterval	Timers, with tasks
javascript.info: Decorators and forwarding javascript.info/call-apply-decorators	Includes debounce and throttle tasks
javascript.info: LocalStorage, sessionStorage javascript.info/localstorage	Both stores, with examples
MDN: Window.localStorage developer.mozilla.org/en-US/docs/Web/API/Window/localStorage	The reference
MDN: Map developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Map	set, get, has, and Map vs Object
Loupe latentflip.com/loupe	Watch the call stack and queue animate your code
What the heck is the event loop anyway? (video) youtube.com/watch?v=8aGhZQkoFbQ	Philip Roberts' famous 26-minute talk
LeetCode 1: Two Sum leetcode.com/problems/two-sum	Today's problem

DAY

20

WEEK 3 · THURSDAY 15 OCTOBER

Build day: modules, the finished to-do list, and going live

Split the code into tidy files, finish the last features, test like a user, and publish.

TIME	YOU BUILD	CODE FOLDER
3–4 hours	**The widgets as ES modules, a to-do list with filters, editing and saving, live on Vercel; and Best Time to Buy and Sell Stock**	`day20-ship`

Today you will

- ✓ Split a program into ES modules with `export` and `import`
- ✓ Explain why modules need a web server, and recognise the CORS error
- ✓ Write a small storage module with JSON and `try / catch`
- ✓ Keep all of a widget's data in one `state` object, and save on every change
- ✓ Update arrays of objects without mutating them (`map`, `filter`, spread)
- ✓ Replace `if / else` chains with lookup objects
- ✓ Finish the to-do list: filters, editing, "Clear done" and empty states
- ✓ Test the widgets like a real user, then deploy them on Vercel
- ✓ Solve LeetCode 121, Best Time to Buy and Sell Stock, two ways

Key terms

Term	Meaning
Module	A JavaScript file with its own scope that shares only what it exports
export / import	Make a value available to other files / use a value from another file
Named export	`export function setupTodo…`, imported by the same name in `{ }`
Entry point	The first module the page loads (`main.js`); it imports the rest
CORS	The browser's rules for loading files from another origin
Immutable update	Making a new array or object with the change, instead of editing the old one
Lookup object	An object that maps a key (a filter name, a key name) to a value or function
Empty state	What a list shows when it has nothing to show
Deploy	Publish the site on a server so anyone can open it

20.1 Why split the code into modules?

IN PLAIN WORDS

A restaurant kitchen has **stations**: grill, salads, desserts. Each has its own tools, and nobody borrows the grill chef's knife by accident. The head chef calls out orders and each station does its part. Modules are stations: each file keeps its own variables, shares only what it **exports**, and `main.js` is the head chef.

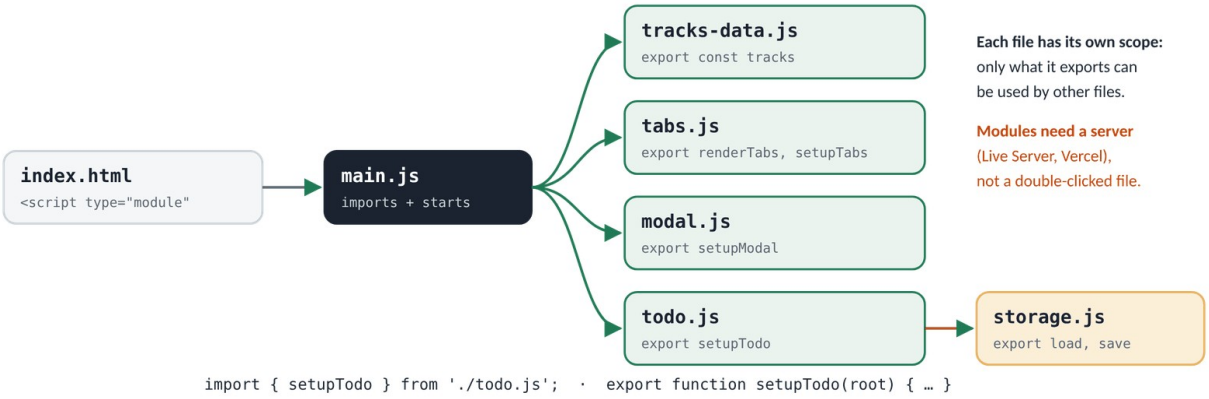


Figure 20.1 · main.js imports one module per widget; todo.js imports storage.js

One 250-line file (Day 19)	Modules (Day 20)
Scroll to find the tabs code	Open `tabs.js`
Every variable is visible everywhere	Each file has its own scope; only exports are shared
Two widgets both want a variable called `list`	No clash: each module has its own `list`
Copy code to reuse it	`import { load, save } from './storage.js'` anywhere
Helpers anyone can call	`selectTab` isn't exported, so it stays private to `tabs.js`

20.2 export and import

JS · what each module shares

```
// tracks-data.js: the data, nothing else
export const tracks = [ /* … the four track objects … */ ];

// tabs.js: two exports and a private helper
export function renderTabs(root, items) { /* … */ }
export function setupTabs(root) { /* … */ }
function selectTab(root, tab) { /* … */ }     // not exported: private to this file

// storage.js
export function load(key, fallback) { /* … */ }
export function save(key, value) { /* … */ }
```

JS · widgets/js/main.js: the entry point connects the modules

```
import { tracks } from './tracks-data.js';
import { renderTabs, setupTabs } from './tabs.js';
import { setupModal } from './modal.js';
import { setupTodo } from './todo.js';

// 1. Tabs
const tracksRoot = document.getElementById('tracks');
renderTabs(tracksRoot, tracks);
setupTabs(tracksRoot);

// 2. Modal
setupModal({
  openButton: document.getElementById('open-call'),
  dialog: document.getElementById('call-dialog'),
  status: document.getElementById('call-status'),
});

// 3. To-do list
setupTodo(document.getElementById('todo-title').closest('.widget'));

// 4. The year in the footer
document.getElementById('year').textContent = new Date().getFullYear();
```

HTML · widgets/index.html: one attribute changes

```
<!-- Before (Days 16–19) -->
<script src="js/main.js" defer></script>

<!-- After (Day 20): main.js can import other files.
     Modules are deferred automatically. -->
<script type="module" src="js/main.js"></script>
```

Write	Means
`export function setupTodo(root) {…}`	a named export
`import { setupTodo } from './todo.js';`	import it by the same name, in `{ }`
`import { load as loadData } from './storage.js';`	import it under a different name
`import * as storage from './storage.js';`	everything as one object: `storage.load(…)`
`export default function setupTodo…`	a default export (one per file)…
`import setupTodo from './todo.js';`	…imported without `{ }`, under any name

⇄ MORE WAYS **Named or default exports?**

WAY 1 Named exports (this course)

```
export function setupModal() {}
import { setupModal } from './modal.js';
```

The name must match, so typos are caught at once, and editors can suggest and auto-import it. A file can export many things.

WAY 2 A default export

```
export default function setupModal() {}
import openThePopup from './modal.js';   // any name works
```

Common in React components (Month 2). Flexible, but the same thing can end up with different names in different files.

WATCH OUT

- In the browser, the path needs `./` and the `.js` ending: `'./todo.js'`, not `'todo'`. (Build tools let you leave them out later; plain browsers don't.)
- A module's top-level variables are **not** globals. You can't type `tracks` in the Console any more. That's the point, but it surprises everyone once. For debugging, add a `console.log` or a breakpoint.
- Modules always run in **strict mode**: mistakes that older scripts ignored, such as assigning to an undeclared variable, now throw errors. Good: bugs show up sooner.

20.3 Modules need a web server

Double-click `index.html` and the widgets are dead. The Console explains:

▶ CONSOLE · a module page opened as a file

```
Access to script at 'file:///C:/…/widgets/js/main.js' from origin 'null'
has been blocked by CORS policy: Cross origin requests are only supported
for protocol schemes: … http, https.
```

For security, browsers only load modules over `http://` or `https://`, never from `file://` (a page on your disk could otherwise read other files on your disk). The fix is not in your code: open the page through a **web server**. In VS Code that's **Live Server** (right-click → Open with Live Server, giving `http://127.0.0.1:5500/…`), and online it's Vercel.

⇄ MORE WAYS **Three ways to run a local web server**

WAY 1 Live Server (VS Code)

Right-click `index.html` → **Open with Live Server**. It also reloads the page when you save. What this course uses.

WAY 2 Python

```
python -m http.server 8000     # then open http://localhost:8000
```

Built in wherever Python is installed. Run it in the folder with `index.html`.

WAY 3 Node.js

```
npx serve .                    # then open the address it prints
```

Needs Node.js (you'll install it in Month 3 anyway).

20.4 A storage module that never crashes the page

JS · widgets/js/storage.js

```
export function load(key, fallback) {
  try {
    const text = localStorage.getItem(key);
    return text === null ? fallback : JSON.parse(text);   // null = nothing saved yet
  } catch (error) {
    console.warn(`Could not load "${key}":`, error.message);
    return fallback;
  }
}

export function save(key, value) {
  try {
    localStorage.setItem(key, JSON.stringify(value));
  } catch (error) {
    console.warn(`Could not save "${key}":`, error.message);
  }
}
```

JS · trying load() with good, missing and broken data

```
function load(key, fallback) {
  try {
    const text = localStorage.getItem(key);
    return text === null ? fallback : JSON.parse(text);
  } catch (error) {
    console.warn(`Could not load "${key}":`, error.name);   // (storage.js: .message)
    return fallback;
  }
}
localStorage.setItem('demo', JSON.stringify([1, 2]));
console.log(load('demo', []));                    // saved data
console.log(load('missing', ['default']));        // nothing saved: the fallback
localStorage.setItem('broken', '{not json');      // someone edited it by hand
console.log(load('broken', []));                  // broken: a warning + the fallback
```

▶ CONSOLE OUTPUT

```
[1, 2]
['default']
⚠ Could not load "broken": SyntaxError
[]
```

This copy logs `error.name` to keep the line short. The real `storage.js` logs `error.message`, the full explanation: `Expected property name or '}' in JSON at position 1 (line 1 column 2)`.

What can go wrong	What happens	The widget
Nothing saved yet (first visit)	`getItem` returns `null`	uses the sample tasks
The saved text isn't valid JSON	`JSON.parse` throws → `catch`	uses the samples, logs a warning
Storage is blocked or full	`setItem` throws → `catch`	keeps working, just doesn't save

> **TIP**
>
> The key is `'stackfield.todos.v1'`. The prefix avoids clashes with other projects on `localhost:5500` (they all share one store). The `v1` lets you change the shape of the data later: save under `v2` and old, differently-shaped data is simply ignored.

20.5 One state object, saved on every change

Yesterday the list had one piece of state (`todos`). The finished version needs three: the tasks, which filter is chosen, and which task (if any) is being edited. They live together in one `state` object, and **every** change goes through `update()`, which saves and then renders.

JS · widgets/js/todo.js: the state (shortened)

```
import { load, save } from './storage.js';

const STORAGE_KEY = 'stackfield.todos.v1';

export function setupTodo(root) {
  // … find the form, input, list, empty message, counter and buttons inside root

  const state = {
    todos: load(STORAGE_KEY, SAMPLE_TODOS),   // saved tasks, or the samples
    filter: 'all',                            // 'all' | 'active' | 'done'
    editingId: null,                          // the task being edited, if any
  };

  function update() {                         // every change to the tasks ends here
    save(STORAGE_KEY, state.todos);
    render();
  }
  // … render(), the actions and the listeners
}
```

State key	Changed by	Saved?
`todos`	add, toggle, edit, delete, clear done	yes: `update()` saves, then renders
`filter`	the All / Active / Done buttons	no: `render()` only (starts at All each visit)
`editingId`	✎ (start), Enter / Escape / click away (end)	no

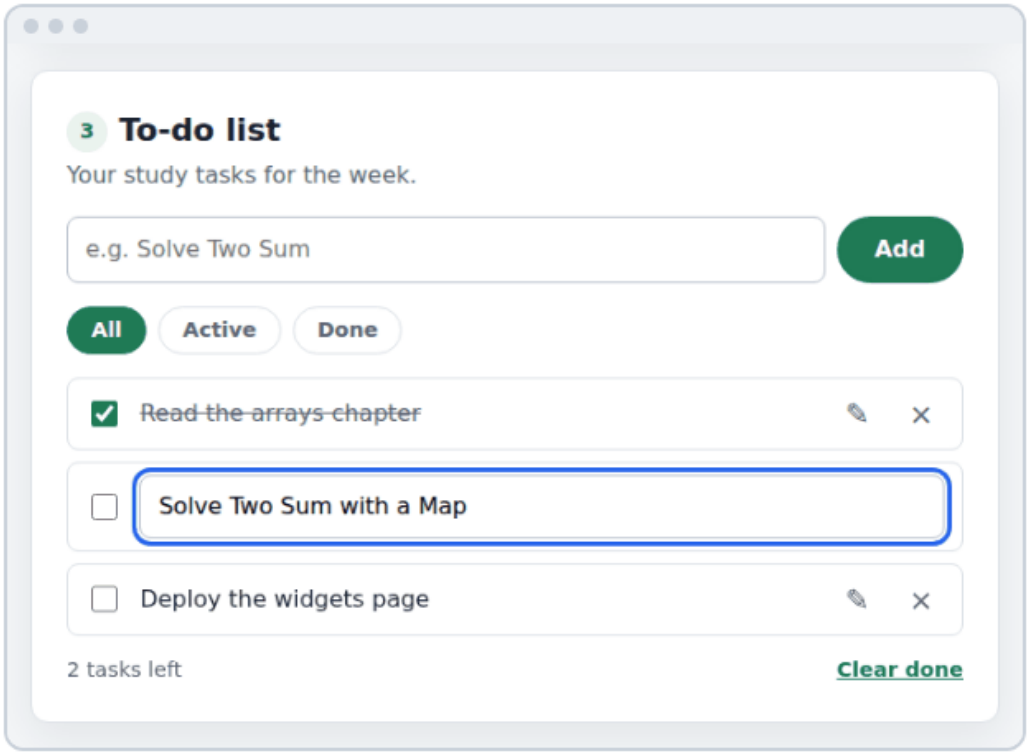


Figure 20.2 · The finished to-do list: filters, a task being edited, "Clear done"

20.6 Changing data without mutating it

On Day 19 the handlers changed the array in place (`push`, `.done = …`). The finished list uses the **immutable** style: every change builds a **new** array (and new objects for the tasks that changed). It's the style React needs in Month 2, and it makes changes easy to follow.

JS · add, toggle, edit and delete, immutably

```
let todos = [
  { id: 't1', text: 'Read', done: true },
  { id: 't2', text: 'Code', done: false },
];
const before = todos;
todos = [...todos, { id: 't3', text: 'Ship', done: false }];               // add
todos = todos.map((t) => (t.id === 't2' ? { ...t, done: true } : t));      // toggle
todos = todos.map((t) => (t.id === 't3' ? { ...t, text: 'Ship it' } : t)); // edit
todos = todos.filter((t) => t.id !== 't1');                                // delete
console.log(todos.map((t) => `${t.text}: ${t.done}`));
console.log(before.length, before[1].done);      // the old array is untouched
```

CONSOLE OUTPUT

```
['Code: true', 'Ship it: false']
2 false
```

Change	Immutable way	Mutating way (Day 19)
Add	`[...todos, newTodo]`	`todos.push(newTodo)`
Change one	`todos.map((t) => t.id === id ? { ...t, done } : t)`	`todos.find(…).done = done`
Remove one	`todos.filter((t) => t.id !== id)`	`todos.splice(index, 1)`
Remove many	`todos.filter((t) => !t.done)`	a loop of `splice` (easy to get wrong)

⇄ MORE WAYS **Two ways to toggle a task**

WAY 1 Mutate: find it and change it

```
const todo = state.todos.find((t) => t.id === id);
todo.done = done;
```

Short and fine in small programs. But any other code holding that object sees it change too.

WAY 2 Immutable: map to a new array

```
state.todos = state.todos.map((todo) =>
  (todo.id === id ? { ...todo, done } : todo));
```

Only the changed task becomes a new object; the rest are reused. You can compare old and new, keep an undo history, and React can tell what changed.

20.7 Filters with a lookup object

JS · a filter name → a test function

```
const FILTERS = {                    // one test function per filter
  all: () => true,
  active: (todo) => !todo.done,
  done: (todo) => todo.done,
};
const todos = [
  { text: 'Read', done: true },
  { text: 'Code', done: false },
  { text: 'Ship', done: false },
];
for (const name of Object.keys(FILTERS)) {
  console.log(name, todos.filter(FILTERS[name]).map((t) => t.text));
}
```

▶ CONSOLE OUTPUT

```
all ['Read', 'Code', 'Ship']
active ['Code', 'Ship']
done ['Read']
```

Because `FILTERS[state.filter]` **is** a function, it can go straight into `filter()`. The same trick gives each filter its own empty-state message, and `FILTERS.active` doubles as the "tasks left" counter and the "Clear done" logic.

JS · todo.js: render() with filters and empty states

```
const EMPTY_MESSAGES = {
  all: 'Nothing here yet. Add your first task above.',
  active: 'No active tasks. Nice work!',
  done: 'No finished tasks yet.',
};

function render() {
  const visible = state.todos.filter(FILTERS[state.filter]);
  list.replaceChildren(...visible.map(createItem));

  empty.hidden = visible.length > 0;                  // a message, not a blank box
  empty.textContent = EMPTY_MESSAGES[state.filter];

  const left = state.todos.filter(FILTERS.active).length;
  count.textContent = `${left} ${left === 1 ? 'task' : 'tasks'} left`;
  clearDone.disabled = !state.todos.some(FILTERS.done);

  filterGroup.querySelectorAll('.filter').forEach((button) => {
    const isOn = button.dataset.filter === state.filter;
    button.setAttribute('aria-pressed', String(isOn));
  });
}
```

HTML · index.html: data-filter says which filter; aria-pressed says which is on

```
<div class="todo-filters" role="group" aria-label="Show tasks">
  <button class="filter" type="button" data-filter="all"
          aria-pressed="true">All</button>
  <button class="filter" type="button" data-filter="active"
          aria-pressed="false">Active</button>
  <button class="filter" type="button" data-filter="done"
          aria-pressed="false">Done</button>
</div>
```

⇄ MORE WAYS Three ways to pick the filter function

WAY 1 if / else if

```
let test;
if (filter === 'active') test = (t) => !t.done;
else if (filter === 'done') test = (t) => t.done;
else test = () => true;
```

Works; grows longer with every new filter.

WAY 2 switch

```
switch (filter) {
  case 'active': return todos.filter((t) => !t.done);
  case 'done': return todos.filter((t) => t.done);
  default: return todos;
}
```

Tidier for many fixed options.

WAY 3 A lookup object (the widget)

```
const visible = todos.filter(FILTERS[filter]);
```

Adding a filter ("today", "urgent") is one new line in `FILTERS`, and nothing else changes.

20.8 Editing a task

Editing is a small state machine. `state.editingId` holds the id of the task being edited, or `null`. When `render()` draws that task, it shows a text box instead of the label and buttons, and puts the cursor in it.

JS · todo.js: the edit flow (shortened)

```
// In the list's click listener: ✎ was clicked
if (event.target.closest('.todo-edit-button')) {
  state.editingId = item.dataset.id;
  render();                          // createItem() now shows an <input> for it
}

// While editing: Enter saves, Escape cancels
list.addEventListener('keydown', (event) => {
  if (!event.target.matches('.todo-edit')) return;
  const id = event.target.closest('.todo-item').dataset.id;
  if (event.key === 'Enter') saveEdit(id, event.target.value);
  if (event.key === 'Escape') cancelEdit();
});

// Clicking somewhere else saves too. 'focusout' bubbles ('blur' doesn't).
list.addEventListener('focusout', (event) => {
  if (!event.target.matches('.todo-edit') || state.editingId === null) return;
  saveEdit(event.target.closest('.todo-item').dataset.id, event.target.value);
});

function saveEdit(id, text) {
  state.editingId = null;
  const clean = text.trim();
  if (clean !== '') {                 // an empty edit keeps the old text
    state.todos = state.todos.map((t) => (t.id === id ? { ...t, text: clean } : t));
  }
  update();
}
```

The person...	Code	Result
clicks ✎	`editingId = id`, `render()`	a text box with the text selected
presses Enter	`saveEdit(id, value)`	saved (unless empty), back to normal
presses Escape	`cancelEdit()`	the old text, nothing saved
clicks elsewhere / Tabs away	`focusout` → `saveEdit`	saved

Why the `state.editingId === null` check? When Enter saves, `render()` removes the text box. Removing a focused element fires `focusout`, which would save a **second** time. `saveEdit` sets `editingId` to `null` first, so the second call returns early. Small guards like this are what make a widget feel solid.

20.9 Test it like a real user

Before you deploy, spend 15 minutes trying to break the page. Work through this list with the page on Live Server, DevTools open, and the Console visible.

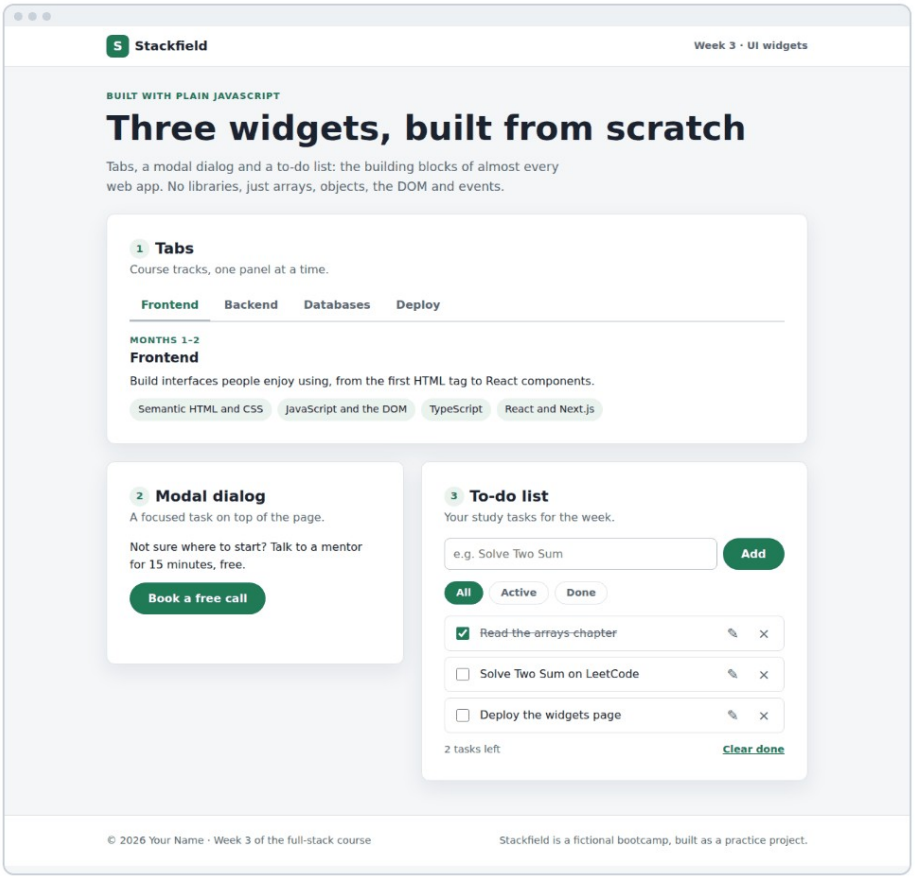
- ☐ Keyboard only: reach and use every tab, the modal and every to-do button with Tab, Enter, Space, arrows and Escape
- ☐ Add, tick, edit (Enter, Escape, click away), delete and "Clear done"; try an empty task and an empty edit
- ☐ Each filter, including the empty-state message for each

- [ ] Reload: the tasks are still there. Close the tab and reopen: still there
- [ ] DevTools → Application → Local Storage: change the saved value to `oops`, reload: the samples appear, with one warning
- [ ] Add a task called `<img src=x onerror=alert(1)>`: it shows as text and no alert appears
- [ ] Device toolbar at 360px: no sideways scrolling; the tabs scroll inside their row
- [ ] No red errors in the Console; Lighthouse Accessibility 90+

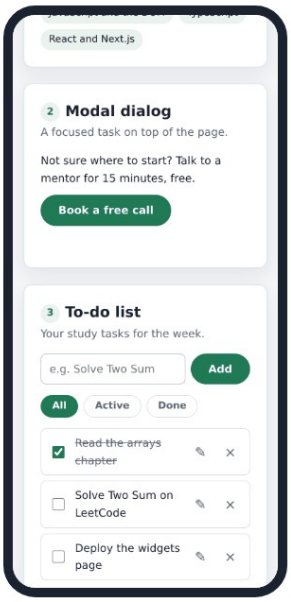
20.10 Deploy on Vercel

1. Create a new GitHub repository, e.g. `stackfield-widgets`, and push the `widgets` folder so that `index.html` is at the **top level** of the repository.
2. On vercel.com: **Add New… → Project → Import** your repository → **Deploy**. There is no build step: it's plain HTML, CSS and JavaScript.
3. Open the address Vercel gives you and run the checklist above once more on the live site (and on your phone).
4. Add the live link to the repository's README and to your portfolio page from Week 1.

Problem	Why	Fix
404: page not found	`index.html` isn't at the top level	Set **Root Directory** in the project settings to the folder that holds it
The page loads, nothing works	a wrong path in an `import` (letter case matters online!)	match file names exactly: `./todo.js`, not `./Todo.js`
You still see the old version	your browser cached it	**Ctrl+Shift+R** (hard reload)
Tasks from Live Server are missing	each site has its own localStorage	expected: the live site starts fresh



Laptop the modal and to-do list side by side



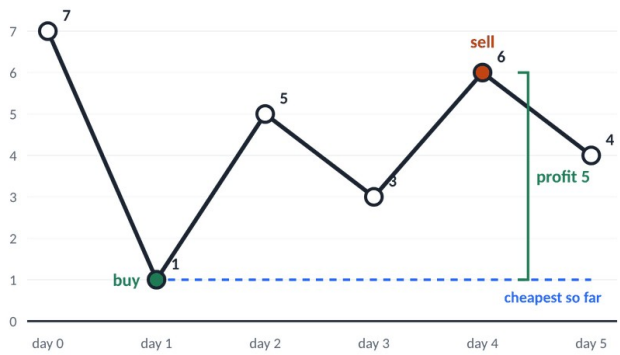
Phone one column

Figure 20.3 · The finished widgets page, live: laptop and phone

> **TIP**
>
> Windows doesn't care about capital letters in file names; Vercel's servers do. `import … from './Storage.js'` works on your laptop and fails online if the file is `storage.js`. Keep every file name lowercase.

20.11 DSA: LeetCode 121, Best Time to Buy and Sell Stock

The problem. `prices[i]` is a stock's price on day `i`. Buy on one day and sell on a **later** day. Return the biggest profit, or `0` if no trade makes money. `[7, 1, 5, 3, 6, 4]` → `5` (buy at 1, sell at 6). `[7, 6, 4, 3, 1]` → `0`.



day	price	cheapest	sell today	best
0	7	7	0	0
1	1	1	0	0
2	5	1	4	4
3	3	1	2	4
4	6	1	5	5
5	4	1	3	5

One pass: remember the lowest price so far; selling today earns price − cheapest.

Answer: 5 (buy at 1, sell at 6)

Figure 20.4 · Walking once: remember the cheapest price so far; selling today earns price − cheapest

1. **Slow way:** try every buy day with every later sell day. O(n^2).
2. **Key insight:** if you sell **today**, the best buy day was the **cheapest day before today**. So you only need to remember one number: the lowest price so far.
3. **Each day:** if today is cheaper than anything before, it's the new best buy day. Otherwise, check whether selling today beats the best profit so far.
4. **Edge cases:** prices that only fall → 0; one day → 0.

⇄ MORE WAYS Best Time to Buy and Sell Stock two ways (dsa/121-best-time-to-buy-and-sell-stock.js)

WAY 1 Every buy day with every later sell day

```
function maxProfitPairs(prices) {
  let best = 0;
  for (let buy = 0; buy < prices.length; buy++) {
    for (let sell = buy + 1; sell < prices.length; sell++) {
      best = Math.max(best, prices[sell] - prices[buy]);
    }
  }
  return best;
}
```

Obviously correct. Too slow for LeetCode's large tests: 100,000 prices ≈ 5 billion pairs.

WAY 2 One pass: the cheapest price so far

```
function maxProfit(prices) {
  let cheapest = Infinity;       // any real price is lower than Infinity
  let best = 0;
  for (const price of prices) {
    if (price < cheapest) {
      cheapest = price;          // a better day to buy
    } else if (price - cheapest > best) {
      best = price - cheapest;   // selling today beats every earlier plan
    }
  }
  return best;
}
```

Time **O(n)**, extra space **O(1)**: two variables, whatever the input size.

Day	Price	Cheapest so far	Sell today earns	Best so far
0	7	7	0	0
1	1	**1**	0	0
2	5	1	4	4
3	3	1	2	4
4	6	1	5	**5**
5	4	1	3	5

▶ RESULT

```
maxProfit([7, 1, 5, 3, 6, 4])    → 5
maxProfit([7, 6, 4, 3, 1])       → 0
maxProfit([3, 2, 6, 5, 0, 3])    → 4
```

The pattern, "walk once and keep the best-so-far", is the same one you used for the maximum in Day 15 and the richest customer on Day 16. Recognising patterns like this is most of what DSA practice is about.

Build it, step by step

1. Create `tracks-data.js`, `tabs.js`, `modal.js`, `storage.js` and `todo.js`; move each widget's code into its file and add `export`.
2. Make `main.js` import and start everything; change the script tag to `type="module"`; open with Live Server.
3. Finish the to-do list: filters with `aria-pressed`, the empty messages, editing (Enter, Escape, click away) and "Clear done".
4. Save with `storage.js` on every change and load on start. Reload to prove it.
5. Work through the test checklist in 20.9 and fix what you find.
6. Deploy on Vercel and test the live link on your phone.
7. Solve **Best Time to Buy and Sell Stock**; commit: `git commit -m "Day 20: widgets as ES modules, finished to-do list, deployed"` and push.

★ MUST KNOW *the ideas to remember from today*

- ✓ `export` shares; `import { name } from './file.js'` uses. Paths need `./` and `.js`.
- ✓ `<script type="module">` is deferred and strict, and needs a web server (not `file://`).
- ✓ Each module has its own scope: no accidental globals.
- ✓ Wrap `localStorage` + `JSON.parse` in `try / catch` and always have a fallback.
- ✓ One `state` object; every change → `update()` = save + render.
- ✓ Immutable updates: `[...a, x]`, `map` with `{ ...t, key }`, `filter`.
- ✓ Lookup objects replace `if / else` chains (`FILTERS[state.filter]`).
- ✓ Test with the keyboard, after a reload, with broken data, and at 360px before you ship.

WATCH OUT

- Opening a module page by double-clicking it (CORS error).
- Forgetting `.js` or `./` in an import path, or the wrong capital letters online.
- Expecting module variables to be available in the Console.
- Saving only in some handlers: route every change through `update()`.
- Saving twice when Enter and `focusout` both fire (guard with `editingId`).
- A list with no empty state: a blank box looks broken.

QUESTIONS LEARNERS ASK

Q Do I need a bundler like Vite for modules?

Not for a small site: every modern browser loads modules natively. From Month 2, React projects use Vite, which also bundles, minifies and lets you import packages.

Q Why not save the filter too?

You could (add it to what you save). Most apps start each visit on "All", so a task never seems to be missing because a filter was left on.

Q Isn't `map` over the whole array slower than changing one item?

For a to-do list it makes no difference you could measure. Clear, predictable updates are worth far more.

Q Can two modules import each other?

They can, but it causes confusing bugs. Keep the arrows one way: `main.js` → widgets → helpers like `storage.js`.

More practice

MORE PRACTICE · try each one, then check the Answers at the back

1. Add a fourth filter, "Today", for tasks created today.
2. Show "All done!" in the counter when no tasks are left.
3. Make "Clear done" ask "Delete 3 finished tasks?" first.
4. Move the modal into a module that can open **any** dialog: `setupModal({ openButton, dialog, onSubmit })`.
5. Write `move(todos, id, direction)` that moves a task up or down, immutably.
6. Best Time to Buy and Sell Stock II (LeetCode 122): you may trade many times. What's the idea?

Check yourself

1. Why does a page with `<script type="module">` fail when you double-click it?
2. How do you toggle a task's `done` without mutating the array?
3. What does `FILTERS[state.filter]` give you, and why is it better than an `if / else` chain?

*Answer out loud before you turn to the **Answers** section at the back.*

DONE WHEN · tick these off, then mark the day done in your Logbook

- ☐ The widgets are split into modules and work on Live Server
- ☐ Filters, editing, "Clear done", empty states and saving all work, and pass the checklist
- ☐ The page is live on Vercel, linked from your README and portfolio
- ☐ Best Time to Buy and Sell Stock accepted; committed and pushed

Resources

All free. Every name is a link, in Word and in the PDF.

Resource (click to open)	Use it for
javascript.info: Modules, introduction javascript.info/modules-intro	What modules are and how browsers load them
javascript.info: Export and Import javascript.info/import-export	Every export and import form
MDN: JavaScript modules developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Modules	The complete guide, with examples
MDN: Using the Web Storage API developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API/Using_the_Web_Storage_API	localStorage and sessionStorage in practice
MDN: try...catch developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/try...catch	Handling errors so the page keeps working
TodoMVC todomvc.com	The same to-do app built in dozens of frameworks: compare with yours
WAI-ARIA Authoring Practices: Keyboard interface w3.org/WAI/ARIA/apg/practices/keyboard-interface	Roving tabindex and other keyboard patterns
Vercel: Configuring a build vercel.com/docs/builds/configure-a-build	Including the Root Directory setting
LeetCode 121: Best Time to Buy and Sell Stock leetcode.com/problems/best-time-to-buy-and-sell-stock	Today's problem

DAY

21

WEEK 3 · FRIDAY 16 OCTOBER

Week 3 review: theory stops here

Prove it stuck, build one thing from scratch, and plan your first real project.

TIME	YOU BUILD	CODE FOLDER
2 hours	**A filterable list from scratch, one problem redone, and the Project 1 plan**	`day21-review`

Today's plan

Time	Block	What you do
30 min	Checkpoint quiz	10 questions below. Answer out loud first; answers are at the back
30 min	Build task	`practice/filter-task.html`: a filterable list, from an empty script
45 min	Project 1 plan	Features, a wireframe and the data shape for the Expense Tracker
15 min	Weekly retro	What worked, what was hard, what changes next week

The whole week on one page

Read this table once, slowly. Everything in it is something you should now be able to do or explain.

Topic	Remember
Arrays	Index from 0; `length`; `at(-1)`; `const` doesn't freeze the items
Ends of an array	`push`/`pop` (end), `unshift`/`shift` (start): all mutate
slice vs splice	`slice` copies a part; `splice` cuts or inserts in the original
Copy or mutate	`[...a]`, `toSorted`, `with`, `toSpliced` give new arrays; `sort`, `reverse`, `splice` mutate
Searching	`includes`, `indexOf` (−1), `find` (undefined), `findIndex`, `some`, `every`
The big three	`map` (same length), `filter` (some items), `reduce(fn, start)` (one value)
Sorting	`(a, b) => a - b` for numbers; `localeCompare` for text; copy first
Set + spread	`[...new Set(a)]` removes duplicates; `Math.max(...a)`; `[...a, ...b]`
Objects	Dot for known keys, brackets for variables; `in`, `delete`, methods and `this`
Destructuring	`const { a, b: renamed, c = 1 } = obj`; `const [x, , z, ...rest] = arr`
Spread + rest	`{ ...obj, key: v }` copy-and-change; `const { id, ...others } = obj`; `fn(...args)`
References	`b = a` shares; spread is shallow; `structuredClone` is deep; `{} !== {}`
Object ↔ array	`Object.keys / values / entries`, `Object.fromEntries`

Topic	Remember
JSON	`stringify` → text, `parse` → new object; double quotes; `try / catch` when parsing
?. and ??	Read if it exists; default only for null/undefined (keeps 0 and '')
The DOM	`querySelector(All)`, `closest`; `textContent`, `value`, `classList`, `dataset`, `hidden`
Build elements	`createElement` → fill → `append`; `remove`; `replaceChildren(...items)`
XSS	Visitor text → `textContent`, never `innerHTML`
Events	`addEventListener(type, fn)`; `target` vs `currentTarget`; `input`, `change`, `submit`, `keydown`
Bubbling + delegation	One listener on the parent + `event.target.closest(sel)`
Forms	`preventDefault()` on submit; `new FormData(form)`; HTML checks `required`, `pattern` first
Modal	`<dialog>` + `showModal()`; close on ×, Cancel, Escape, backdrop; focus back on `close`
Closures	A function + the variables it remembers: private state, `once`, `debounce`
Timers	`setTimeout` / `setInterval` run later; keep the id and `clear…` it
localStorage	Strings only (JSON); per site; ~5 MB; never secrets; always a fallback
State → render	Handlers change the data; `render()` redraws from it; restore focus
Modules	`export` / `import { x } from './x.js'`; `type="module"`; needs a server
Immutable updates	`[...a, x]`, `map` + `{ ...t, key }`, `filter`
DSA patterns	Running total; best-so-far; Set / Map "seen before"; counting letters

Checkpoint quiz

Answer each question out loud, then check the **Answers** section.

1. What is the difference between `map`, `filter` and `reduce`?
2. Why does `[10, 9, 100].sort()` give `[10, 100, 9]`, and how do you sort numbers correctly?
3. What is the difference between `slice` and `splice`?
4. How do you copy an object and change one key, without changing the original?
5. Why do you need `JSON.stringify` before saving to localStorage?
6. Why is `element.innerHTML = userInput` dangerous, and what should you use instead?
7. What is the difference between `event.target` and `event.currentTarget`?
8. What is event delegation, and why is it useful for a list?
9. What is a closure? Give an example from this week.
10. What does debouncing do, and where would you use it?

Build task: a filterable list (30 minutes, no copying)

Open `practice/filter-task.html` with Live Server. The HTML, the CSS and the data (12 resources) are ready; `filter-task-starter.js` has only the data and five TODOs. Build it with the pattern you used all week: **state**

→ work out what to show → render. Compare with `filter-task-solution.js` only when it works, or after 30 minutes.

It must…	Hint
show one card per resource	`createCard(resource)` returns an `<li class="card">`; use `textContent`
filter by the search text	lower-case both; `''.includes` is always true, so an empty box shows all
filter by level	`level === 'all' || resource.level === level`
show "6 of 12 resources"	a template literal with `visible.length` and `resources.length`
show a message when nothing matches	`empty.hidden = visible.length > 0`
update on every key and choice	`input` on the search box, `change` on the select, both call `render`

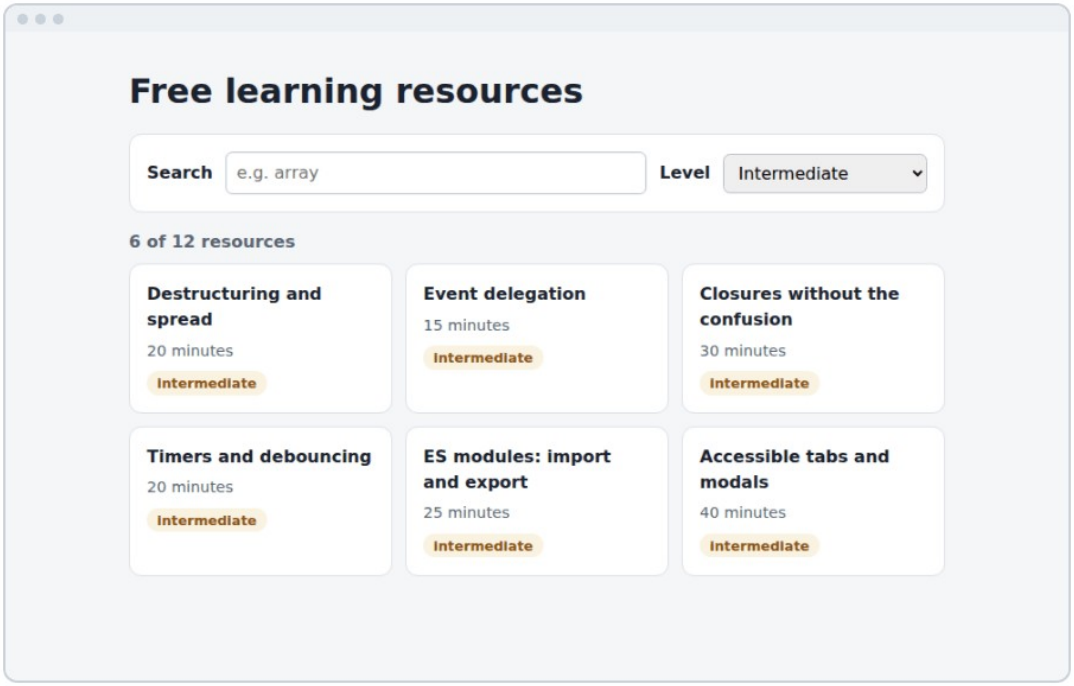


Figure 21.1 · The goal: level "Intermediate" chosen, 6 of 12 resources shown

⇄ MORE WAYS One way to build it (review/filter-task-solution.js)

WAY 1 getVisible: both conditions must pass

```
function getVisible() {
  const query = searchInput.value.trim().toLowerCase();
  const level = levelSelect.value;
  return resources.filter((resource) => {
    const matchesText = resource.title.toLowerCase().includes(query);
    const matchesLevel = level === 'all' || resource.level === level;
    return matchesText && matchesLevel;
  });
}
```

The controls **are** the state here: `getVisible` reads them every time.

WAY 2 createCard: one resource → one element

```
function createCard({ title, level, minutes }) {
  const card = document.createElement('li');
  card.className = 'card';
  const heading = document.createElement('h2');
  heading.textContent = title;
  const time = document.createElement('p');
  time.textContent = `${minutes} minutes`;
  const badge = document.createElement('span');
  badge.className = `badge ${level}`;
  badge.textContent = level;
  card.append(heading, time, badge);
  return card;
}
```

Destructuring in the parameter picks just the three keys it needs.

WAY 3 render + two listeners

```
function render() {
  const visible = getVisible();
  cards.replaceChildren(...visible.map(createCard));
  summary.textContent = `${visible.length} of ${resources.length} resources`;
  empty.hidden = visible.length > 0;
}
searchInput.addEventListener('input', render);
levelSelect.addEventListener('change', render);
render();
```

Every event simply calls `render()` again. No event changes the page directly.

TIP

Finished early? Add a third control: "Sort by" (shortest first / longest first). Copy the filtered array, sort it by `minutes`, then render. Or debounce the search box with your `debounce` from Day 19.

Redo one problem from memory

Pick the problem from this week that felt hardest and solve it again in a fresh LeetCode editor, without looking. Then say its pattern out loud: patterns are what you carry to new problems.

Problem	Pattern	The key line
1480 · Running Sum	running total (prefix sum)	`total += n; result.push(total);`
1672 · Richest Customer Wealth	best so far, over rows	`richest = Math.max(richest, wealth)`
217 · Contains Duplicate	a Set of "seen" values	`if (seen.has(n)) return true;`
242 · Valid Anagram	counting with an object / Map	`counts[letter] = (counts[letter] ?? 0) + 1`
1 · Two Sum	a Map of "seen" values	`if (seen.has(target - n)) …`
121 · Best Time to Buy and Sell	best so far + cheapest so far	`best = Math.max(best, price - cheapest)`

Plan Project 1: the Expense Tracker

From tomorrow you learn inside projects. Week 4 builds **Project 1**, an expense tracker, and almost every function it needs you wrote this week. Read the example plan in `project1-plan/README.md`, then write **your own** version: features, a wireframe and the shape of your data. Planning first is how you avoid rewriting half the app on day three.

1 · Features (version 1 must have)

- ☐ Add an expense: title, amount, category, date
- ☐ See all expenses in a list, newest first; delete one
- ☐ The total, and the total per category
- ☐ Filter by category and by month
- ☐ Everything saved in localStorage

Later (version 2): edit an expense, a bar chart per category, export to CSV, and exchange rates from a real API with `fetch` (Week 4).

2 · A wireframe

TEXT · project1-plan/README.md: laptop layout (one column on phones)

```
Expense Tracker                                  Total this month Tk 4,390
```

```
Add an expense                  [All categories ▾]  [October ▾]
Title    [          ]           05 Oct  Tea and snacks Food        Tk 240 ×
Amount   [      ]              03 Oct  Internet bill  Bills     Tk 1,200 ×
Category [Food ▾]               01 Oct  Bus pass       Transport   Tk 600 ×
Date     [10/05 ]
[Add expense]                   By category: Food Tk 1,090 · Bills …
```

3 · The data

JS · the data shape

```
// One expense
const example = {
  id: 'm2x9k1ab',            // createId(): unique
  title: 'Bus pass',
  amount: 600,               // a NUMBER in taka: convert the input with Number()
  category: 'Transport',     // one of CATEGORIES
  date: '2026-10-01',        // the <input type="date"> format: easy to sort and slice
};

// The state of the whole app
const state = {
  expenses: [],              // loaded from localStorage
  filters: { category: 'all', month: '2026-10' },
};

const CATEGORIES = ['Food', 'Transport', 'Bills', 'Learning', 'Other'];
```

You already wrote...	This week, where
`totalAmount`, `totalsByCategory`, `groupByMonth`	Day 16 · expenses practice
`formatTaka`	Day 16 · `toLocaleString('en-IN')`
`load(key, fallback)` / `save(key, value)`	Day 20 · `storage.js`
state → `render()`, delegated delete	Days 19–20 · the to-do list
filters with a lookup object	Day 20 · `FILTERS`
a form read with `FormData`	Day 18 · the modal

TIP

Keep the plan in the project's README. Tick features off as you build them, and write down every decision you make (e.g. "amounts are whole taka"). Future you, and anyone reviewing your portfolio, will thank you.

Weekly retro

What worked this week?	
What was hard?	
One thing I'll do differently in Week 4	
Hours studied this week	

Are you ready for Week 4?

Quiz score	What to do next
9–10 correct	Ready for Week 4 and Project 1. Keep the same routine.
6–8 correct	Re-read the chapters for the questions you missed, redo their "More practice", then retry the quiz.
Below 6	Redo those days before starting the project. Project 1 uses arrays of objects, the DOM, events and localStorage every day.

Coming up in Week 4

Week 4 · Asynchronous JavaScript and Project 1 (17–23 October). You'll learn how JavaScript waits without freezing (promises, `async` / `await`), load real data from the web with `fetch`, handle errors and loading states, and build and ship **Project 1: the Expense Tracker** you planned today. Plus five more LeetCode problems, starting with Valid Parentheses (a stack!).

> **TIP**
>
> Week 4 leans on Day 19 (timers and the event loop) and Day 16 (JSON). If either still feels shaky, spend 20 minutes on its "More practice" before Saturday.

DONE WHEN · tick these off, then mark the day done in your Logbook

- ☐ Quiz score of 9 or more (after a re-read if needed)
- ☐ The filterable list works, built without copying
- ☐ Your Project 1 plan is written in its README
- ☐ Widgets page live, 6 LeetCode problems solved, and all seven days marked done in your Logbook

BACK OF THE BOOK

Quick reference

Everything you typed this week, on a few pages. Keep them open while you code.

Array methods

Method	Returns	Changes the array?
`push(x)` / `unshift(x)`	the new length	**yes**
`pop()` / `shift()`	the removed item	**yes**
`splice(i, n, …add)`	the removed items	**yes**
`sort(fn)` / `reverse()`	the same array	**yes**
`slice(start, end)`	a copy of part	no
`map(fn)`	a new array, same length	no
`filter(fn)`	a new array of the items that pass	no
`reduce(fn, start)`	one value	no
`find(fn)` / `findIndex(fn)`	the first match / its index (or `undefined` / `-1`)	no
`some(fn)` / `every(fn)`	`true` / `false`	no
`includes(x)` / `indexOf(x)`	`true` / `false`; an index or `-1`	no
`join(sep)`	a string	no
`toSorted` / `toReversed` / `toSpliced` / `with`	a changed copy	no
`forEach(fn)`	`undefined` (for side effects)	no

Array recipes

JS · copy, then adapt

```
const total = items.reduce((sum, item) => sum + item.price, 0);
const names = users.map((user) => user.name);
const adults = users.filter((user) => user.age >= 18);
const user = users.find((u) => u.id === id);
const hasAdmin = users.some((u) => u.role === 'admin');
const sorted = [...numbers].sort((a, b) => a - b);        // small → big
const byName = [...users].sort((a, b) => a.name.localeCompare(b.name));
const unique = [...new Set(list)];
const biggest = Math.max(...numbers);
const lastItem = list.at(-1);
const counts = words.reduce((acc, w) => {
  acc[w] = (acc[w] ?? 0) + 1;
  return acc;
}, {});
const range = Array.from({ length: 5 }, (_, i) => i + 1);   // [1, 2, 3, 4, 5]
```

Objects

Code	Does
`obj.key` / `obj['key']` / `obj[variable]`	read a value
`obj.key = v` / `delete obj.key`	add or change / remove a key
`'key' in obj` / `Object.hasOwn(obj, 'key')`	does the key exist?
`Object.keys(obj)` / `values` / `entries`	arrays of keys / values / `[key, value]` pairs
`Object.fromEntries(pairs)`	pairs → an object
`{ title, price }`	shorthand for `{ title: title, price: price }`
`{ [name]: value }`	a computed key
`structuredClone(obj)`	a deep copy
`obj?.a?.b` / `obj.fn?.()`	read or call only if it exists
`value ?? fallback`	fallback only for `null` / `undefined`

Destructuring, spread and rest

JS · the three-dot family

```
const { name, age = 18, city: town } = user;    // pick, default, rename
const { profile: { email } } = user;            // nested
const [first, second, ...rest] = list;          // by position, rest collects
[a, b] = [b, a];                                // swap
function show({ title, amount }) { … }          // in a parameter

const copy = { ...obj };                        // shallow copy
const changed = { ...obj, done: true };         // copy and change one key
const merged = { ...defaults, ...saved };       // later keys win
const { password, ...safe } = user;             // everything except password
const joined = [...a, ...b];
function total(...numbers) { … }                // any number of arguments
```

JSON and localStorage

Code	Does
`JSON.stringify(value)`	data → a JSON string
`JSON.stringify(value, null, 2)`	the same, pretty-printed
`JSON.parse(text)`	a JSON string → new data (throws on bad text)
`localStorage.setItem(key, JSON.stringify(v))`	save
`JSON.parse(localStorage.getItem(key))`	load (`getItem` gives `null` if missing)
`localStorage.removeItem(key)` / `.clear()`	delete one / everything for this site
`try { … } catch (error) { … }`	keep going when something throws

The DOM

Code	Does
`document.querySelector('.item')`	first match, or `null`
`document.querySelectorAll('li')`	all matches (NodeList: `forEach`, or spread for `map`)
`el.closest('.card')` / `el.matches('.card')`	nearest ancestor that matches / is it a match?
`el.textContent = text`	set text safely
`input.value`	what a form field holds
`el.classList.toggle('is-done', done)`	class on if `done`, off if not
`el.dataset.id`	reads `data-id`
`el.setAttribute('aria-selected', 'true')`	set an attribute (strings)
`el.hidden = true`	hide it
`document.createElement('li')`	a new element (not on the page yet)
`parent.append(a, b)` / `prepend` / `before` / `after`	insert
`el.remove()` / `parent.replaceChildren(...items)`	remove / replace everything inside

Events

Code	Does
`el.addEventListener('click', fn)`	run `fn` on each click
`{ once: true }` / `{ capture: true }`	run once / listen on the way down
`event.target` / `event.currentTarget`	where it started / whose listener runs
`event.target.closest('.todo-item')`	the delegation pattern
`event.key`	`'Enter'`, `'Escape'`, `'ArrowRight'`, `'a'`…
`event.preventDefault()`	no link navigation / no form reload
`new FormData(form)` + `Object.fromEntries`	every named field as an object
`dialog.showModal()` / `dialog.close()`	open / close a modal `<dialog>`
`input` / `change` / `submit` / `keydown` / `focusout`	the events used this week

Patterns from this week

JS · the shapes you'll reuse in every project

```
// State → render
const state = { items: load('app.items.v1', []), filter: 'all' };
function update() { save('app.items.v1', state.items); render(); }
function render() { list.replaceChildren(...state.items.map(createItem)); }

// Delegation
list.addEventListener('click', (event) => {
  const row = event.target.closest('.item');
  if (!row) return;
  if (event.target.closest('.delete')) deleteItem(row.dataset.id);
});

// Immutable updates
state.items = [...state.items, newItem];                                    // add
state.items = state.items.map((i) => (i.id === id ? { ...i, done } : i)); // change
state.items = state.items.filter((i) => i.id !== id);                       // remove

// Debounce
function debounce(fn, ms) {
  let timerId;
  return (...args) => {
    clearTimeout(timerId);
    timerId = setTimeout(() => fn(...args), ms);
  };
}
```

This week's LeetCode problems

* Valid Anagram's counts hold at most 26 letters, however long the words are.

Problem	Best idea	Time	Extra space
1480 · Running Sum of 1d Array	running total	O(n)	O(1) in place
1672 · Richest Customer Wealth	sum rows, best so far	O(m × n)	O(1)
217 · Contains Duplicate	Set of seen values	O(n)	O(n)
242 · Valid Anagram	count letters	O(n)	O(1)*
1 · Two Sum	Map value → index	O(n)	O(n)
121 · Best Time to Buy and Sell Stock	cheapest so far	O(n)	O(1)

DevTools for this week

Where / keys	Does
Console: `$0`	the element selected in the Elements panel
Console: `copy(value)`	copies any value to the clipboard
Console: `console.table(array)`	an array of objects as a table
Elements: double-click an attribute	edit it live (test `aria-selected`, `hidden`, classes)
Elements → Event Listeners tab	which listeners an element has
Application → Local Storage	see, edit and delete saved keys
Sources: breakpoint inside a callback	pause inside `map`, `filter` or a listener
Console setting "Preserve log"	keep messages when the page reloads

Where to practise more

Resource (click to open)	Use it for
javascript.info javascript.info	Every chapter ends with small tasks and solutions
JavaScript30 javascript30.com	30 small DOM and events projects, free
Exercism: JavaScript exercism.org/tracks/javascript	Exercises with free mentor feedback
freeCodeCamp freecodecamp.org/learn	Free, step-by-step JavaScript projects in the browser
NeetCode roadmap neetcode.io/roadmap	LeetCode problems grouped by pattern (start with "Arrays & Hashing")
MDN: JavaScript Guide developer.mozilla.org/en-US/docs/Web/JavaScript/Guide	The reference to check any detail

BACK OF THE BOOK

Answers

Try every question on your own first. Reading an answer before trying teaches much less than getting it wrong and then checking.

Day 15 · Arrays

Q1. What is the difference between `slice` and `splice`?

`slice(start, end)` returns a copy of part of the array and leaves the original unchanged. `splice(start, count, …items)` removes and/or inserts items in the original, and returns the removed items.

Q2. Why does `[10, 9, 100].sort()` give `[10, 100, 9]`, and how do you fix it?

Without a compare function `sort` compares items as text, and `'1'` comes before `'9'`. Use `[...numbers].sort((a, b) => a - b)` (or `toSorted`).

Q3. You need the total price of a cart. Which method would you use, and what start value?

`reduce`, with `0` as the start value: `cart.reduce((sum, item) => sum + item.price, 0)`.

MORE PRACTICE

P1. Given `const fruits = ['mango', 'banana', 'lychee']`, write code that prints `3 fruits: mango, banana, lychee`.

`console.log(fruits.length + ' fruits: ' + fruits.join(', '));` A template literal works too. `join(', ')` puts a comma and a space between the items.

P2. Make an array of the squares of 1 to 10 without typing them.

`Array.from({ length: 10 }, (_, i) => (i + 1) ** 2)` → `[1, 4, 9, …, 100]`.

P3. From `['  js', 'CSS ', ' Html']`, make `['js', 'css', 'html']`.

`list.map((s) => s.trim().toLowerCase())`.

P4. Count how many words in `'the cat and the hat'` have 3 letters.

`text.split(' ').filter((w) => w.length === 3).length` → **4** (the, cat, the, hat).

P5. Find the longest word in `['grid', 'flexbox', 'css']`.

`words.reduce((longest, w) => (w.length > longest.length ? w : longest), '')` → `'flexbox'`. Or sort a copy by length, big first, and take `[0]`.

P6. Write `last(arr, n)` that returns the last n items, e.g. `last([1, 2, 3, 4], 2)` → `[3, 4]`.

`const last = (arr, n) => arr.slice(-n);` (watch out: `slice(-0)` is `slice(0)`, the whole array, so handle n = 0 with an `if` if you need it).

P7. Move the first item of an array to the end, without mutating it: `[1, 2, 3]` → `[2, 3, 1]`.

`const rotated = [...arr.slice(1), arr[0]];`

P8. Why does `[1, 2, 3].includes('2')` return `false`?

`includes` compares with `===`, and the string `'2'` is not the number `2`.

Day 16 · Objects + JSON

Q1. When must you use brackets instead of a dot to read a key?

When the key is in a variable (`obj[key]`), or isn't a valid name: it has spaces or dashes, or starts with a digit (`obj['favourite topic']`).

Q2. **After `const copy = { ...original }`, why can changing `copy.tags` still change `original`?**

Spread makes a **shallow** copy: a new outer object, but `tags` still points to the same array. Use `structuredClone(original)` for a deep copy, or copy the array too: `{ ...original, tags: [...original.tags] }`.

Q3. **Why do you need `JSON.stringify` before saving an object, and what does `JSON.parse` give back?**

Storage and networks only carry text, so the object must become a string. `JSON.parse` turns the string back into a **new** object with the same data (not the same object).

MORE PRACTICE

P1. **Create an object `book` with a title, an author, a year and a method `age()` that returns how many years old it is in 2026.**

`const book = { title: '…', author: '…', year: 2008, age() { return 2026 - this.year; } };` → `book.age()` is `18`.

P2. **Given `const scores = { math: 80, english: 72, physics: 91 }`, find the subject with the highest score.**

`Object.entries(scores).sort((a, b) => b[1] - a[1])[0][0]` → `'physics'`. Or loop over the entries and keep the best.

P3. **Swap the keys and values of `{ a: 1, b: 2 }` to get `{ 1: 'a', 2: 'b' }`.**

`Object.fromEntries(Object.entries(obj).map(([k, v]) => [v, k]))`.

P4. **Remove the `password` key from a user object without changing the original.**

`const { password, ...safeUser } = user;` then use `safeUser`.

P5. **Count how many expenses each category has: `{ Food: 2, Transport: 1, … }`.**

`expenses.reduce((acc, e) => { acc[e.category] = (acc[e.category] ?? 0) + 1; return acc; }, {})`.

P6. **What does `JSON.parse(JSON.stringify({ a: [1, 2] }))` give, and is it a deep copy?**

A new object `{ a: [1, 2] }` with a new inner array: yes, a deep copy, but it loses functions, `undefined` values and dates. `structuredClone` is the better tool.

P7. **Write `pick(obj, keys)` so that `pick({ a: 1, b: 2, c: 3 }, ['a', 'c'])` gives `{ a: 1, c: 3 }`.**

`const pick = (obj, keys) => Object.fromEntries(keys.map((k) => [k, obj[k]]));`

Day 17 · The DOM

Q1. **What does `querySelectorAll` return, and why can't you call `map` on it straight away?**

A NodeList: a list-like object, not a real array. It has `forEach` and `length`, but not `map` or `filter`. Spread it first: `[...nodeList].map(…)` (or `Array.from`).

Q2. **Why is `element.innerHTML = userInput` dangerous, and what should you use instead?**

The browser turns the text into real HTML, so a visitor could inject elements whose code runs on your page (XSS). Use `textContent`, which always shows text as text.

Q3. **What three things does `selectTab` change on each tab and panel?**

`aria-selected` (`"true"` only on the chosen tab), `tabIndex` (0 for the chosen tab, −1 for the others) and the panel's `hidden` (false only for the chosen tab's panel).

MORE PRACTICE

P1. **Log the text of every `<li>` on a page in one line.**

`console.log([...document.querySelectorAll('li')].map((li) => li.textContent));`

P2. **Add the class `highlight` to every even item in a list (2nd, 4th, …).**

`document.querySelectorAll('li:nth-child(even)').forEach((li) => li.classList.add('highlight'));` or check `index % 2 === 1` in `forEach`.

P3. Write `el(tag, text, className)` that creates an element in one call.

```
function el(tag, text, className = '') { const e = document.createElement(tag); e.textContent = text; e.className = className; return e; }
```

P4. Build a `<table>` from `expenses` (Day 16) with a row per expense and cells for title and amount.

For each expense: `const tr = document.createElement('tr');` then append two `td` elements with `textContent` set; append each `tr` to the `tbody`. Or map to rows and use `tbody.replaceChildren(...rows)`.

P5. Make the counter never go above 10 and disable + at 10.

In `renderCount`: `plusButton.disabled = count === 10;`, and guard the listener: `if (count < 10) count++;`.

P6. Move the first item of a list to the end with one line.

`list.append(list.firstElementChild);`: appending an element that's already on the page **moves** it.

Day 18 · Events

Q1. What is the difference between `event.target` and `event.currentTarget`?

`target` is the element where the event started (what was actually clicked). `currentTarget` is the element whose listener is running now; it changes as the event bubbles up.

Q2. What is event delegation, and why is it useful for a list?

One listener on a parent handles events from all its children, because events bubble up. It needs fewer listeners and works for items added later, which is exactly what a changing list needs.

Q3. Name three ways the modal can close, and what happens to keyboard focus afterwards.

Any three of: ×, Cancel, Escape, a click on the backdrop, submitting the form. In every case the `close` event runs `openButton.focus()`, so focus returns to "Book a free call".

MORE PRACTICE

P1. Log the x and y position of every click on the page.

```
document.addEventListener('click', (e) => console.log(e.clientX, e.clientY));
```

P2. Make a text box that shows its value in upper case in a `<p>` as you type.

```
input.addEventListener('input', () => { output.textContent = input.value.toUpperCase(); });
```

P3. Press "/" anywhere on the page to focus the search box (like GitHub and YouTube).

`document.addEventListener('keydown', (e) => { if (e.key === '/' && document.activeElement !== search) { e.preventDefault(); search.focus(); } });`: `preventDefault` stops the "/" being typed into the box.

P4. In a delegated list, make a click on the text toggle `.done`, and a click on × delete the item.

In one listener: `const item = e.target.closest('li'); if (!item) return; if (e.target.closest('.delete')) item.remove(); else item.classList.toggle('done');`: see `delegation.js`.

P5. Add a keyboard shortcut: Ctrl+Enter submits the modal's form.

On the form: `form.addEventListener('keydown', (e) => { if (e.key === 'Enter' && e.ctrlKey) form.requestSubmit(); });`. `requestSubmit()` still runs the `required` checks.

P6. Why does `-1 % 4` give `-1` in JavaScript, and how do you get 3?

JavaScript's `%` keeps the sign of the left number. Add the length first: `(-1 + 4) % 4` → `3`.

Day 19 · Closures

Q1. What is a closure? Give an example from this week.

A function that remembers the variables from the place where it was created, even after that outer function has finished. Example: `createCounter()` returns methods that share a private `count`; `debounce` keeps its `timerId` between calls.

Q2. Why do you need `JSON.stringify` before saving to localStorage?

localStorage only stores strings. `JSON.stringify` turns arrays and objects into a string; without it you save `"[object Object]"`. `JSON.parse` turns the string back into data.

Q3. What does debouncing do, and where would you use it?

It waits until calls stop for a set time, then runs once with the latest value. Use it for search boxes, autosave, username checks and window resizing.

MORE PRACTICE

P1. Write `createToggle()` returning a function that returns `true`, `false`, `true`, … on each call.

```
function createToggle() { let on = false; return () => (on = !on); }
```

P2. Write `rateLimit(fn, max)` that lets `fn` run at most `max` times, then ignores calls.

```
function rateLimit(fn, max) { let calls = 0; return (...args) => { if (calls < max) { calls++; return fn(...args); } }; }
```

P3. Print "3", "2", "1", "Go!" one second apart.

Either a `setInterval` that counts down and clears itself at 0, or three `setTimeout`s with delays 1000, 2000, 3000 (and 4000 for "Go!").

P4. Count how many times a page has been opened, and show "Visit number N".

```
const visits = Number(localStorage.getItem('visits') ?? 0) + 1;
localStorage.setItem('visits', visits);
```

P5. Add "Mark all done" to the to-do list.

`todos.forEach((t) => { t.done = true; }); render();` (or `todos = todos.map((t) => ({ ...t, done: true }))`, the Day 20 style).

P6. Solve Two Sum when the array is already sorted, without a Map.

Two pointers from both ends: sum too small → `left++`; too big → `right--` (LeetCode 167).

Day 20 · Ship it

Q1. Why does a page with `<script type="module">` fail when you double-click it?

Browsers only load modules over http(s). Opened from `file://`, the import is blocked by the CORS rules. Use Live Server locally, and Vercel online.

Q2. How do you toggle a task's `done` without mutating the array?

`state.todos = state.todos.map((t) => (t.id === id ? { ...t, done: !t.done } : t));`: a new array, with a new object only for the task that changed.

Q3. What does `FILTERS[state.filter]` give you, and why is it better than an `if / else` chain?

The test function for the chosen filter, ready to pass to `filter()`. Adding a filter is one line of data instead of another branch of logic, and the same object can be used for counts and empty messages.

MORE PRACTICE

P1. Add a fourth filter, "Today", for tasks created today.

Save `createdAt: Date.now()` on new tasks; add a `today` test to `FILTERS` that compares `toDateString()` of `createdAt` and of now; add a button and an empty message.

P2. Show "All done!" in the counter when no tasks are left.

In `render()`, choose the text with a ternary: `left === 0 ? 'All done!' : …`, with the usual "N tasks left" text in the other branch.

P3. Make "Clear done" ask "Delete 3 finished tasks?" first.

`if (!confirm('Delete ' + n + ' finished tasks?')) return;` before changing the state. (A second `<dialog>` is the nicer way.)

P4. Move the modal into a module that can open any dialog: `setupModal({ openButton, dialog, onSubmit })`.

Accept a callback `onSubmit(data)` and call it instead of writing the thank-you text inside the module.

P5. Write `move(todos, id, direction)` that moves a task up or down, immutably.

Copy with `[...todos]`, then swap the item with its neighbour: `[a[i], a[j]] = [a[j], a[i]]`.

P6. Best Time to Buy and Sell Stock II (LeetCode 122): you may trade many times. What's the idea?

Add up every rise: for each day, if `prices[i] > prices[i - 1]`, add the difference. That captures every profitable stretch.

Day 21 · Checkpoint quiz

Q1. What is the difference between `map`, `filter` and `reduce`?

`map` changes every item (same length); `filter` keeps some items (a shorter array); `reduce` combines all items into one value.

Q2. Why does `[10, 9, 100].sort()` give `[10, 100, 9]`, and how do you sort numbers correctly?

Without a compare function `sort` compares items as text, and `'1'` comes before `'9'`. Use `[...numbers].sort((a, b) => a - b)` (or `toSorted`).

Q3. What is the difference between `slice` and `splice`?

`slice(start, end)` returns a copy of part of the array and leaves it unchanged; `splice(start, count, …items)` removes and/or inserts items in the original.

Q4. How do you copy an object and change one key, without changing the original?

`const copy = { ...original, key: newValue };`: a shallow copy. Use `structuredClone` first if you will change nested data.

Q5. Why do you need `JSON.stringify` before saving to localStorage?

localStorage only stores strings. `JSON.stringify` turns arrays and objects into a string; `JSON.parse` turns it back.

Q6. Why is `element.innerHTML = userInput` dangerous, and what should you use instead?

The browser turns the text into real HTML, so a visitor could inject code that runs on your page (XSS). Use `textContent`.

Q7. What is the difference between `event.target` and `event.currentTarget`?

`target` is the element where the event started (what was actually clicked); `currentTarget` is the element whose listener is running.

Q8. What is event delegation, and why is it useful for a list?

One listener on a parent handles events from all its children, because events bubble up. It works for items added later and needs fewer listeners.

Q9. What is a closure? Give an example from this week.

A function that remembers the variables from where it was created, e.g. `createCounter()` returning functions that share a private `count`, or `debounce` keeping its `timerId`.

Q10. What does debouncing do, and where would you use it?

It waits until calls stop for a set time, then runs once. Use it for search boxes, autosave and window resizing.

BACK OF THE BOOK

Glossary

Term	Meaning
Accumulator	The running value `reduce` passes from one item to the next
aria-pressed / aria-selected	Attributes that tell screen readers a button is on / a tab is chosen
Array	An ordered list of values, indexed from 0
Bubbling	An event travelling up from its target through every ancestor
Callback	A function passed to another function, to be called by it
Capture	The first phase of an event, travelling down to the target
Closure	A function plus the variables it remembers from where it was created
Compare function	`(a, b) => …` telling `sort` which item goes first
CORS	Browser rules for loading files across origins; why modules need a server
currentTarget	The element whose event listener is running
data-* attribute	Your own attribute on an element, read with `el.dataset`
Debounce	Run a function only after calls have stopped for a set time
Deep copy	A copy that also copies every nested array and object
Default action	What the browser does on its own, e.g. following a link
Delegation	One listener on a parent handling events from its children
Destructuring	Unpacking values from objects or arrays into variables
Dialog (modal)	A box on top of the page that must be closed before going back
DOM	The page as a tree of objects that JavaScript can read and change
Empty state	The message a list shows when there is nothing in it
Entry point	The first module a page loads; it imports the others
Event loop	How JavaScript runs one task at a time from a queue
Event object	The details of an event, passed to the listener
Factory function	A function that creates and returns objects or functions
FormData	An object that reads all named fields of a form
Immutable update	Building a new array or object instead of changing the old one
Index	An item's position in an array, starting at 0
JSON	A text format for data, with double-quoted keys and strings
localStorage	A per-site key–value store of strings that survives reloads
Lookup object	An object used as a table: a key in, a value or function out
Map	A key–value collection whose keys can be any type
Method	A function stored in an object and called with a dot
Module	A file with its own scope that shares values with `export`

Term	Meaning
Mutate	Change a value in place
NodeList	The list-like result of `querySelectorAll`
Optional chaining	`?.`: stop and give `undefined` if the left side is null or undefined
Predicate	A callback that returns true or false for each item
Prefix sum	A running total of an array: each item is the sum so far
Reference	What a variable holds for an object: a link to it, not a copy
Render	Build part of the page from data
Rest	`...` collecting the remaining items, keys or arguments
Roving tabindex	Only the active item in a group can be reached with Tab
Set	A collection that holds each value only once
Shallow copy	A new outer object or array whose inner values are still shared
Spread	`...` unpacking an array's items or an object's keys
State	The data that describes what the page should show
target	The element where an event started
this	Inside a method, the object before the dot
Throttle	Run a function at most once every so many milliseconds
XSS	Cross-site scripting: visitor text running as code on your page

BACK OF THE BOOK

Notes

Questions to ask, bugs you fixed, ideas for your projects.